

ViFinanceNews – Final Project Report

Dat T. Tran, Cuong K.T Nguyen, Dung K.V Nguyen, An H. Nguyen , Duc D. Nguyen

Faculty of Mathematics and Informatics, Hanoi University of Science and Technology

CS-3332: Software Engineering

Prof. Dung T. Cao

May 26, 2025

PREFACE & INTRODUCTION

This documentation was the final & official Report of the ViFinanceNews team project that as the cousework for the course of CS-3332 (Software Engineering)

The document was include 3 main section as the following

1. The SRS Documentation (System Requirement Specification) – start from page 3 to 64
2. The SDS Documentation (System Documentation Specification) – start from page 65 to 192
3. The Software Feasibility Study Documentation – start from page 193 to 213

Due to time constraints and team limitations, this documentation may contain minor imperfections. We appreciate your understanding and hope they do not hinder your experience.

Thank you for your attention !

ViFinanceNews

Group 2 – ViFinanceNews Team

Dat Tran Tien

Nguyen Kim Tuan Cuong

Nguyen Khanh Viet Dung

Nguyen Hai An

Nguyen Duy Duc

Software Requirements Specification Documentation

Version: (1.0.0) Date: (Apr/30/2025)

TABLE OF CONTENT

1. Introduction	6
1.1 Purpose & Motivation	6
1.2 High-Level Requirement & Scope	7
1.3 Definitions, Acronyms, and Abbreviations.	8
1.4 Overview	10
2. The Overall Description	11
2.1 Product Perspective	11
2.1.1 Interfaces	12
2.1.2 Hardware Interfaces	13
2.1.3 Memory Constraints	13
2.1.4 Operations	14
2.2 Product Functions	14
2.3 User Characteristics	15
2.4 Constraints	15
2.4.1 Availability	15
2.4.2 Security	15
2.4.3 Reliability	16
2.4.4 Robustness	16
2.4.5 External Requirements	16
3. Specific Requirements	17
3.1 External Interfaces (EI)	17

Part 1: Software Requirement Specification Documentation

3.1.1 User Interface (UI) Inputs and Outputs	25
3.1.2 Accessibility Features	28
3.1.3 Data and Command Interface	29
3.1.4 Data Storage and API Responses	32
3.2 Functions	37
3.3 Performance Efficiency Requirements (PER)	40
3.4 Logical Database Requirements	41
3.4.1 Structure of Database (SD)	41
3.4.2 Relationships Constraints	45
3.4.3 Other Constrains (OC)	46
3.5 Software System Attributes	48
3.5.1 Reliability	48
3.5.2 Availability	48
3.5.3 Security	49
3.5.4 Robustness	52
3.6 Priority List of Project Quality Characteristic	53
3.7 Use-Cases	54
4. Apportioning of Requirements.	62
5. Document Contribution	63

1. Introduction

We aim to address the problem of information overload during the search process by providing a smart, holistic search tool with advanced filtering and specification capabilities. Our solution (ViFinanceNews) enhances the efficiency of information retrieval while minimizing bias and reducing the risk of misinformation.

Our system must be able to collect multiple new sources from user search queries (in the scope of this project, we would handle the field of Finance and Economy), and our system would return the analysis (both Qualitative and Quantitative in the level of bias, logical fallacies, and level of informational toxicity), and fact-checking by using inter-source comparisons.

1.1 Purpose & Motivation

The ViFinanceNews project aims to build an intelligence search tool by using a data-centric database design, combined with the use of Artificial intelligence that can support users by providing a more holistic and reduced level of bias and information overloading while searching the news and articles from the mainstream media.

Our intended users include the general user who is heavily consuming information everyday (especially from the age gaps between adolescents and elders), who can be at risk of getting misinformation and bias from the mainstream publisher.

With the scope of general users, the goal is to provide a sufficient amount of fact-checked information based on the user's inquiry and reduce users' cognitive bias by presenting a full picture derived from the user's question. Our product can serve as a supportive tool to help

users have a safer and more informative experience in cyberspace and can be utilized to educate Vietnamese people on critical analysis skills.

1.2 High-Level Requirement & Scope

This section will describe the product's high-level requirements, including the user constraints on functionality, potential application, and goal.

- **Software Product:**

- Web-Based Article Search and Interaction System.

- **High-level Requirement:**

- The users shall be able to access the search-system search for article base using their query with filtered returning results.
- The user shall only be able to interact with the articles (like adding tags and saving) and read the abstract of the article with statistical and qualitative analysis to determine whether they should invest their attention and use the article resource for personal purposes such as entertainment, education, and decision-making.

- **Application of the Software:**

- **Objective:** Enhance user experience by providing efficient article search and evaluation tools.
- **Benefits:** Saves time by delivering relevant articles with comprehensive summaries and analysis.
- **Goals:** Enable informed decision-making for users seeking content for entertainment, education, or personal use.

1.3 Definitions, Acronyms, and Abbreviations.

These are some definitions and acronyms that we may use in the project description.

- **AI** – Artificial Intelligence – The study and development of computer systems that can copy intelligent human behavior.
- **LLM** – Large Language Model – an advanced artificial intelligence model that can work with human natural language.
- **Gemini** [©] – An LLM model published by Google [©] that our team is leveraging in processing and summarizing the articles.
- **OTP** – One-Time Password.
- **ML** - Machine Learning - A type of *artificial intelligence* in which computers use huge amounts of data to learn how to do tasks rather than being programmed to do them.
- **GUI** – Graphical User Interface.
- **UI** – User Interface (can be used as a replacement for GUI).
- **str** (or) **STR** – String (Datatype represented by a sequence of characters)
- **int** (or) **INT** – Integer (Datatype)
- **OAuth2** - An open-standard authorization framework that enables third-party applications to obtain limited access to a protected resource on behalf of a resource owner, without exposing the user's credentials. It operates by issuing access tokens to clients after successful authentication and user consent.
- **API** - Application Programming Interface
- **Docker** - an open-source platform that automates the deployment, scaling, and management of applications by packaging them and their dependencies into lightweight,

portable containers. These containers ensure consistent runtime environments across development, testing, and production systems.

- **Kubernetes (or k8S)** - an open-source container orchestration platform designed to automate the deployment, scaling, and management of containerized applications. It provides tools for service discovery, load balancing, self-healing, automated rollouts and rollbacks, and resource management across clusters of hosts.
- **KONG** - an open-source API gateway and microservices management layer that provides a scalable and extensible platform for managing, securing, and monitoring API traffic. It acts as a reverse proxy and supports features such as load balancing, authentication, rate limiting, logging, and service discovery. Kong is built on top of NGINX (a software framework) and is designed to handle high-throughput, low-latency traffic across distributed systems – in this project, we use KONG as an API gateway for our microservice systems.
- **Microservice** - an architectural style that structures an application as a collection of small, loosely coupled, and independently deployable services, each responsible for a specific business capability. Microservices communicate with each other using lightweight protocols (typically HTTP/REST or messaging queues), and each service can be developed, deployed, and scaled independently. This approach enhances modularity, facilitates continuous delivery, and improves system scalability and resilience. – Our chosen architecture for this project deployment
- **S3 / Amazon S3 (Simple Storage Service)** - a scalable, high-availability object storage service provided by Amazon Web Services (AWS). It enables users to store and retrieve any amount of data at any time, from anywhere on the web. S3 is commonly used for

storing static assets, backups, media files, and large datasets. It provides features such as versioning, lifecycle management, encryption, access control policies, and durability guarantees of 99.999999999% (11 nines).

- **HTTPS** - An extension of the HTTP protocol that adds a layer of encryption using Transport Layer Security (TLS) to secure data transmitted between a client (typically a web browser) and a server. HTTPS ensures the confidentiality, integrity, and authenticity of the data by encrypting the communication channel and protecting it from eavesdropping, tampering, and man-in-the-middle attacks. It is the standard protocol for secure communication on the web.
- **SQL** – Stands for: Structured Query Language - a standardized programming language used to manage and manipulate relational databases. It enables users to define, query, insert, update, and delete data using structured commands. SQL also supports the creation and management of database schemas, access control, and transactional operations. It is widely used in both small-scale and enterprise-level applications for efficient and consistent data handling.
- **Cockroach DB** – Database platform based on Postgres SQL that our team employ

1.4 Overview

This section will provide an overall description of the documentation and the upcoming sections.

In section 2, we will provide the overall description of the product for the user of our system. In section 3, we will discuss the specific requirements and the design of the system architecture and some constraints for developers. Section 4 contains all the references and

additional information that inspired our team to build the product and the tools we used during the implementation process.

2. The Overall Description

2.1 Product Perspective

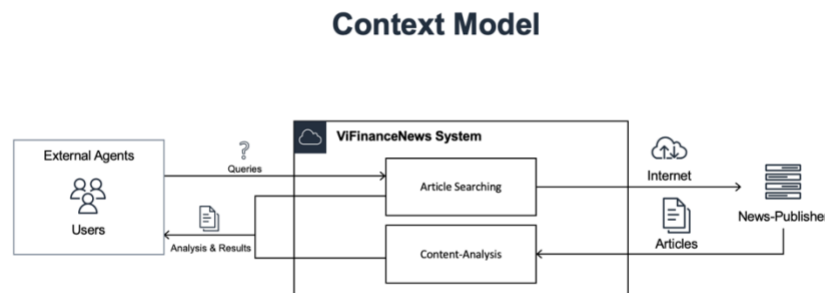
ViFinanceNews was an interactive and multi-factors web app building on the foundation of user-centric database design combined with the internet, and integrating with the current innovations in developing a Machine Learning system working with Vietnamese that is open-sourced for public usage.

Our product was inspired by the implementation of an intelligence news-reading tool such as the *Ground.News* © for the mass-media system major in the American market – dues to the complicated situation of the US politic (by applying AI for analyzing the level of bias and political orientation of the publisher to direct users toward neutral and factual information), and *Elicit* © - an intelligence search tool to support the scholar in articles selection during the researching process. Additionally, other products that our team considers as competitor, and prime-example include Google’s AI search tool function on their search engine and *ChatGPT*.

Even though, as we had mentioned above (check [Motivation section](#)), the potential of applying AI to support user information searching is fertile and being applied widely , majority of applications were not for the Vietnamese market (as *GroundNew*) or not for main-stream public readers (as *Elicit* – direct heavily for the scholarly content), even accessible to the user with the best quality (Google AI search was open only for some certain markets and for pre-registered users

– not a default system and chatGPT was offered the service but only for paid subscriptions to get the best analysis – but the result is usable).

Difference from the current implementations on the market, our ViFinanceNews search tool was built to be optimized for the Vietnamese market (by using the ML system that is customized for the Vietnamese information demographic, and the language). Also, the data source is



customized for the specified field to reduce the limitations of irrelevant and explicit contents. Furthermore, the project was open-source for reference and education to the Vietnamese community.

Figure 1: The Product Context Model of the ViFinanceNews Project

2.1.1 Interfaces

- The Application has to be defined as an interactive web app through supported browsers that users can access including Chrome, Microsoft Edge, Firefox, and Opera.
- The web app must include a Graphical User Interface (that can adjust the font-size, and color (including light, dark, and “High-contrast” themes) to adjust with the user’s visibility.
- The user shall be able to navigate through the table including:

- Homepage. (include the big search bar and the application name)
 - Search page. (the page of returned searched articles & the analysis result)
 - About-us page. (introduction on the scope and the purpose of the project)
 - Settings page. (For authenticated user has created the account)
- For disability users, the web page must include the option “Voice Over” mode for users with a deficiency in sight.
 - The web page has to include a small tab (as the inbox for the user to issue their complaints or connect to the development team)
 - For each analysis article – the user shall observe both the Qualitative and Quantitative results – both must include the annotation explaining the meaning of the metric and value of the returned result (such as “Độ tin cậy, “Mức độ độc hại,” etc.)
 - Users can interact with each system by searching, reading abstracts and synthesis, upvoting (downvoting) an article, saving (unsaved) articles to a personal account, and going to the original page through the searched article link.

2.1.2 Hardware Interfaces

- This web app has no hardware interface – it can run on multiple devices (including smartphones and desktops) – as long the user device connects to the internet and supports a web browser.

2.1.3 Memory Constraints

- The system has to be deployed on the end-user device as a web app. Ideally, the application shall be in the range of (50 to 500 MB)

2.1.4 Operations

- For operations the user can implement – check [section 2.1.1](#)
- The developer team shall be able to bring the system down on the last day of every month for maintenance (from 8:00 to 17:00), backup user data, and clean unused data.

2.2 Product Functions

- Check [section 1.2](#) for executive summarization, this session below would include what the software shall do and not do.

• What the Software Shall Do:

- Allow users to search and retrieve articles based on queries.
- Provide detailed article summaries, including quantitative and qualitative analysis.
- Enable users to save, vote, and tag articles for personal use.
- Support user authentication and profile management, including login, account creation, and logout.
- Implement security measures, such as OTP for authentication and automatic logout after 5 hours of inactivity.
- Provide navigation between multiple pages (e.g., homepage, search, settings).

• What the Software Shall Not Do:

- Shall not allow content creation or real-time article updates.
- Shall not provide article interaction with invalid user query (both in format and content).
- Shall not provide article interaction beyond tagging, saving, or voting.

- Shall not share user data for commercial purposes and will comply with privacy regulations.

2.3 User Characteristics

- Our target users would include general internet users who are:
 - Capable of understanding Vietnamese
 - Older than 13 years old
 - Able to use the Internet, or to competence to use the Internet and system after 3 hours of training)
 - The lowest level of education – secondary school education (due to the complexity of the analysis and information, need a certain level of knowledge)

2.4 Constraints

2.4.1 Availability

- The Web App shall be available from 8:00 to 17:00 every day for the user within the range of an internet connection on their desktop environment.
- System has to be closed for maintenance on the first day of every month

2.4.2 Security

- OTP (One-Time Password) is used as a second authorization method to enhance security – the OTP must be sent to the user's actual email address.
- Automatic logout improves session security and resource management.
- User data has to be encrypted to preserve privacy

2.4.3 Reliability

- The display of the generated content must include the “Disclaimer” about the potential hazard and the bias of the generated content (such as summarization, and analysis) – with observable to the user.
- Systems must not return sources without content, without the author’s name or publisher.
- The content of the analysis must exclude explicit and degenerated content – and search-tool and analysis tool would not react to these above user queries
- Annotate the level of reliability based on the metric of [\[NATO AJP-2.1 Source Reliability and Information Credibility Scales\]](#) (Rate from A1 to F6)

2.4.4 Robustness

- The system must be able to recover from the crash and able to reload to the home screen or the page that the user previously worked on.
- The system must not return results with unwanted or faulty input queries from users by displaying an error message or informing that searching with an intoxicated query is not allowed.

2.4.5 External Requirements

- The system has to give creditability to the original publisher and writer through citations and hyperlinks to the original publication without distortion and misinterpretation as follows in [section 25 of the Intellectual Properties Law - 36/2009/QH12 from the Vietnamese National Assembly](#)

- User data maintenance and usage must be protected and not for commercial usage and not disclosed under any circumstance. belong to the user, and be determined by the user – no in-authenticated agent (*except the ViFinancesNews team for developmental product purposes*) – this must follow the [21st section in the Constitution of the Socialist Republic of Vietnam \(2013\)](#) about Civilian Privacy Rights, and [Laws of Data - 60/2024/QH15 \(2024\)](#)

3. Specific Requirements

3.1 External Interfaces (EI)

The upcoming table is a **table summarizing the UI Inputs and Outputs** for Section 3.1 based our description.

ID	Name	Input/Description	Source / Destination	Valid Range / Units	Timing	Screen Format	Data Format (JSON/Other)	Command Format	End Message
EI-01	Search	User inputs	Homepage	1–200	≤ 2 sec	Text input, responsive	UTF-8 string	POST	Error:
	Article	search term	search bar	characters, Text String				/get_cached_result	Empty or invalid query
EI-02	Synthesis	List of article url	Search	≤ 10	≤ 3 sec +	List view, pagination, sortable	JSON (title, summary, tags, score)	POST	"Invalid input "
	Service	(meta-data)	results panel	articles, Summary < 50 – 100 words				/synthesis	

ID	Name	Input/Description	Source / Destination	Valid Range / Units	Timing	Screen Format	Data Format (JSON/Other)	Command Format	End Message
EI-03	Article Analysis	URL of the interested article	Specific Article Panel	1 url of the article - under 2000 characters	10 to 30 seconds is feasible level	Json-data object	Json (the analysis & meta-data of the analysis-result)	POST	
								/toxicity_analysis	
								POST	{Err
								/sentiment_analysis	message:- str,
								POST	err-code: int"
								/factcheck	
								POST	
								/summary	

ID	Name	Input/Description	Source / Destination	Valid Range / Units	Timing	Screen Format	Data Format (JSON/Other)	Command Format	End Message
EI-04	Voice Over	Toggles audio narration for accessibility	Settings Page	On/Off, Boolean	Immediate (1-5ms)	Toggle in Accessibility Settings	{"voice_over": true/false}	POST /settings	“Voice-Over đã kích hoạt/đã tắt”
EI-05	User Authentication	Logs user in with email, password, OTP, OAuth2 Login	Login form	Email format, 8–32 char pwd, 6-digit OTP	≤ 1 sec	Email/pass word fields + OTP pop-up	{"email": "", "password": "", "OTP": ""}	POST /login (or Simmilar)	Success / Invalid login / OTP / Locked

ID	Name	Input/Description	Source / Destination	Valid Range / Units	Timing	Screen Format	Data Format (JSON/Other)	Command Format	End Message
EI-06	Log-out	Logs user out, clears session	Settings Page / Nav menu	Authenticated users only	Immediate (1-5 ms)	“Đăng xuất” button	-	POST /logout	“Bạn đã đăng xuất thành công.”
EI-07 (*)	Feedback Submission	Sends complaints/suggestions	Feedback bubble (homepage corner)	5–500 characters, Text String	Instant DB store (1-5 ms)	Text input + “Gửi góp ý” button	{"user_id": "", "message": ""}	POST /feedback	“Cảm ơn vì góp ý của bạn”

ID	Name	Input/Description	Source / Destination	Valid Range / Units	Timing	Screen Format	Data Format (JSON/Other)	Command Format	End Message
EI-08	Save Article	Save article to user list	Search Page “Luu” button	≤ 50 articles per user	Immediate DB update (1-5 ms)	List with “Gỡ bài viết” and vote buttons	{"user_id": str, "saved_articles": str" }	POST /save_article	“Luu bài viết thành công”
EI-09 (*)	Remove Saved Article	Remove article from saved list	Saved Articles → click “Gỡ bài viết”	Article must be in saved list	Immediate DB update (1-5 ms)	Same as above	{"user_id": "", "article_id": ""}	DELETE /remove_saved_ar ticle	“Gỡ bài viết thành công.”

ID	Name	Input/Description	Source / Destination	Valid Range / Units	Timing	Screen Format	Data Format (JSON/Other)	Command Format	End Message
EI-10	Upvote/Downvote Article	User votes on an article	Search / Article View Page	1 vote per article, editable	Immediate DB update (1-5 ms)	▲ / ▼ buttons, color-coded	{"user_id": "", "article_id": "", "vote": "upvote/downvote"}	POST /vote_article	“Vote của bạn đã được lưu.”
									Message confirming soft deletion (30-day grace period)
EI-11	Delete Account	Permanently delete user account	Settings Page “Xóa tài khoản”	Authenticated + OTP/password	Immediate upon confirm (1-5 ms)	Dialog box with warning	{"user_id": ""}	DELETE /api/user/delete	

ID	Name	Input/Description	Source / Destination	Valid Range / Units	Timing	Screen Format	Data Format (JSON/Other)	Command Format	End Message
EI-12	Login Using Google Account	Login using Google OAuth2 token	The “login with Google Button” in Login Page	OAuth2 token	Immediate upon confirm Using Google API	Google’s API windows	GoogleAPI token object	-	-

(-) This implies that our team has no constraints on this functionality

(*) The function we can delay on the later version – check section 4 (Apportioning Requirement)

3.1.1 User Interface (UI) Inputs and Outputs

EI-01. Search Article

- Descriptions: To retrieve relevant articles, users can enter a search term in the homepage search bar (or on the top of the page header).
- Source: User input via the homepage search bar
- Valid Range:
 - Minimum: 1 characters
 - Maximum: 200 characters
- Unit of Measures: Text String
- Timing: Real-time response expected in the worst case no more than 2 seconds
- Relationships: Triggers search results displayed on the **Search Page**
- Screen Format: Text input field, responsive to font size and color settings
- Data Format: UTF-8 encoded string (support for Vietnamese tonal characters)
- Command Format: POST /get_cached_result
- End Message: Error message if the query is invalid (e.g., “Yêu cầu tìm kiếm không được bỏ trống”, or “Không thể tìm kiếm do yêu cầu tìm kiếm vi phạm yêu cầu về Điều Khoản về trách nhiệm của người dùng”)

EI-02. Synthesis Service

- Description: Return a synthesis summarization of the articles that have been retrieved to provide a general answer to a user query.
- Destination: Search results panel on the Search Page.
- Valid Range: Returns up to at most 10 articles per request, ranked by relevance and semantic similarity to the user's query. Additionally – a synthesis summarization of the retrieved article.
- Units of Measure: List of article objects (title, abstract, link, tags, scores, published date, author).
- Timing: Display within 3 seconds after search submission – and shall be updated in the cache after return.
- Relationships: Links to detailed analysis page for each article.
- Screen Format: List view with pagination and sorting options. Each article shall be numbered based on their order in the result list)
- Data Format: JSON response (title, summary (under 50 words), tags, rating score).
- Command Format:

"POST/synthesis"
- End Message: "No results found" message if no articles match the query.

EI-03. Article Analysis

- **Description:** Displays the Analysis for one Article being selected from the Search Results List.

- **Destination:** When the user clicks on a search-result page, it opens a pop-up to the central point of their screen as a fixed pop-up window– the background surrounding has to be shadowed to enhance focus.

- **Units of Measure:**

A dedicated sub-page for each article displays essential details, including the article title, metadata (author name, publication date, and tags), a save button, and an upvote button, direct-link to publisher button.

The sub-page contains three tabs: (able to navigate horizontally)

1. **Summarization (Abstract):**

- A concise summary of the article, limited to 150 words.
- A list of tags to capture the keyword of the article
- A filling box for the user to add their own label

2. **Quantitative Analysis:**

- Data visualization using a radar chart to represent semantic similarity levels.
- Reliability and bias analysis scores, with tooltips providing metric explanations when hovered over.

3. **Qualitative Analysis:**

- A brief (max 150 words) explanation of potential bias in the article.

- A reliability tag, as referenced in [Section 2.4.3.](#)
- Timing: Display under 1 second since the user clicks on the result
- Relationships: Subpage of the search result page.
- Screen Format: Pop-windows windows on the right side of the user screen contain three tabs as described above – screen ratio must follow the ratio (4:3) - landscape
- Data Format:

JSON response shall includes: (Data object can be separated to smaller batches)

- title
- Tags
- Author
- publication-date
- articleURL
- End Message:
 - “Không thể tìm kiếm kết quả” message if the analysis result is not available
 - “Tìm kiếm thành công” message if the result return success

3.1.2 Accessibility Features

EI-04. Voice Over Mode Activation

- Description: Enables audio narration of page content for visually impaired users.
- Source: User toggles setting in the Settings Page
- Valid Range: On/Off
- Units of Measure: Boolean
- Timing: Immediate activation
- Relationships: Affects all UI elements across the web app
- Screen Format: Toggle button within Accessibility Settings
- Data Format: JSON config update ("voice_over": true/false)
- Command Format: "POST /settings" {"voice_over": true}
- End Message: Confirmation message: “Chế độ Voice-Over đã kích hoạt/ đã tắt”

3.1.3 Data and Command Interface

EI-05. User Authentication Input

- Description: Allows registered users to log in and access personalized features (e.g., saved articles, settings).
- Source: Login form on the Settings Page
- Valid Range:
 - Email: Valid email format

- Password: 8-32 characters, alphanumeric with special characters
- OTP: 6-digit values must be sent to the user's mailbox – has is valid in 30 seconds
- Units of Measure: Email/password/OTP
- Timing: Response within 1 second after submission
- Relationships:
 - If successful: Redirects to Settings Page
 - If unsuccessful: Displays login error message
 - If unsuccessful after at most 5 times: Displays account has been locked for security reason
- Screen Format: Two input fields with a “Đăng nhập” button and a pop-up window asking for the user's OTP code
- Data Format:
 - JSON object ({“email”: “<user_email>”, “password”: “<hashed_password>”, “OTP”: “<OTP>”})
- Command Format: "POST /login"
- End Message:
 - “Đăng nhập thành công” or “Email/mật khẩu không hợp lệ” or “Mã OTP không hợp lệ”

EI-06. Log-out

- Description: Logs the user out of their account, ending the session and clearing authentication tokens.

- Source: User action on the Settings Page (clicking “Đăng xuất”) – or the user can select the option “Log-out” on the navigation option on the header (symbol ≡).
- Valid Range: Only authenticated users can log out.
- Units of Measure: User session token.
- Timing: Immediate session termination upon request.
- Relationships:
- If successful: Redirects the user to the Homepage.
- If unsuccessful: Displays an error message.
- Screen Format: “Đăng xuất” button in the Settings Page.
- Data Format: JSON object (`{"user_id": "<UUID>", "session_token": "<token>"}`).
- Command Format: "POST /logout"
- End Message: "Bạn đã đăng nhập thành công."

EI-07. User Feedback Submission

- Description: Allows users to submit complaints or suggestions via the feedback bubble (on the right-conner of the homepage)
- Source: User input in the feedback form.
- Valid Range:
- Minimum: 5 characters

- Maximum: 500 characters
- Units of Measure: Text string
- Timing: Stored instantly in the database
- Relationships: Viewable by the development team through an admin dashboard
- Screen Format: Text input field with a “Gửi góp ý” button
- Data Format: JSON object (`{"user_id": "<UUID>", "message": "<feedback_text>"}`)
- Command Format: "POST /feedback"
- End Message: “Cảm ơn vì góp ý của bạn”

3.1.4 Data Storage and API Responses

EI-08. Saved Articles

- Description: Stores user-selected articles for later retrieval.
- Source: User action on the Search Page (clicking “Luu”)
- Valid Range: Up to 50 saved articles per user
- Units of Measure: List of article IDs
- Timing: Immediate database update
- Relationships: Appears in Saved Articles Section in Settings Page

- Screen Format: List view with “Gỡ bài viết” and “Upvote” buttons (the Upvote button green means upvote, and red means downvote; else neutral – set with black-and-white)
- Data Format: JSON object (`{"user_id": "<UUID>", "saved_articles": ["<article_id_1>", "<article_id_2>"]}`)
- Command Format: `"POST /save_article"`
- End Message: “Lưu bài viết thành công”

EI-09. Remove Saved Article

- Description: Removes a user-selected article from their saved list.
- Source: User action on the Saved Articles section (clicking “Remove”).
- Valid Range: The article must exist in the user’s saved list.
- Units of Measure: Article ID. (not displayed to the user - only stored in the database)
- Timing: Immediate database update.
- Relationships: Updates the Saved Articles section in the Settings Page.
- Screen Format: List view with the “Gỡ bài viết” and “Upvote” buttons next to each article.
- Data Format: JSON object (`{"user_id": "<UUID>", "article_id": "<article_id>"}`).
- Command Format: `"DELETE /remove_saved_article"`

- End Message: "Gỡ bài viết thành công."

EI-10. Upvote or Downvote Article

- Description: Allows users to express approval (upvote) or disapproval (downvote) of an article – user vote can change in the saved list.
- Source: User action on the Search Page or Article View Page by clicking the “Upvote” button.
- Valid Range: Each user can upvote or downvote an article (modifying is allowed) in the search page or article-saved page in the “Cài đặt.”
- Units of Measure: Article ID and vote type (upvote or downvote).
- Timing: Immediate database update.
- Relationships: Updates the article’s vote count and user history.
- Screen Format: Upvote (▲) and Downvote (▼) buttons with vote count display.
- Data Format: JSON object ({"user_id": "<UUID>", "article_id": "<article_id>", "vote": "upvote"} or {"vote": "downvote"}).
- Command Format: "POST /vote_article"
- End Message: "Vote của bạn đã được lưu."

EI-11. Delete Account

- Description: Permanently removes a user's account and all associated data.
- Source: User action on the Settings Page (clicking "Xóa tài khoản").
- Valid Range: Only authenticated users can request account deletion.
- Units of Measure: User ID (UUID).
- Timing: Immediate account removal upon confirmation (by asking the user for the password and sending OTP as login process).
- Relationships: Removes all user-related data, including saved articles and preferences.
- Screen Format: Confirmation dialog with a warning message and a "Xóa" button.
- Data Format: JSON object (`{"user_id": "<UUID>"}`).
- Command Format: `"DELETE /user/account"`
- End Message: Message confirms soft-deletion (30-day grace period)

EI-12. Login Using Google Account

- Description: Login using Google OAuth2 token
- Source: The “Login with Google” button in Login Page
- Valid Range: OAuth2 token
- Units of Measure: OAuth2 token
- Timing: Immediate upon confirmation (using Google API)
- Relationships: Navigate the user to log in with Google Windows
- Screen Format: open news Google Windows (can be login using Gmail)
- Data Format: GoogleAPI token object
- Command Format: NONE
- End Message: NONE

3.2 Functions

Below is the summarised table of this project's functional requirements (More functions we will provide in the next version of our product)

Functional Requirements Table (FR)

ID	Feature	Description
FR-01	Guest & Registered User Access	Users can access the system as guests or registered users. They can log in, build profiles, and log out manually.
FR-02	Automatic Logout	The system shall automatically log out users after 1 hours of inactivity.
FR-03	User Profile Management	Registered users can change their profile name, avatar, and delete their account at will.
FR-04	User Authentication	Users can log in via email and verify identity using OTP as a second authentication method.
FR-05	Article Retrieval & Presentation	Users can retrieve up to 10 articles per query. Results include title, full content, summary, image, URL, and qualitative/quantitative analysis.
FR-06	Article Interaction	Users can upvote/downvote articles, save them to their profile/database, and add tags.

FR-07	Page Navigation	Users can navigate between pages including Homepage, Search, About-us, and Settings.
-------	-----------------	--

FR-01. Guest & Registered User Access

- The users can use the system in guest mode (default model without login).
- Users can build their profile by creating an account (login).
- Users can log out manually from the application.

FR-02. Automatic Logout

- The system shall log out the user automatically if the user is inactive for **1 hours**.

FR-03. User Profile Management

- With the personal profile, users can:
 - *Change their profile name.*
 - *Change their avatar.*
 - *Delete their account at will.*

FR-04. User Authentication

- The system shall authenticate users, including:
 - *Email login process.*
 - *Using OTP (One-Time Password) as a second authorization method.*

FR-05. Article Retrieval & Presentation

- The user shall be able to retrieve **at most 10 articles** based on their query.
- Each query result shall include:
 - *Title.*
 - *Full article content.*
 - *Summary (abstract) of the article content.*
 - *Image.*
 - *URL (or button directing to the original publisher).*
 - *Quantitative and qualitative analysis of the article content.*

FR-06. Article Interaction

- Users can:
 - *Upvote/Downvote articles.*
 - *Save articles to their profile/database.*
 - *Add tags to articles.*

FR-07. Page Navigation:

- Users shall be able to navigate between pages:
 - *Homepage.*
 - *Search page.*
 - *About-us page.*
 - *Settings page.*

3.3 Performance Efficiency Requirements (PER)

PER-01. Application shall be under 10GB (ideal is with 500MB to 1 GB)

PER-02. Response time for the tab navigation has to be within 1 to 2 seconds
(ideal can be in milliseconds)

PER-03. Response time for the summary function shall be under 5 seconds, while the search tool has to return the content (the meta-data) within 2 to 3 seconds, and the content analysis results for one searched article have to return under 5 seconds.

PER-04. Total analysis run-time of all search results in the worst-case must be under a minute (ideals is within 20 to 30 seconds for at most 10 search results).

3.4 Logical Database Requirements

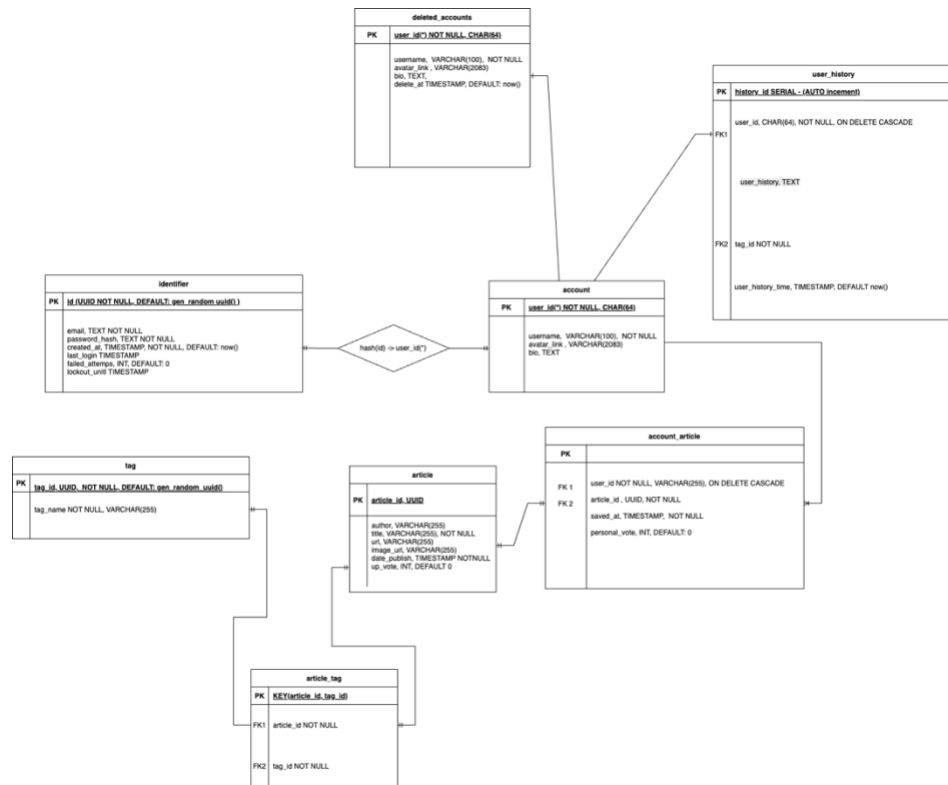


Figure 2: The Entity–Relationship diagram describes the logic of the VifinanceNews database system

3.4.1 Structure of Database (SD)

SD-01. identifier Table

- a. The `identifier` table shall have the following attributes:
 - i. `id` as the primary key, which shall not be null.
 - ii. `email`, which shall not be null.
 - iii. `password_hash`, which shall not be null.
 - iv. `created_at`, which shall not be null.

- v. `last_login`, which shall not be null and shall be of type DATE-TIME.
 - vi. `failed_attempts`, which shall not be null and shall be of type INT.
 - vii. `lockout_until`, which shall be of type DATE-TIME.
- b. The `lockout_until` attribute should be nullable to allow for accounts that are not locked out.

SD-02. account Table

- a. The `account` table shall have the following attributes:
- i. `user_id`, which shall not be null, hashed from `id` (the identifier table).
 - ii. `user_name`, which shall not be null.
 - iii. `bio_description`, for user biography (maximum 255 character)
- b. The `bio_description` should allow for a reasonable character limit to accommodate detailed user profiles.

SD-03. article Table

- a. The `article` table shall have the following attributes:
- i. `article_id` as the primary key, which shall not be null.
 - ii. `author`, which shall not be null.
 - iii. `title`, which shall not be null.
 - iv. `url`, which shall not be null.
 - v. `image_uri`, which shall not be null.
 - vi. `date_publish`, which shall not be null.
 - vii. `Up_vote`, which shall not be null.

- b. The `up_vote` attributes should be initialized to 0 for new articles.

SD-04. tag Table

- a. The `tag` table shall have the following attributes:
 - i. `tag_id` as the primary key, which shall not be null.
 - ii. `tag_name`, which shall not be null.
- b. The `tag_name` should be unique to prevent duplicate tags.

SD-05. article_tag Table

- a. The `article_tag` table shall have the following attributes:
 - i. `article_id` as a foreign key referencing `article(article_id)`, which shall not be null.
 - ii. `tag_id` as a foreign key referencing `tag(tag_id)`, which shall not be null.
 - iii. The combination of `article_id` and `tag_id` shall form a composite primary key.
- b. The `article_tag` table should prevent duplicate associations between the same article and tag.

SD-06. account_article Table

- a. The `account_article` table shall have the following attributes:
 - i. `id` as a foreign key referencing `account(account_id)`, which shall not be null.
 - ii. `article_id` as a foreign key referencing `article(article_id)`, which shall not be null.
 - iii. `saved_at`, which shall not be null and shall be of type DATE-TIME.

- iv. The combination of `user_id` and `article_id` shall form a composite primary key.
- b. The `saved_at` attribute should automatically record the current timestamp when a user saves an article.

SD-07. delete_account Table

- a. The `delete_account` Table shall have these attribute:
 - i. `user_id` – NOT NULL (as primary key)
 - ii. `username` – VARCHAR(100), NOT NULL
 - iii. `avatar_link` – VARCHAR(2083)
 - iv. `bio` – TEXT
 - v. `delete_at` – TIMESTAMP (DATETIME), default is the “now()” = the time user delete their account

SD-08. user_history Table

- a. The `user_history` Table shall have these attributes:
 - i. `history_id` – SERIAL (integer) – auto increment after each insertion
 - ii. `user_history` - TEXT - user data is encrypted by AES (Advanced Encryption Standard) algorithm
 - iii. `username` – VARCHAR(100), NOT NULL
 - iv. `avatar_link` – VARCHAR(2083)
 - v. `bio` – TEXT
 - vi. `delete_at` – TIMESTAMP (DATETIME), default is the “now()” = the time user delete their account

SD-09. Redis cache

- a. Redis Cache shall contain all of the short-term data for the user, including the result of the article_scraping process (as the article table and tag tables), and the voting selection of the user for one article.
- b. The Data in the Redis cache shall exist for 1 hour – so backing up and uploading to the SQL is essential for later retrieval.
- c. Redis cache only pushes the data to SQL after the user navigates out from the session (sign out, exit the personal setting console).

3.4.2 Relationships Constraints

RC-01. identifier to account

- a. The identifier table shall be linked to the account table through a one-to-one relationship, where the id in dbo_identifier shall map to the user_id in the account via a hash function (hash(id) -> account_id).
- b. The hash function should be secure and consistent to ensure accurate mapping between identifier and account.

RC-02. account to account_article

- a. The account table shall have a one-to-many relationship with the account_article table, where one account _id in the account shall be associated with multiple entries in account_article.
- b. The system should allow users to save an article only once to avoid duplicate entries in account_article.

RC-03. article to account_article

- a. The article table shall have a one-to-many relationship with the account_article table, where one article_id in the article shall be associated with multiple entries in account_article.

- b. The system should provide a mechanism to track the number of users who have saved an article.

RC-04. article to article_tag

- a. The `article` table shall have a one-to-many relationship with the `article_tag` table, where one `article_id` in the `article` shall be associated with multiple entries in the `article_tag`.
- b. The system should allow an article to have at least one tag for better categorization.

RC-05. tag to article_tag

- a. The `tag` table shall have a one-to-many relationship with the `article_tag` table, where one `tag_id` in `tag` shall be associated with multiple entries in `article_tag`.
- b. The system should provide a way to retrieve all articles associated with a specific tag efficiently.

RC-06. account to article

- a. The `author` attribute in the `article` table shall reference the `user_id` in the `account` table, establishing a one-to-many relationship where one account can author multiple articles.
- b. The system should validate that the `author` exists in the `account` table before an article is created.

3.4.3 Other Constrains (OC)

OC-01. Primary Key Constraints

- a. Each table shall have a primary key:
 - i. `identifier` shall use `id`.

- ii. `account` shall use `account_id`.
 - iii. The `article` shall use `article_id`.
 - iv. `tag` shall use `tag_id`.
 - v. `article_tag` shall use a composite key of (`article_id`,
`tag_id`).
 - vi. `account_article` shall use a composite key of (`user_id`,
`article_id`).
- b. The primary keys should be indexed for efficient querying.

OC-02. Foreign Key Constraints

- a. Foreign keys shall be enforced as follows:
- i. In `account_article`, `user_id` shall reference `account(user_id)`, and `article_id` shall reference `article(article_id)`.
 - ii. In `article_tag`, `article_id` shall reference `article(article_id)`, and `tag_id` shall reference `tag(tag_id)`.
- b. The system should cascade deletes or updates on foreign keys to maintain referential integrity (e.g., if an article is deleted, its entries in `article_tag` and `account_article` should also be removed).

OC-03. NOT NULL Constraints

- a. All attributes in all tables shall be NOT-NULL except for `lockout_until` in the `identifier`, which may be nullable.
- b. The system should provide default values for attributes like `total_up_vote` in the `article` to ensure they are never null.

OC-04. Other Constraints

- a. The `failed_attempts` attribute in the `identifier` shall be of type `INT`, and date-related attributes (`last_login`, `lockout_until`, `created_at`, `saved_at`, `date_publish`) shall be of type `DATE-TIME`.
- b. The `tag_name` in the `tag` table should be unique to avoid redundancy in tagging.

3.5 Software System Attributes

3.5.1 Reliability

- Error Rate: The system should maintain an error rate ideally of $\leq 1\%$ for API requests, however, the acceptable level of failure in worse-case = **10%** for an API request

3.5.2 Availability

- The user Database has to be updated on CockroachDB (with time & type of backup)
- The System has to be deployed on Kubernetes using Docker-containerization technical stack to ease maintenance and self-healing mechanism (restart with high-traffic – about 1 to 5 ms restart is ideal)
- The System shall update the user database on Cockroach DB in real-time of operation – a backup plan can be updated in the forthcoming deployment of the project.
- The system shall be available to handle requests from 08:00 to 17:00 every day, with occasional downtime permitted only for maintenance and feature updates on the first day of each month (the work-spans of the application can be extended in upcoming updates with actual testing with more user involvement).
- The system shall notify users before any scheduled maintenance or downtime, ensuring minimal disruption to service availability (suggest using email as a means for notification).

3.5.3 Security

1. Access Control & Authentication

a. Authentication Mechanisms

The system shall implement multi-factor authentication (MFA) for enhanced security.

The authentication flow consists of:

- **Username/Password Authentication:** Users must provide valid credentials, which are securely hashed before storage. (The Credential can be either from the third-party platform (OAuth2) as Google, or using a password, and username)
- **One-Time Password (OTP) Verification:** After successful credential validation, users receive an OTP via email (Gmail SMTP integration), which is required for login.
- **Session-Based Authentication:** The system uses Java EE HttpSession for session management, ensuring session expiration and renewal policies.

b. API Endpoint Protection.

- **Request Validation:** API requests are validated against the session-based authentication mechanism.
- **User-specific Operations:** User-specific operations require session-based user verification.
- **Implement the Kong-API gateways & Kubernetes Configuration:** To handle the unambiguous clusters of the API structure – to reduce the potential of API leaking while maintaining robustness.

c. Session Management

- **Session Timeout:** Sessions expire after 30 minutes of inactivity.
- **Session Caching:** Session IDs are hashed and stored in a NoSQL database, mapped to encrypted user information. This allows for faster retrieval while protecting user identity from potential attacks. The user information in the cache must be encrypted to ensure security.
- **Token Expiry:** OTPs are stored in **Upstash Redis** and expire after 1 minute, or after successful verification from the user immediately.
- **Session Termination:** Logout immediately invalidates the session.

2. Cryptography & Data Protection

a. Encryption Algorithms

- **Data in Transit:** All communications use **TLS 1.3** encryption. (Using the HTTPS data-encryption protocol)

b. Password & Data Security

- **Password Hashing:** Uses **bcrypt** for strong password hashing.

- **UUID Hashing:** The userID is stored in **SHA-256 hash format** in the account table, preventing direct ID exposure.
- **OTP Handling:** OTPs are stored temporarily in **UpStash Redis**, ensuring no database persistence.

c. Secure Authorization

- *Users must pass **two-step verification** before accessing sensitive operations.*

3. Logging & Monitoring

a. Logging Mechanisms

The system logs all critical activities, including:

- **Authentication logs:** Successful and failed login attempts.
- **Access logs:** API requests and responses with masked identifiers.
- **Security logs:** Unauthorized access attempts and modifications.
- **Error logs:** System errors and failures.

b. Intrusion Detection & Monitoring

- **Real-time alerts** notify admins about suspicious login attempts.
- **Failed login tracking** helps detect brute-force attempts.

4. Data Integrity & Communication Restriction

a. Data Integrity Checks

- Data integrity is verified through operations and validations directly in CockroachDB.

b. Restricted Communications

- Database access is restricted to the backend services only, with no direct external access.

3.5.4 Robustness

- The system shall return a failure message if the user passes an invalid message exceeding 200 characters in the search tool or 250 characters for the authentication console (both username and password).
- Avoid exposing system errors (such as SQL errors or backend error messages); instead, display a meaningful message.
- Remove or escape special characters in the search query and login data to prevent SQL injection.
- The system shall return a black screen or blank screen, displaying a failure message as "We're experiencing issues. Please try again later." in the case of high traffic.
- If someone sends over 100 login attempts per second (DDoS or brute-force attack), limit the rate by displaying "Too many attempts. Try again in 5 minutes."

3.6 Priority List of Project Quality Characteristic

ID	Characteristic	H/M/L	1	2	3	4	5	6	7	8	9	10
1	Performance Efficiency	H								x		
2	Reliability	H							x			
3	Security	M								x		
4	Availability	M							x			
5	Robustness	H									x	
6	Usability	M									x	

Definitions of the quality characteristics in the table are as follows:

- Performance Efficiency – the level of resource consumption and responsiveness of the software to the user
- Reliability – the extent to which the program performs with required precision & and returns trustable results (without bias, logical fallacies, or error format)
- Availability – the extent the application is ready for user consumption (even within normal conditions and extreme conditions such as high-traffic or user encounter error messages)
- Robustness – The capability the software can self-healing after getting an error or unwanted event
- Usability – the effort required to learn, operate, prepare input, and interpret the output

- Essential tier for each criterion as follows with the next-by column, considers the numerical representation of priority of the attribute
 - H: High (8 to 10)
 - M: Medium (5 to 7)
 - L: Low (1 to 4)

3.7 Use-Cases

This section will describe the use case for our system (provide both in Diagram, and tabular description)

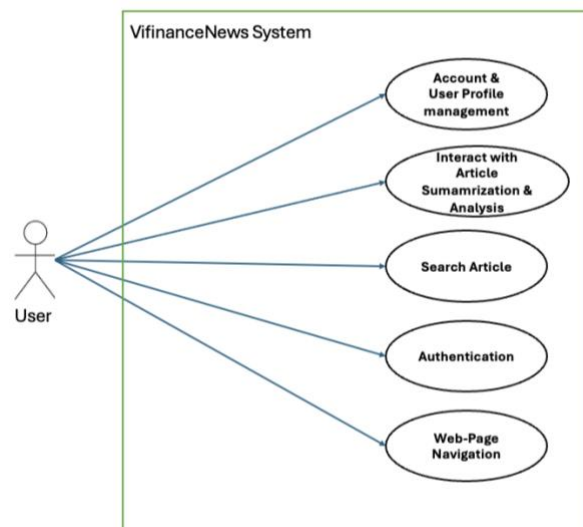


Figure 3: The General Use-Case Diagram of VifinanceNews System

Users shall be able to interact with the VifinanceNews System as an internet web-based system with basic functionality such as searching, reading the analysis of the article – and creating their personal profile to personalize user experience.

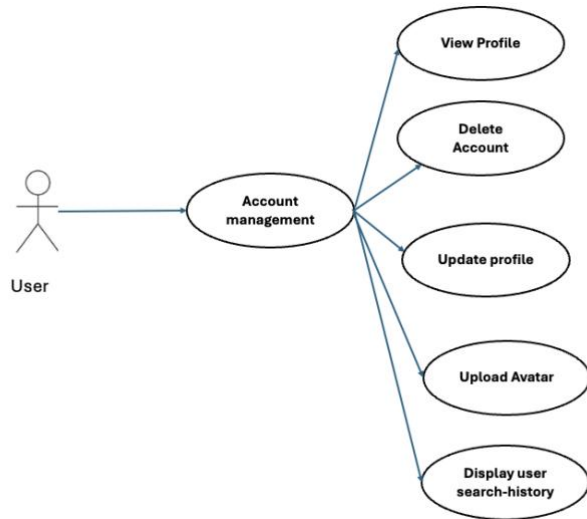


Figure 4: The UC:01 Use-Case Diagram

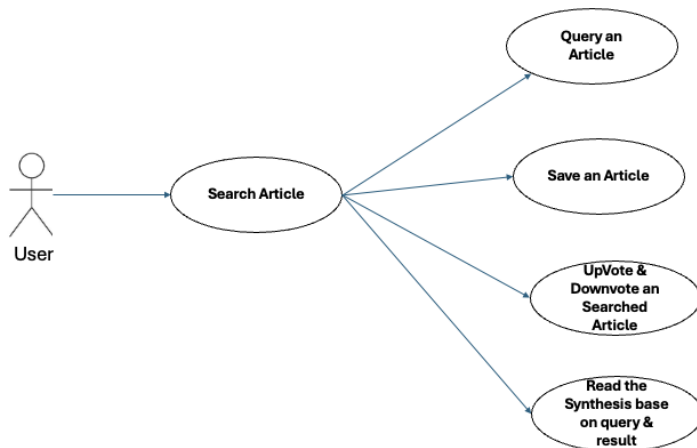


Figure 5: The UC:02 Use-Case Diagram

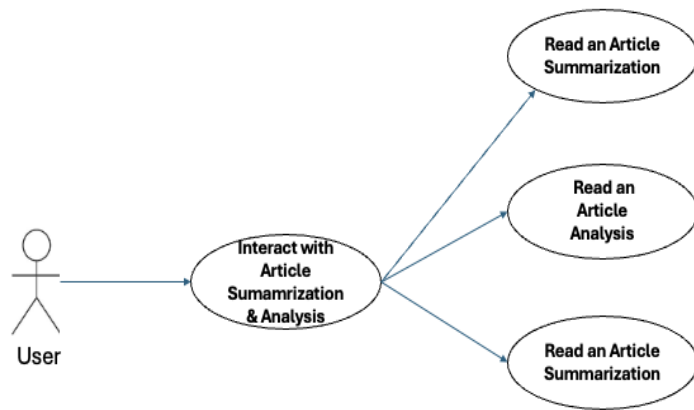


Figure 6: The UC:03 Use-Case Diagram

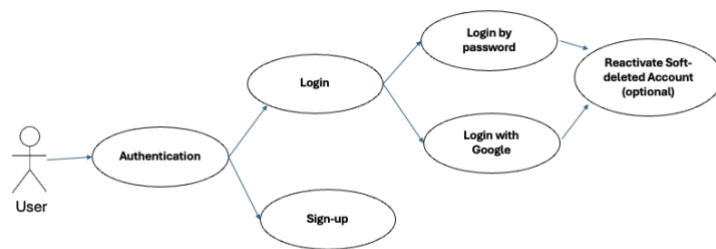


Figure 7: The UC:04 Use-Case Diagram

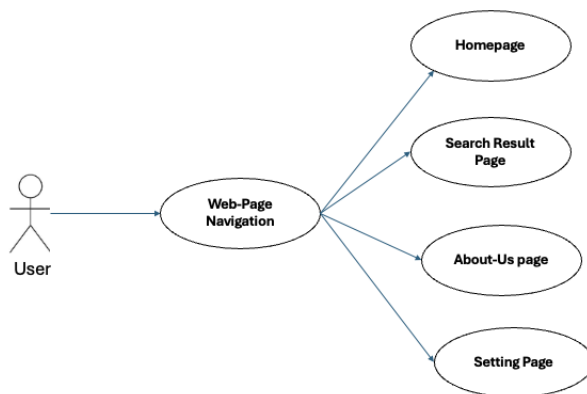


Figure 8: The UC:05 Use-Case Diagram

UC:01	<i>VifinanceNews : Account management</i>
Actors	User shall interact the web-base system through Internet had created account with VifinanceNews
Description	User shall able manage their account by viewing their personal profile (include username, biography, avatar) – which able to be modify (changing the biography information and upload a new avatar); manage their search-history; display search-history; and delete the account
Data	User interaction behavior to the app interface, and the user information (as new biography, new image-file, etc.)
Stimulus	User command issue by their own will on changing personal-information , or delete their account
Response	Confirm personal profile been update, or the account been delete or being cool-off process in wait for delete-process
Comments	None

UC:02	<i>VifinanceNews : Search Article</i>
Actors	Guest User or User-with existed account on VifinanceNews
Description	User shall interact with the app by querying to search for their interested articles, publisher or match with the keywords. User shall be able to save the article to their profile (if they had created the account on the system) – for each article user can upvote or downvote the search-result, or reading the synthesis from the system to analysis the returned the user-query.
Data	User interaction behavior to the app interface, and the user information as search-query, and user-vote activity
Stimulus	User command issue by their own will on sending the query, or saving-trigger to the server.
Response	Returned the search-result, synthesis that displayed on the GUI
Comments	None

UC:03	<i>VifinanceNews : Specified Article Interaction</i>
Actors	Guest User or User-with existed account on VifinanceNews
Description	User after found the list of article, they interact with specifed result by clicking on the article – which navigate to a separate tab that contain more analysis result (include the bias-analysis, fact-check of the result, sentiment-analysis, and offensive-level checking) – and some other meta-data user-can use to understand the result (tags)
Data	User interaction behavior to the app interface (as clicking), and the url-link of the article user-choose (as the meta-data to implement the analysis)
Stimulus	User command issue by their own will on clicking
Response	Returned the analysis displayed on the GUI
Comments	None

UC:04	<i>VifinanceNews : Authentication</i>
Actors	Guess User or User-with existed account on VifinanceNews
Description	Users (in guess mode) click on the Login (if exist the password) or Sign-up if their first-time using the service – there are 2 option (using Password & Email Credential), or using 3rd parties Credential (Google)
Data	User information (as the password, email); user-interaction (mouse-click); OAuth2 token from 3rd parties (Google)
Stimulus	User command issue by their own will on trigger the login by Google option or Filling the Credentials as password and username
Response	Returned the sucessful in case (user-created the account sucessfully), fail (if email already exist, invalid email or password)
Comments	None

UC:05	<i>VifinanceNews : WebPage Navigation</i>
Actors	Guest User or User-with existed account on VifinanceNews
Description	User in either mode shall be navigation and tab-changing between 4 main page (the homepage), the search-result page (for the recent finding base on the query), the About-Us page (for introduce the product and team), their personal-setting page (for getting user-data and manage the behavior of user on the system)
Data	User activity (as move-click, or scrolling activity) – which the system not holding back and track
Stimulus	User command issue by their own will on interaction
Response	Return follow to user physical interaction to the system on the screen, and navigate to correct page and service that user in need
Comments	None

4. Apportioning of Requirements.

These are the list of features that our team will delay on the next version of the project due to team feasibility

- Voice-Over feature
- User-Support feature
- Remove Saved Article
- Visualization of Analysis Result
- Deployment on Kubernetes & Docker
- User-Data Encryption
- User's Article Voting System
- Forgot password
- Google Login Update
- Refine & Update the Caching the Sesion_ID for Specific User

5. Document Contribution

The table below includes the document-contribution information of this SRS documentation.

Name	Role	Fullname	Date
Tran T Dat	Project Manager, Main Editorial, Main Writer	<i>Dat Tran Tien</i>	2025-04-30
Nguyen K.T Cuong	Support Writer	<i>Nguyen Kim Tuan Cuong</i>	2025-04-30
Nguyen D Duc	Support Writer, Support Editorial	<i>Nguyen Duy Duc</i>	2025-04-30
Nguyen H An	Support Writer	<i>Nguyen Hai An</i>	2025-04-30
Nguyen K.V Dung	Support Consultant	<i>Nguyen Khanh Viet Dung</i>	2025-04-30

ViFinanceNews

Group 2 – ViFinanceNews Team

Dat Tran Tien

Nguyen Kim Tuan Cuong

Nguyen Khanh Viet Dung

Nguyen Hai An

Nguyen Duy Duc

Software Design Specification

Documentation

Version: (1.0.0) Date: (May/21/2025)

TABLE OF CONTENTS

Introduction	66
1.1 <i>Document Overview</i>	67
2 Software Architecture Overview	67
2.1 <i>System Architecture</i>	67
2.2 <i>Operating Environment</i>	70
3 Software design specification	70
3.1 <i>Service Interfaces & API Documentation</i>	70
3.2 <i>SearchService</i>	71
3.3 <i>AnalysisService</i>	76
3.4 <i>SummariserService</i>	84
3.5 <i>AuthService</i>	87
3.6 <i>UserService</i>	91
3.7 <i>Logging Service</i>	96
3.8 <i>Front-end (Client-System)</i>	97
3.9 <i>ViFinanceCommon & Java Code Base</i>	110
3.10 <i>ViFinanceCrawLib</i>	114
4 Contribution	195

Introduction

The ViFinanceNews project is a comprehensive financial news analysis platform built on a microservices architecture. This system integrates the ViFinanceCrawLib submodule for data extraction and provides advanced analysis capabilities for financial news articles.

This Software Design Specification document provides a detailed description of the system architecture, component design, and implementation details for the ViFinanceNews platform. The document is structured to give both a high-level overview of the system and detailed specifications for each component.

For the ViFinanceNews project, given its microservices architecture with multiple specialized components, this document provides a comprehensive overview of the entire system in Section 2, followed by detailed specifications for each major component in Section 3. This approach allows for a clear understanding of both the overall system architecture and the specific implementation details of each service – the utility package our team developed for this project (ViFinanceCommon & ViFinanceCrawLib).

The system consists of several key microservices including SearchService, SummariserService, AnalysisService, LoggingService, AuthService, and UserService along with supporting services for authentication and user management.

Document Overview

The document outlines the architecture, components, and interactions of the ViFinanceNews system, which consists of several microservices:

- **SearchService:** Responsible for retrieving and indexing financial news articles
- **AnalysisService:** Performs various types of analysis on news articles
- **SummariserService:** Generates summaries of individual articles and synthesizes information from multiple sources
- **LoggingService:** Provides centralized logging functionality
- **AuthService:** Handles user authentication and authorization
- **UserService:** Manages user profiles and user's data

Each component is described in detail, including its purpose, design, API endpoints, dependencies, and implementation details. The document serves as a comprehensive reference for developers working on the ViFinanceNews platform, providing the necessary information to understand, maintain, and extend the system.

Software Architecture Overview

System Architecture

The ViFinanceNews system is designed as a microservices architecture to maintain the simplicity of system design – and flexibility to adapt with incremental development.

The system employs multiple independent services to handle specific aspects of functionality while maintaining interoperability through well-defined interfaces.

The next session would provide the architecture of the app based on the level of specificity & the interface with external components (as the library and dependencies).

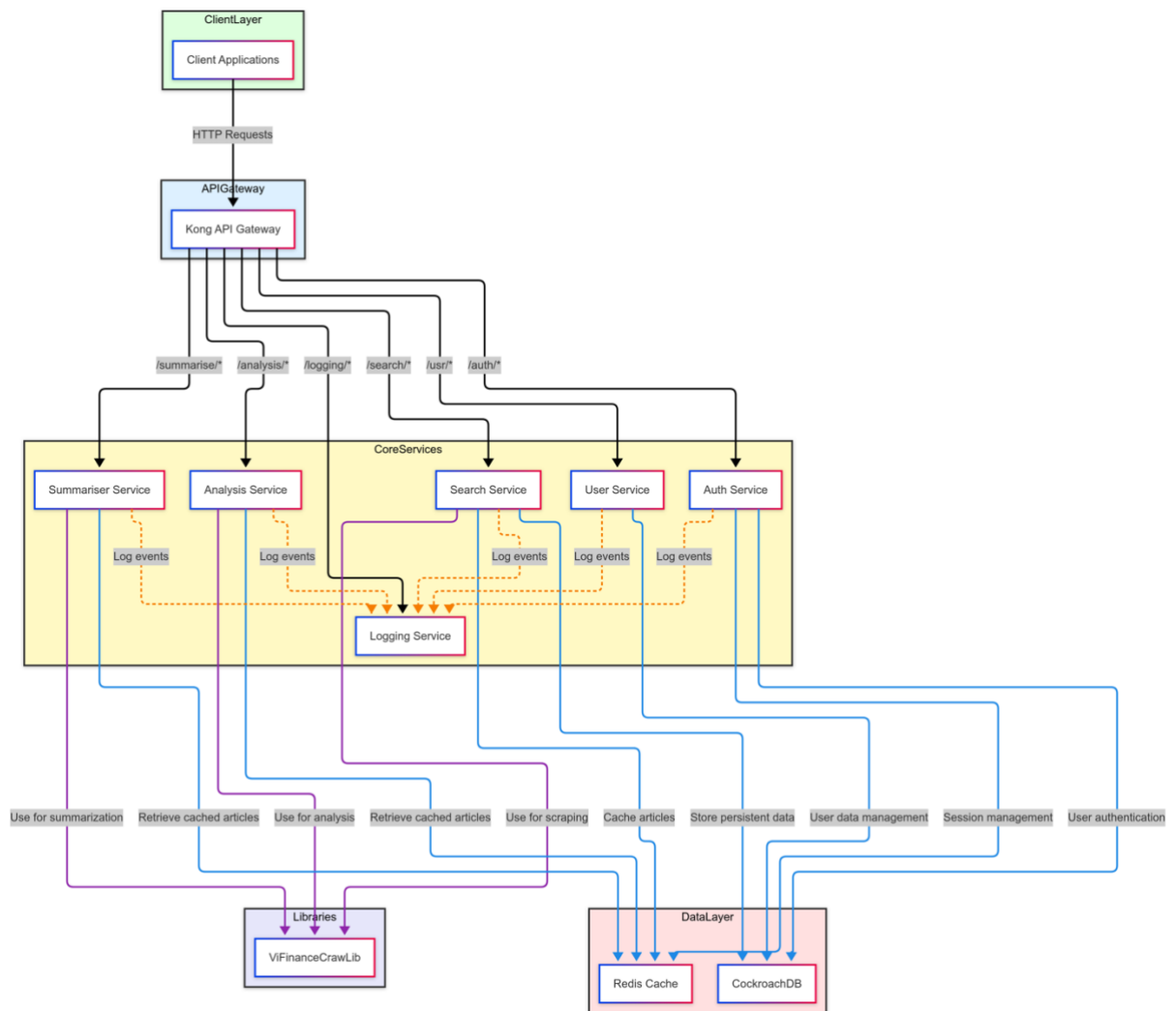


Figure 9: Architecture Diagram with Detailed Components and Dependencies

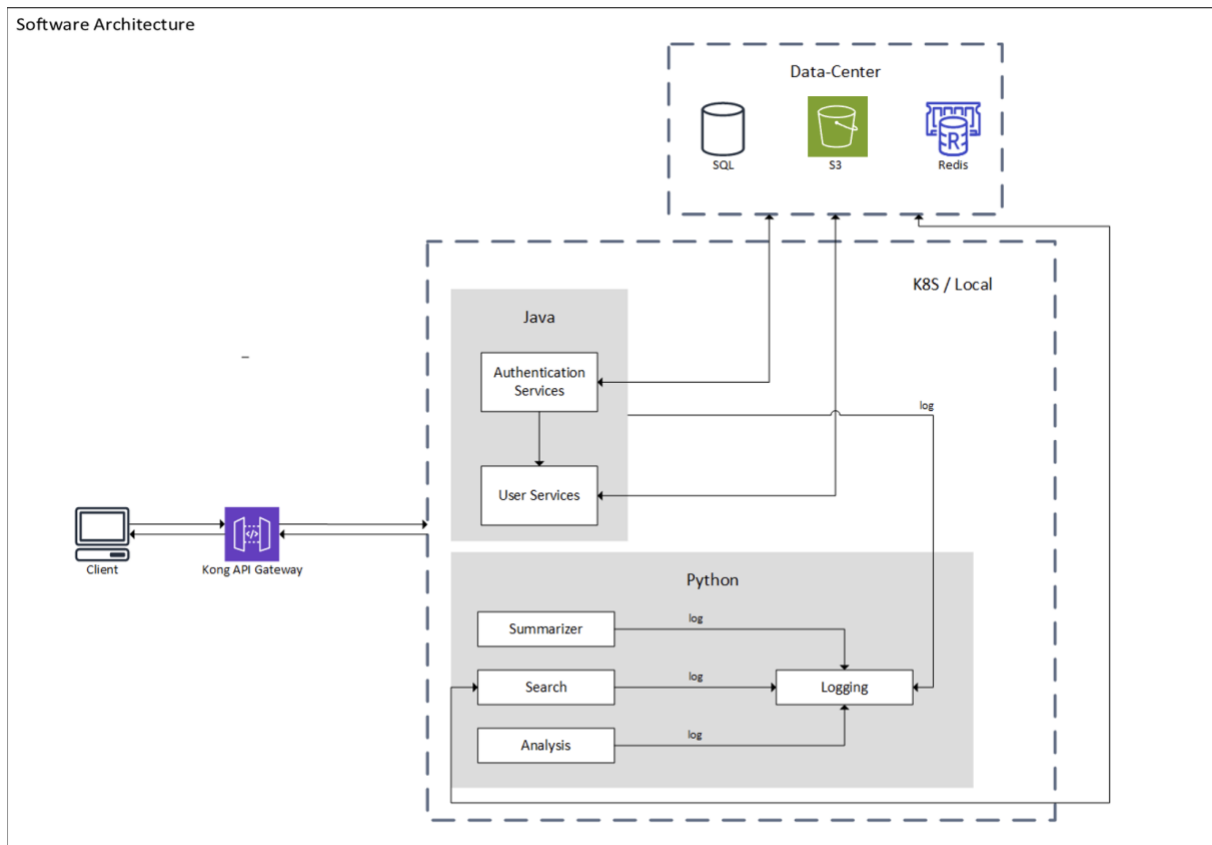


Figure 10: The Simplified Version of the Architecture with Interaction on External Components

Brief Description of the Architecture Design

- Implement inspired on Repository-Based & Layered-Base Architecture
- Repository implementation by using centralized Databases & Storage such as S3, CockroachDB, and Redis for the fast acceleration of user data & organization in dataflow
- Layered Design for security (as Authentication & Verification functionality of user's session) – while the U.I & Interfaces can be replaced and updated across version maintenance
- Optimize the Model-View-Controller by separation of Controller & View in the AuthService & UserService – for logical & intuitive design and debugging

- Modular-Design (introduce of Utility Library as ViFinanceCrawl lib & ViFinanceCommon for the usability of the code)
- In our ideal feasibility – we choose to implement the KONG-API gateways for handle the simplicity in the routing API , but base on the actual timeline of the project, this functionality highly in the experiment and not ready to be integrated in this version

Operating Environment

Containerization: Docker-based deployment for all services [*]

API Gateway: Kong API Gateway (version 3.6) [*]

Database: CockroachDB (v23.1.11)

Caching: Redis

Backend Services: Python Flask-based microservices (Python 11 is ideal) + Java (Version 22 is at least)

Software design specification

Service Interfaces & API Documentation

The services expose REST APIs that are mapped by Ngnix:

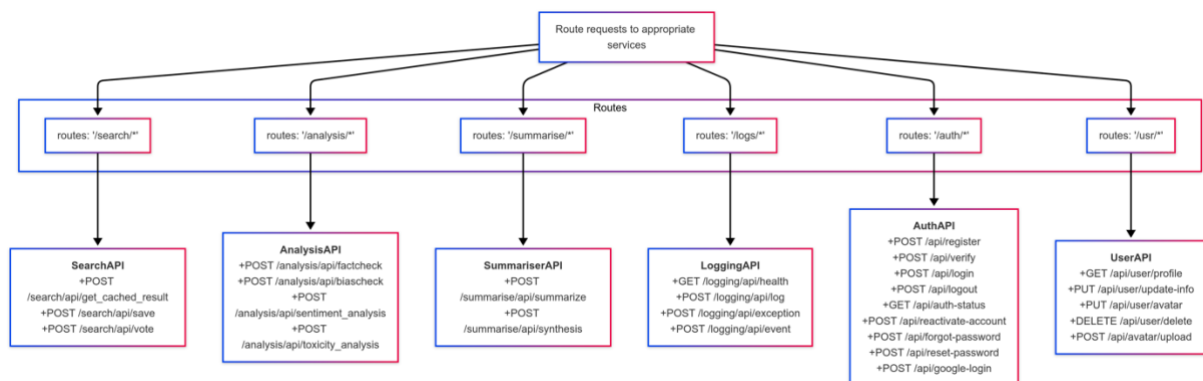


Figure 11: The Diagram Represents the RestAPI of the system

For getting more detailed description of the API Routing Path – please check the detailed description here:

[VifinanceNews-API-Documentation-Version 1.0.0](#)

SearchService

The module was written in Python (ideal for working with Python version 11). Building on the Flask framework by using Flask-CORS for cross-origin request handling.

The SearchService shall be able to implement:

- Implement the search & store results in the cache
- Implement the save an article to the DB
- Implement the vote from the user for 1 article:
 - Doing the upvote with `get_up_vote`
 - Doing the downvote with `get_down_vote`

3.2.1: Searching & Caching

For the searching & saving of the result to the cache we do provide the algorithm following the below activity diagram

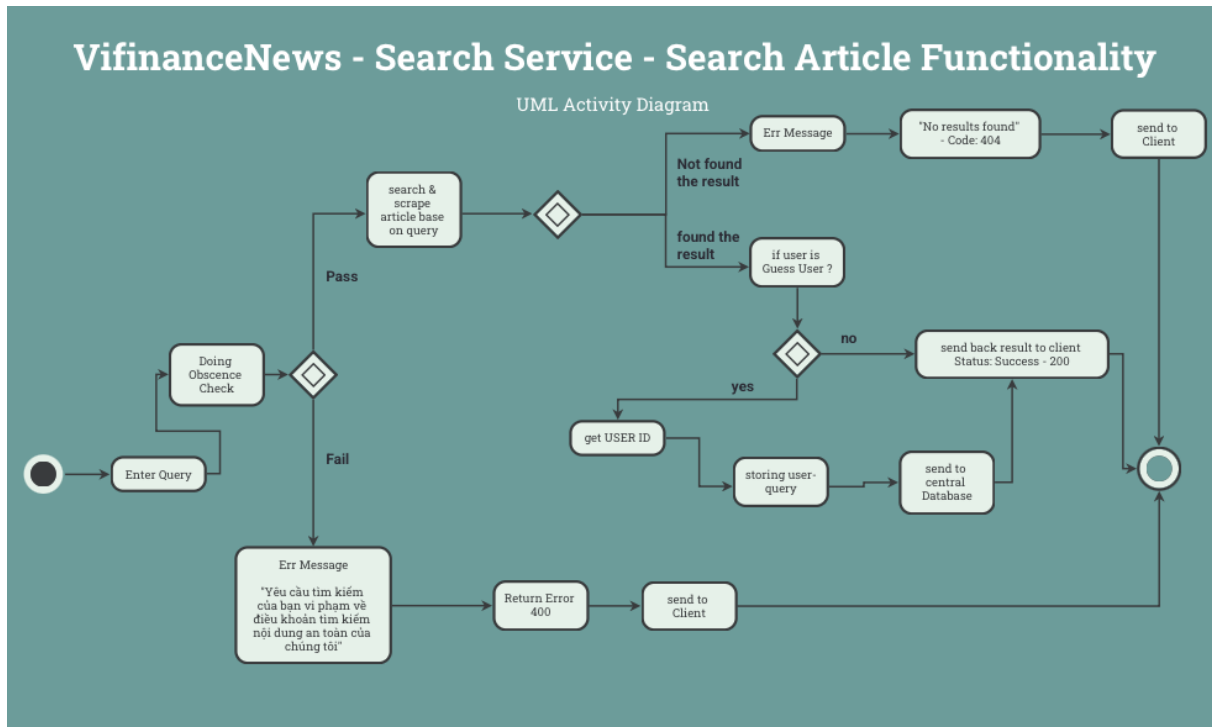


Figure 12: The Idea for the Searching functionality – the implementation was: `get_cached_result()`

The query is essential for the search to be successful; if the user is in guess mode (their searched query would not be stored in the DB – currently the data of the user is not client-side encrypted, ideally this can be included in a later version).

Also to enhance the safeness of content – that the search engine uses (including an obscene check) as a gatekeeper from sending explicit content from users – the obscene check was implemented with prompt-engineering technique with Gemini model

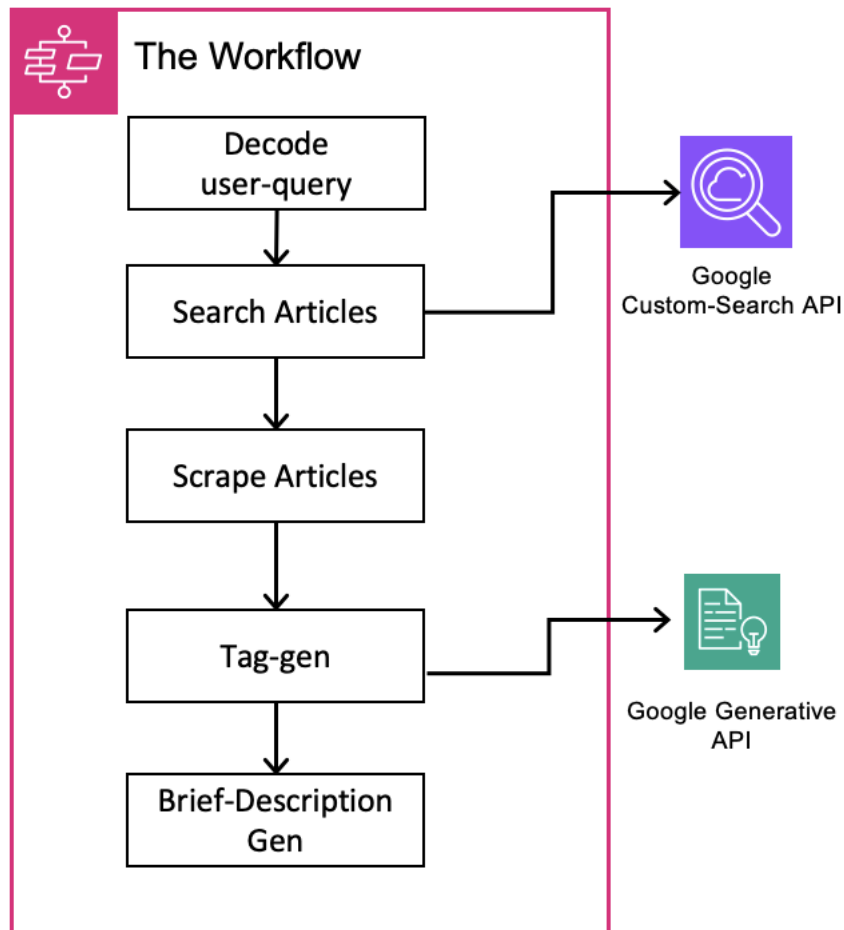


Figure 13: The idea of preparing multiple articles for return to the cache

We are using parallel processing to process the scrapped action & also handling separated articles can be costly & over-use the quota of Google-Generative API

Our team divided the batch size into 5 articles – for each batch, one time of the search, the user can return at most 10 results (after some filtration of ill-formatted results – the expected level of return is 7 articles at most)

Batch processing was also used to generate the tag and the descriptions for the articles

3.2.2: Article Saving

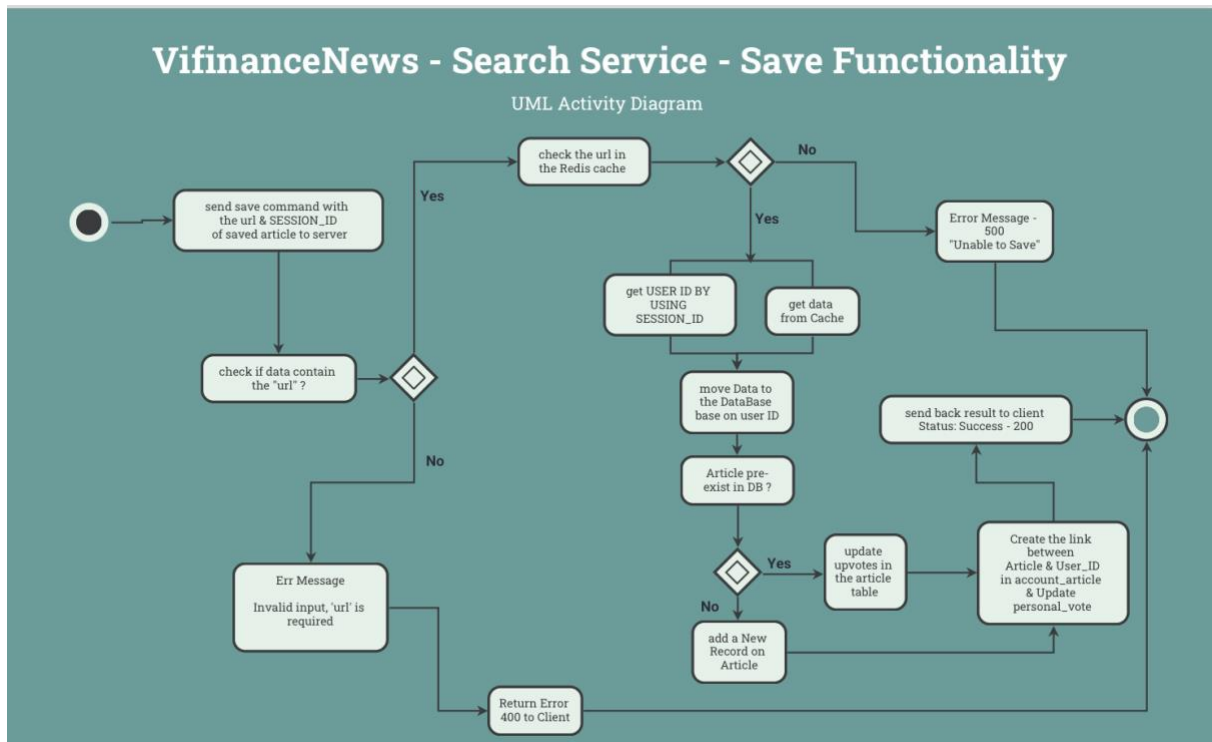


Figure 14: The Activity Diagram Implement the Saving Mechanism

The UML Activity Diagram above describes the process of saving – as we include the selected article URL (triggered by user-mouse click on the UI), the URL was for retrieving the article’s meta-data on the cache + combining with the SESSION_ID (for accessing the USER_ID in the cache as well).

We would integrate the USER_ID + Articles’ meta-data to implement the search with 5 steps as follows

1. Access & Retrieve the Table from the Redis cache
2. Insert the Articles into the “article” table in the Database ([Check the SRS – Page 35 -Figure 2 for the ER chart](#))
3. Retrieve the Vote with the USER_ID, the article’s URL from the cache
 - The key in the Redis for storing the vote as: `user:{ USER_ID }:``personal_vote`

- *Note: One user can vote for multiple articles in the same session – so vote values always come with the URL of the articles to distinguish*

4. Linking the article with tags (by using the TAG_ID & ARTICLE_ID) in the Database
5. Access the user vote by using USER_ID and URL
6. Get the value of the vote the user made on the selected article: VOTE_TYPE
7. Link the user to the article (include USER_ID, ARTICLE_ID, VOTE_TYPE)

3.2.3: Article Voting

Voting Values in our system include 2 types: total vote for one article (upvotes: int), and personal_vote (categorical value: -1, 0, 1) described in the figure below

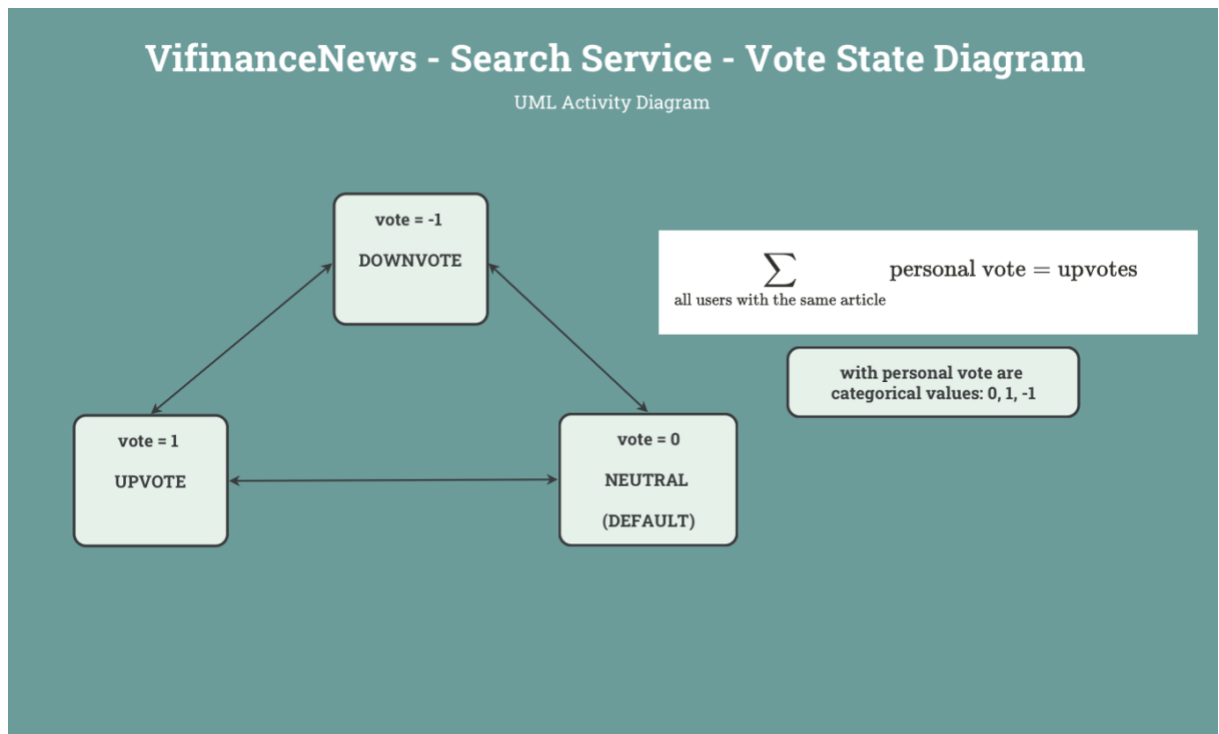


Figure 15: Voting State-Diagram of personal_vote – the total upvotes is the sum of all user's vote for one articles

The meaning of each category is as follows:

UPVOTE = Like || Positive || The article content is useful

DOWNVOTE = Dislike || Negative || The article is poor , irrelevant

NEUTRAL = Default Value || Not showing interest

AnalysisService

The Module was for implementing the Analysis of the Article content that users find interesting. (Building on the Flask framework by using Flask-cors for cross-origin request handling)

The module shall do:

- FactCheck (check if the article is factual basing inter-source comparison)
- BiasCheck (for measuring the level of bias in the writing)
- Sentiment Analysis (measuring the level of associated information to the user: Positive, Negative, or Neutral with a value score 1 to 10 – each map to as: Very Negative (Rất tiêu cực), Negative (Tiêu cực), Neutral (Trung lập), Positive (Tích cực), Very Positive (Rất tích cực)) – the higher the sentiment score show the higher tendency of the article content follow that major sentiment.
- Measuring the level of Toxicity in variety of categories (including identity attack, racism, vulgar language, etc.) – the value range for scores in toxicity mapped from domain of [0,1] continuous to discrete [1,10] – with near to 10 show higher intensity of the toxic level

The FactCheck and BiasCheck were implemented using ViFinanceCrawLib by integrating the prompt-engineering technique with LLM (with trained reasoning) and using the Search Engine to access the reference for evaluate the factuality of the source.

Sentiment Analysis uses the LLM – but with a Text-Classification Model that has been trained on a large dataset by using a Hugging-Face interface for model interaction

and data-processing & inference (Check [Sentiment Analysis Model](#) to learn more on the model that we use)

Toxicity Analysis was similar to Sentiment Analysis – but we used the specified model that had been trained with the down-stream task for return predefined output – but the toxicity analysis only performed well with English – so we do need to pass the Vietnamese’s original text (or en-shorten version of the data) to the Gemini API for translation for efficiency & semantics preservation (check [Detoxify Repository](#))

For efficiency and accuracy of the NLP (natural language processing) tasks – our team has to pre-process the textual input – our team implements [spaCy](#) (a Python base for word-Segmentation), and [vncoreNLP](#) (a Java-based toolkit for word segmentation in Vietnamese)

The below figures provide the activity diagram for the pipeline on implementing the task

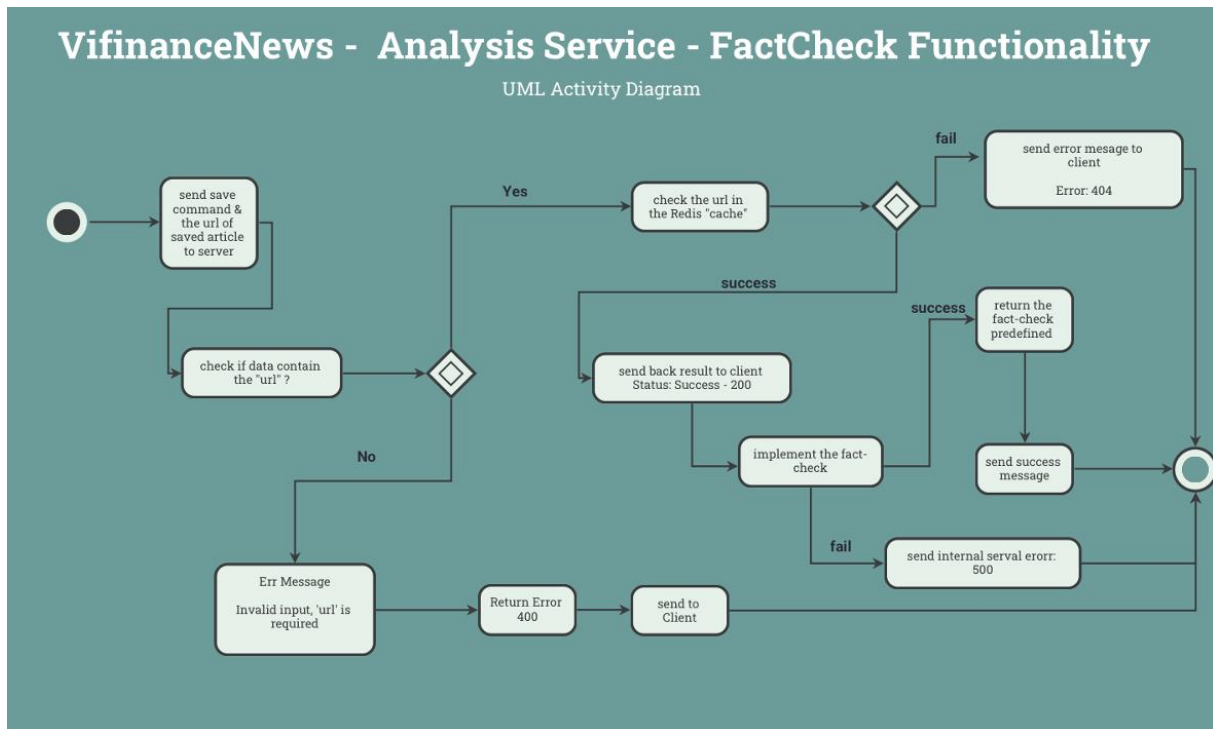


Figure 16: The Working Pipeline of the FactCheck

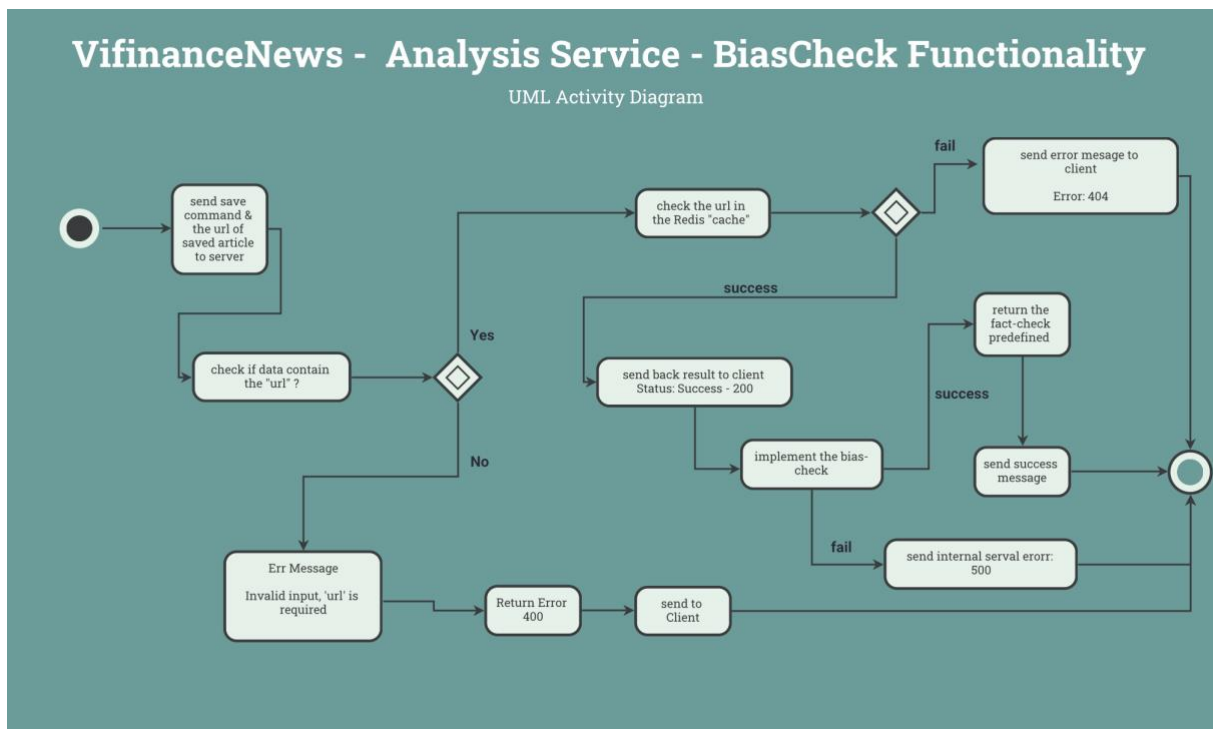


Figure 17: The Working Pipeline for the Bias-Analysis (or Bias-Check functionality)

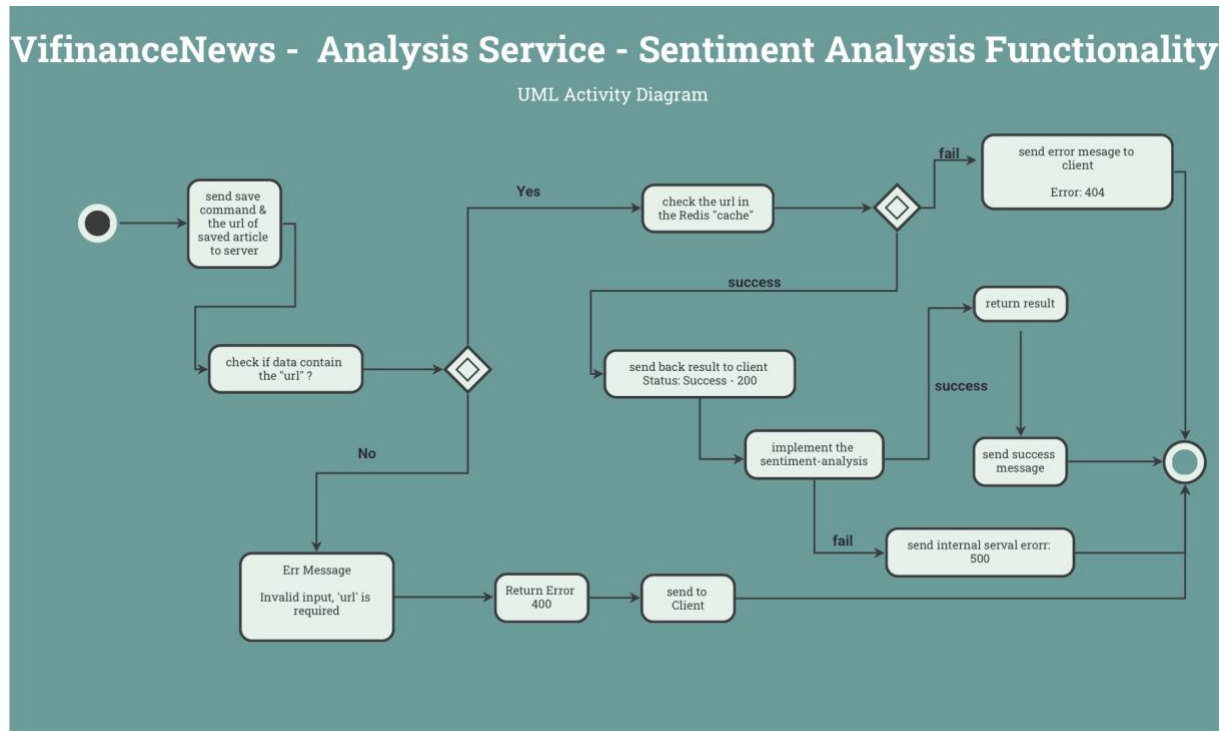


Figure 18: The Pipeline of Sentiment Analysis Functionality

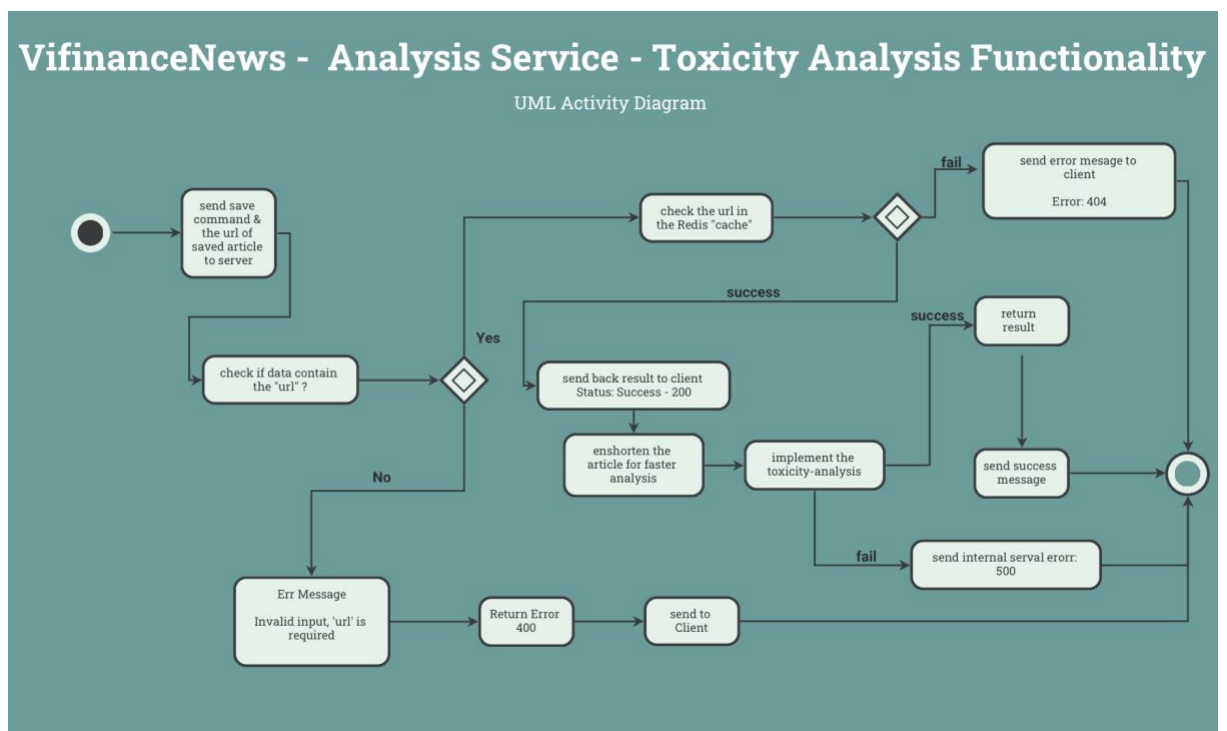


Figure 19: The Pipeline of Toxicity Analysis Functionality

3.3.1 Explain the Fact-Checking Method

For implementing the fact-checking of the content we implement the following pipeline:

1. Provide the Articles to the LLM to generate the question and answer related to the topics from the input
2. Searching the Web & Scrapping Content that is related to the generated question and returning the evidence
3. Provide the input Article & extracted Evidence for the LLM to implement cross-analysis (the metric in measuring reliability of output result was based on [this](#) standard)

Source Reliability		Information Creditability	
A	Completely reliable	1	Confirmed by other sources
B	Usually reliable	2	Probable
C	Fairly reliable	3	Possible
D	Not usually reliable	4	Doubtful
E	Unreliable	5	Improbable
F	Reliability cannot be judged	6	Truth cannot be judged

Figure 20: The metric to evaluate the factuality of the articles in the fact-check – base on [Royal United Services Institute of Nova Scotia](#)

Notice: Cache only stores the data of found articles, not the analysis result – all results will be lost after the user's session ends or after leaving the app.

The prompt that ours's team use to guide the L.L.M in generating the creditability of the source as follow (this prompt will only capture a section of output not include the example for guiding the output format)

`prompt = f"""`Bạn là một trợ lý phân tích và đánh giá tính xác thực của thông tin. Dưới đây là một mệnh đề & thông tin và một tập hợp bằng chứng. Hãy đánh giá mức độ mà bằng chứng hỗ trợ, mâu thuẫn hoặc trung lập đối với thông tin, bằng cách xem xét:

- Mối quan hệ logic giữa tuyên bố và bằng chứng.
- Độ mạnh của bằng chứng, bao gồm nguồn gốc, tính chính xác và mức độ liên quan.
- Bối cảnh và giả định tiềm ẩn có thể ảnh hưởng đến diễn giải bằng chứng.

`###` ****Hệ thống đánh giá độ tin cậy****

Hãy đánh giá mức độ tin cậy của từng nguồn và từng thông tin bằng hệ thống NATO:

- ****Đánh giá độ tin cậy của nguồn**** (Chữ cái):
 - ****A****: Hoàn toàn đáng tin cậy
 - ****B****: Đáng tin cậy
 - ****C****: Khá đáng tin cậy
 - ****D****: Không đáng tin cậy
 - ****E****: Không thể đánh giá
- ****Đánh giá độ chính xác của thông tin**** (Chữ số):
 - ****1****: Đã được xác minh
 - ****2****: Có khả năng đúng

- ****3****: Có thể đúng

- ****4****: Không chắc chắn

- ****5****: Không thể đánh giá

Kết quả đánh giá sẽ được biểu diễn dưới dạng ****A1, B2, C3, v.v.****, trong đó:

- ****A1**** là thông tin đáng tin cậy nhất, có nguồn mạnh và đã được xác minh.

- ****E5**** là thông tin đáng tin cậy kém nhất, có nguồn yếu và không thể đánh giá.

Mệnh đề thông tin: {statement}

Bằng chứng:

{evidence}

****Hãy trả lời chú ý các ràng buộc phía dưới:****

- Tổng Hợp Cuối Cùng: [Tóm tắt thông tin đã kiểm tra để đưa ra kết luận cuối cùng về chủ đề.]

- Kết luận: [Hỗ trợ/Mâu thuẫn/Trung lập]

- Phân tích bằng chứng: [Các dẫn chứng trên có mối liên hệ như thế nào trong việc đưa ra kết luận về vấn

đề người dùng tìm hiểu]

- Mức độ tin cậy: [Ví dụ: A1, B3, D5] và chú thích của mức độ [ví dụ: A1 - Đáng Tin Cậy và Đã Được Xác

Minh]

```
- Giải thích: [Giải thích ngắn gọn về lý do của bạn, có đề cập đến nguồn bằng chứng và mức độ tin cậy của chúng.]

- Lời khuyên cho người dùng về cách nhìn nhận hiện tại: [Một lời khuyên ngắn gọn]

- Danh sách các dẫn chứng (mỗi bài báo là một string):

    [Số thứ tự bài báo ]: Tiêu đề - nguồn - [url] \n

    [Số thứ tự bài báo ]: Tiêu đề - nguồn - [url] \n

    ....
```

SummariserService

SummariserService was to implement the 2 downstream tasks including:

- Summarize 1 selected Articles
- Synthesis multiple Articles

For the Synthesis task, we implemented the prompt-engineering technique with a predefined format and provided data for the LLM returning the result (some post-processing task is necessary for making the output readable and visible to the user)

To summarize the task we need a more sophisticated pipeline (inspired by ([Vuong et al., 2024](#)) finding, later we do mention that in the [ViFianceCrawlLib section](#) as the algorithm for implementing summarization)

3.4.1: Summarize Endpoint

Section 3.4.1 mentions the workflow of summarization functionality as follows

Workflow:

1. **Input Parsing:** Accepts either a raw string or JSON object with a URL key.
2. **URL Decoding:** Ensures safe and valid string input via `urllib.parse.unquote`.

3. **Article Retrieval:** Calls `scraper.get_an_article(URL)` to fetch article content.
4. **Summarization:**
 - Uses `summarizer.summarize(article_text)` from `SummarizerAlbert`.
 - Internally performs:
 - **Preprocessing** (tokenization, stopword filtering)
 - **Extractive summarization** using ALBERT sentence embeddings + TextRank algorithm.
 - **Abstractive summarization** with Gemini via prompted generation.
5. **Response:** Returns JSON-formatted summary text.

3.4.2 Synthesis Endpoint

Section 3.4.2 mentions the workflow of synthesis functionality as follows

Workflow:

1. **Input Validation:** Expect a JSON list of article URLs.
2. **Batch Scraping:** Uses `scraper.get_multiple_article(URLs)` to retrieve multiple article contents.
3. **Error Handling:** If any article fails to load, returns a 404 with an error message.
4. **Multi-Article Summarization:**
 - Uses `summarizer.multi_article_synthesis(articles)`.
 - Internally:
 - Performs extractive summarization of each article individually.
 - Merges key sentences from all sources.
 - Synthesizes a cohesive paragraph using Gemini LLM with citation-aware prompting (APA style).
5. **Response:** Returns the final synthesized summary across sources.

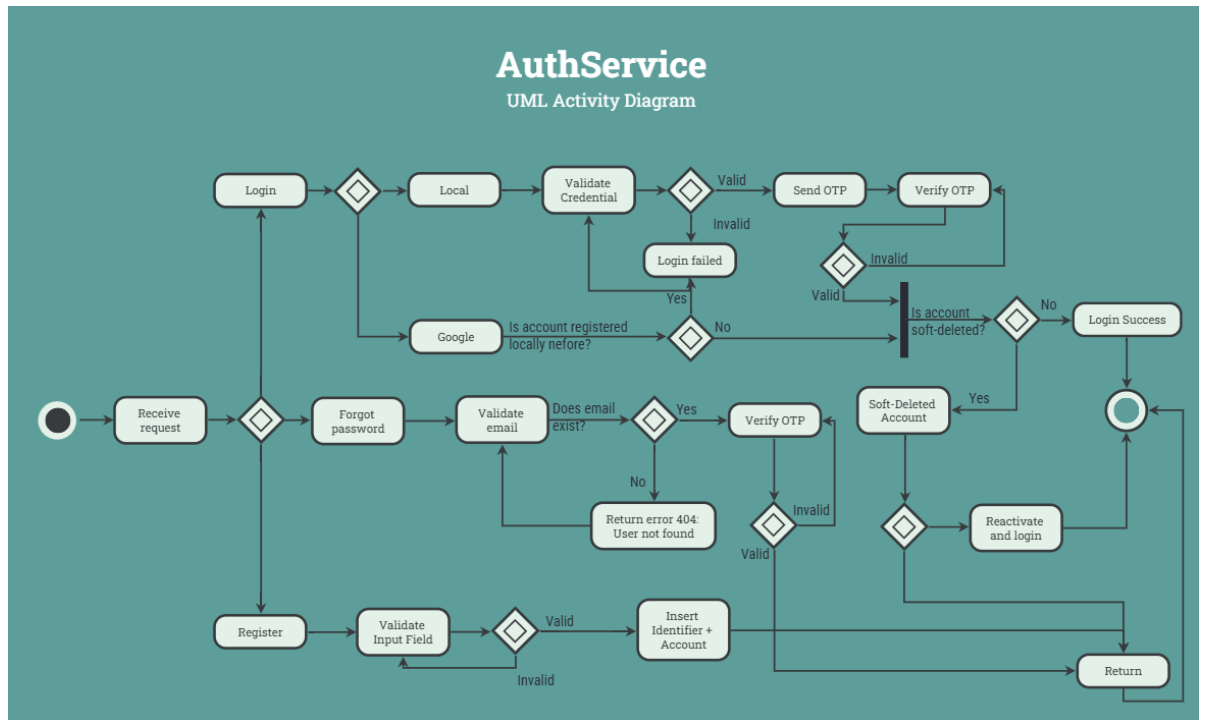
AuthService

Figure 21: The Activity Diagram of AuthService – the Return in the diagram means: Return to the Homepage after successful login

The **AuthService** (Authentication Service) is implemented in Java using the Javalin web framework. Its structure is influenced by the Model-View-Controller (MVC) architecture, with controllers handling API routes and services encapsulating business logic. It serves as the central component responsible for managing all user authentication flows within the ViFinance system. The following activity diagram illustrates the high-level workflow of the AuthService (Figure 12).

3.5.1 Login Methods

The system currently supports **two methods of authentication**:

- **Local Authentication (Credential-Based):** Users provide a valid email and password.

- **Google OAuth Login:** Users log in via their Google account using OAuth 2.0. This option simplifies access and reduces the need for password management.

Currently, **OTP (One-Time Password) verification is enforced only for credential-based login**, as a security measure to validate that the registered email is active and not automated. Google-authenticated users are verified by Google's identity provider.

3.5.2 Supported Authentication Operations

The **AuthService** provides the following major functionalities:

- **Register New Account:**

Users create an account by providing their email and password. An OTP is sent via email to verify ownership. The account is created only after OTP validation.

- **Login (Local or Google):**

- *Local Login:* Users submit email and password. After credentials are validated, an OTP is required for final login.
- *Google Login:* If the user logs in with Google and no associated account exists, one is automatically created.

- **OTP Verification:**

Handled via Redis to ensure fast and temporary OTP management without storing sensitive information in the main database.

- **Forgot Password:**

Users may request a password reset, which triggers the sending of an OTP for verification. Upon successful verification, they are allowed to create a new password.

- **Soft Delete Account (Temporary Deactivation):**

Users can deactivate their account. This moves their data to a `deleted_accounts` table

without removing their identifier. They are still allowed to log in during a reactivation period, but are notified that the account is pending deletion.

- **Reactivate Account:**

Temporarily deleted users can reactivate their accounts through the login process by confirming OTP and opting to restore their account.

- **Logout:**

Logging out terminates the session. If inactive for over 1 hour, sessions expire automatically to ensure security.

3.5.3 Security Mechanisms

- **Password Hashing:**

All passwords are hashed using a secure algorithm (e.g., bcrypt or SHA-256) before storage in the identifier table.

- **ID Hashing (Data Obfuscation):**

The internal user identifier (id) is a UUID stored in the identifier table. To enhance security and prevent correlation or inference attacks:

- A **SHA-256 hash of the id** is computed and stored as `userId` in other tables, such as `account`.
- This ensures that no direct foreign key relationship exposes the original UUID to external queries.
- Whenever user-related data needs to be accessed (e.g., `account` info), the system hashes the `id` at runtime to match records via the hashed value.
- This mechanism strengthens data isolation between the core identity layer and the public-facing services.

- **OTP Verification via Redis:**

OTPs are stored in Upstash Redis with a short expiration time, ensuring quick access and disposal. No OTPs are persisted in the SQL database to maintain system hygiene and reduce risk.

- **Account Lockout Mechanism:**

To prevent brute-force attacks, repeated failed login attempts trigger a temporary lockout based on `failedAttempts` and `lockoutUntil` fields.

3.5.4 Integration

AuthService is integrated with other components such as:

- **UserService:** For profile creation, updates, and user-related logic.
- **Redis (Upstash):** For OTP storage and expiry.
- **MailSlurp (Testing):** For validating OTP flows in automated tests.
- **KONG API Gateway:** For routing authentication-related requests from external clients to the AuthService.
- **Frontend:** Through endpoints with `/api/` prefix.

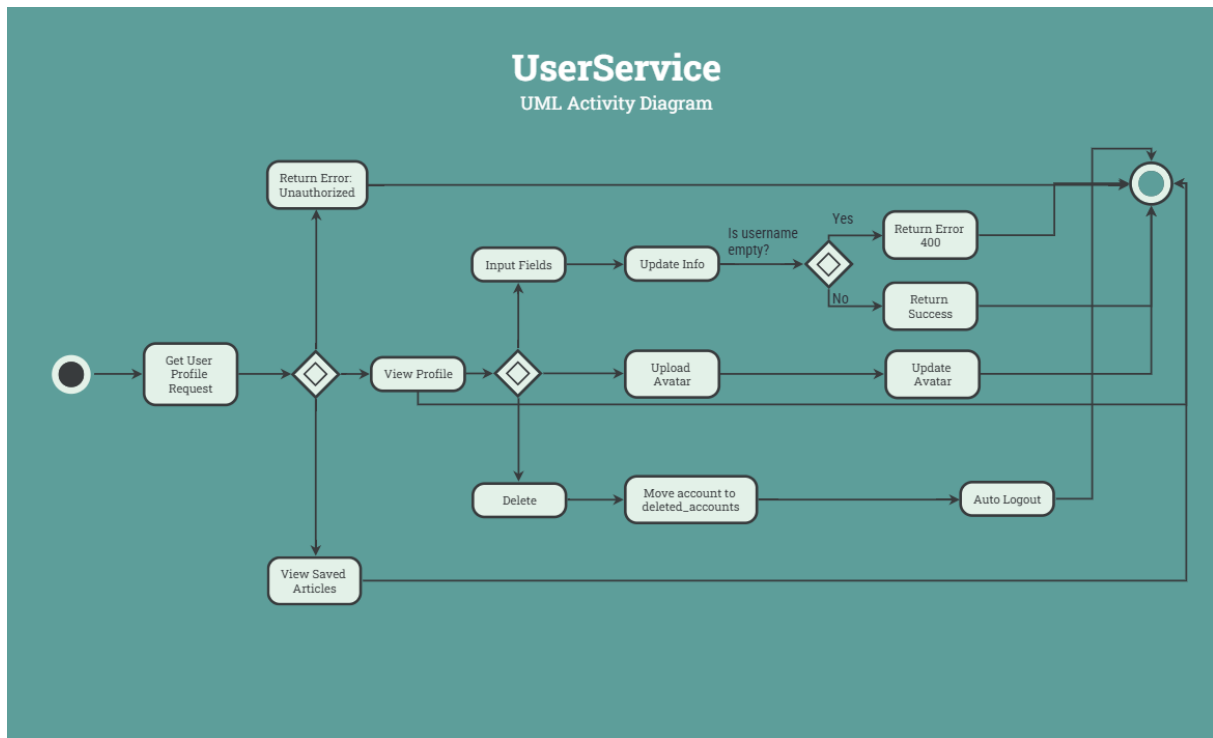
UserService

Figure 22: The Activity Diagram of the UserService

The **UserService** is responsible for handling all logic related to managing user profiles and user-specific data, such as saved articles. It acts as the business logic layer between controllers (e.g., API handlers) and DAO (Data Access Objects), ensuring data is processed and validated correctly before interacting with the database.

- User-related information includes:
 - username
 - bio (biography)
 - avatarLink (user profile image URL)
- The **UserService** implements the following major functions:
 - **View User Profile:**

Retrieves and returns the user's profile details (username, avatar link, and bio)

for display in the front end. All profile views are based on hashed userIDs to ensure privacy and decoupled access.

- **Update User Profile:**

Allows the user to modify their username, avatarLink, or bio. Changes are persisted to the account table using hashed identifiers for reference.

- **Delete User Profile:**

Initiates a soft deletion process by migrating user-related data into the deleted_accounts table. The identifier record remains untouched during this period, allowing reactivation.

- **Reactivation Support:**

If a user logs in again after soft deletion but within the grace period, the account is automatically restored from the soft-deletion buffer.

- **View Saved Articles:**

Returns a paginated list of articles saved by the user. The query is optimized to reduce database load through caching and cursor-based pagination.

3.6.1: Return the Saved Articles

The **UserService** provides a method to **return the list of articles that a user has saved** to their profile. This feature is critical for personalizing the user experience, enabling users to revisit articles of interest. In section 3.6.1 – we explain the algorithm for returning the saved articles effectively. The detailed pipeline we do provide is the following the UML activity diagram.

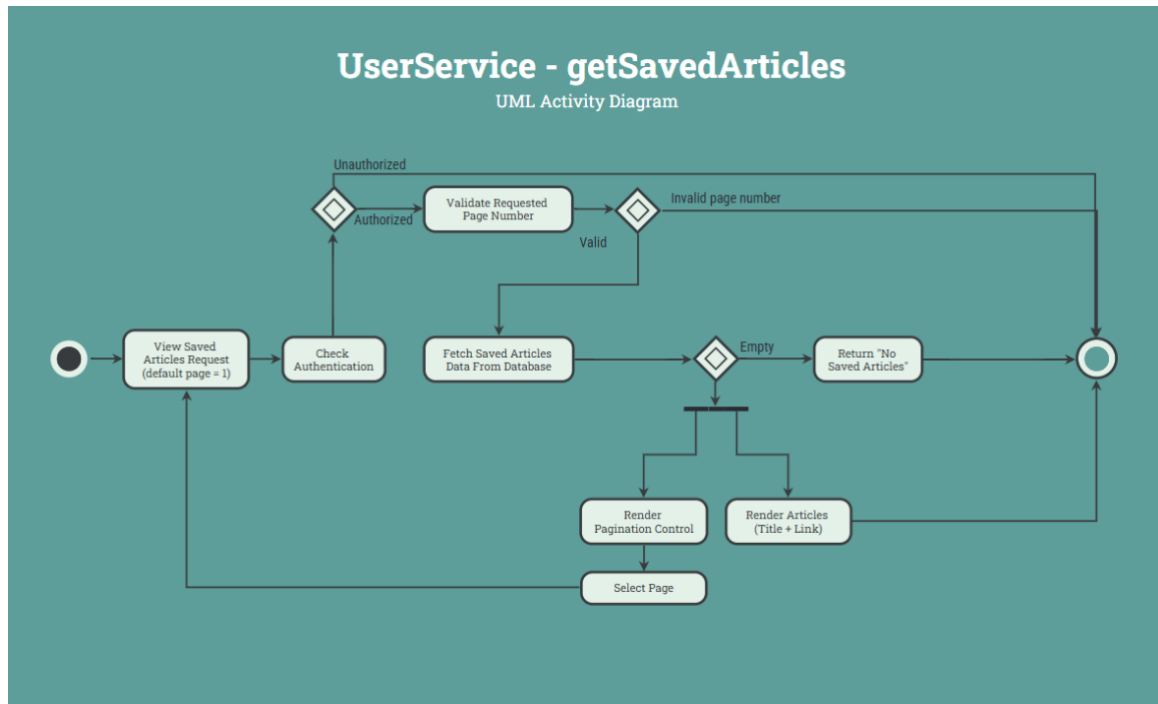


Figure 23: The Activity Diagram represent the “get saved articles list” from user effectively

Since database access is an expensive operation in terms of performance, this method is designed with **efficiency and scalability in mind**.

System Overview and Implementation

To avoid excessive database queries and improve scalability, we implemented the logic using **cursor-based pagination** (also referred to as offset-based pagination in this context).

The key design points include:

- **Page-based retrieval:** Articles are retrieved in small, fixed-size batches (default: 5 per page), using SQL LIMIT and OFFSET clauses.
- **Sorted ordering:** Results are ordered by the saved_at timestamp in descending order, ensuring the most recently saved articles appear first.
- **Two-query system:**
 - One SQL query retrieves the current page of saved articles for the user.

- A second query retrieves the **total count** of saved articles to support pagination controls on the frontend.
- **Pagination controls:** The frontend dynamically generates page buttons based on the total article count returned, and clicking on any page number loads that specific set of saved articles using a backend API call.

SQL Query

The backend performs two database queries (via AccountDAO of ViFinanceCommon):

```
-- Fetch current page of articles
SELECT a.article_id, a.saved_at, ar.title, ar.url
FROM account_article a
JOIN article ar ON a.article_id = ar.article_id
WHERE a.user_id = ?
ORDER BY a.saved_at DESC
LIMIT ? OFFSET ?;
-- Count total saved articles
SELECT COUNT(*) FROM account_article WHERE user_id = ?;
```

Figure 24: The SQL Query that use to handle the get the saved list articles from user

Backend Logic

The Java backend (inside **UserService**) executes the above queries using JDBC, then returns a `Map<String, Object>` containing:

- `articles`: A list of article metadata (ID, title, URL, saved time).
- `totalCount`: The total number of saved articles for pagination.

The API handler (`/api/user/saved-articles`) parses the current page from the query string and returns the corresponding data as JSON.

Frontend Integration

On the frontend: The pagination is dynamically calculated using `totalCount` divide for the number of articles per page.

Performance Considerations

While this pagination approach is performant for small to moderate datasets, it relies on OFFSET-based querying, which may become inefficient for large datasets due to the way SQL engines must skip rows. Currently, no index is used on the `saved_at` column, which is acceptable given the limited number of users and articles in early development.

Logging Service

The logging service is for record everything that happens in the system (the system behavior, the error message of other service had been encounter for easy of tracking.

The system uses Azure Application Insights as the logging backend. Logs are recorded with structured fields such as service name, event type, and timestamp to maintain clarity and traceability.

The backend relies on the opencensus library to handle communication with Azure. Console logging is also enabled to support development. Logs are grouped by severity and event type to support consistent formatting and easier filtering.

The logging functionality is wrapped into a single log event function, using python decorator to log the events with as little redundancy as possible

The design focuses on keeping the logging flow lightweight and efficient. No extra storage systems (like local log files) are used to avoid unnecessary resource usage.

Environment variables are used to manage sensitive configuration like instrumentation keys. CORS is enabled globally to support cross-service communication. The system can be extended in the future with minimal changes—for example, by adding file-based logs or external queue integrations.

Front-end (Client-System)

Overview

The frontend is a single-page application (SPA) that communicates with multiple backend services through RESTful APIs. It provides an intuitive interface for users to search, analyze, and interact with financial news articles.

The frontend application provides:

- News article search and display functionality
- Advanced article analysis tools (sentiment, bias, fact-checking, toxicity analysis)
- User authentication and profile management
- Article synthesis and summarization capabilities
- Dark/light theme support
- Responsive design for multiple screen sizes

Used Technology

- Framework: React 19.0.0 with functional components and hooks
- Build Tool: Vite 6.2.0
- Styling: Tailwind CSS 4.1.3 with custom theme system
- Routing: React Router DOM 7.5.1
- Icons: Lucide React and React Icons
- Language: JavaScript (ES6+)
- Development Environment: Node.js 20.18.0

Conceptual Model

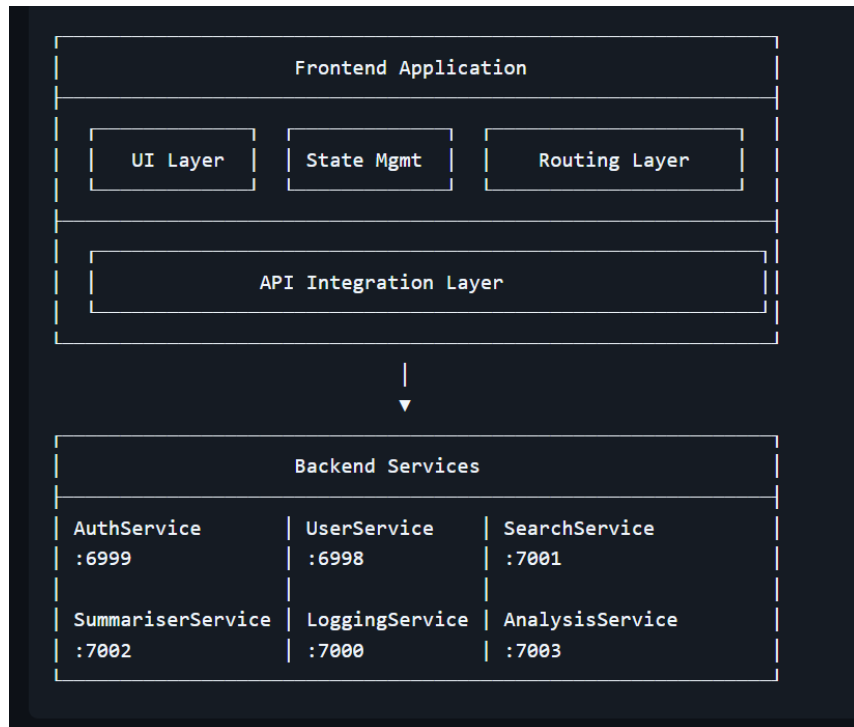


Figure 25: The Conceptual Model of the system

Based on the conceptual model our team proposed in figure 17 – the frontend had been adapted to the design by introduce 3 news layers: UI Layer, State Management (mgnt), and a Routing Layer.

UI Layer: Contains the presentational components — buttons, forms, modals, cards, tables, etc. which renders content to the user, handles user interactions (e.g., clicks, inputs)

State Management: Manages the app’s state — such as user data, authentication status, etc. which centralizes shared state to make data flow predictable and accessible across components.

Routing Layer: Manages navigation between views/pages in the app by defining routes and maps them to specific components or layouts.

Component Hierarchy

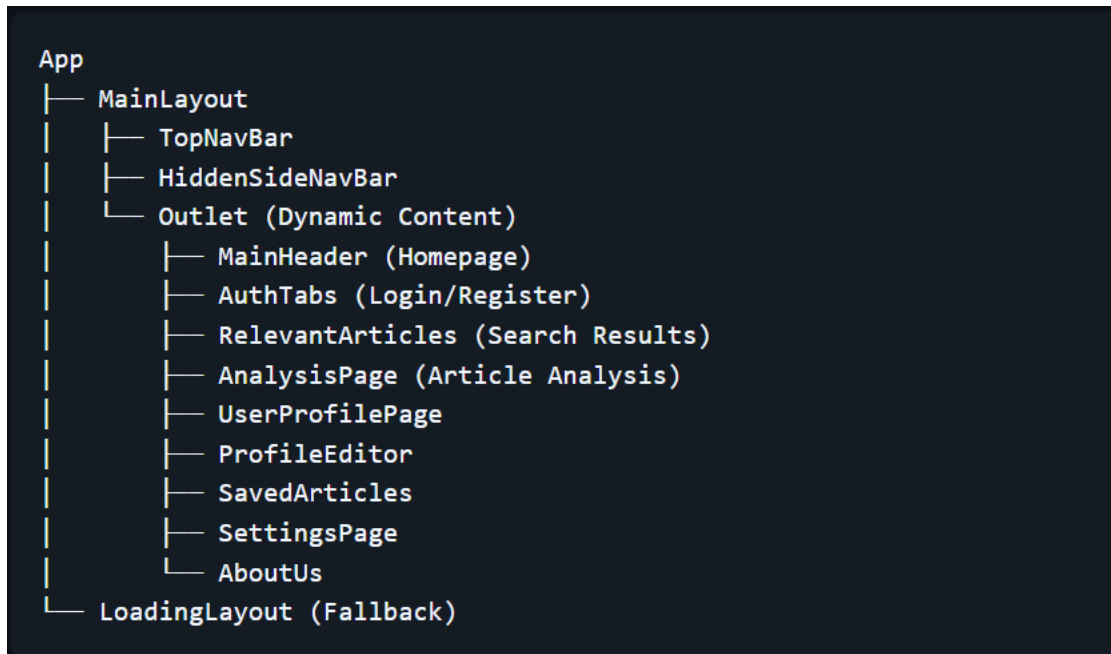


Figure 18: Structure of the Front-end Source Code Workspace

Data Flow

1. User interacts with the UI and triggers some action.

2. Action updates state or navigates to a new route.
3. API Integration Layer makes a request to the appropriate backend service.
4. Backend service processes the request and sends back a response.
5. UI updates based on the response (e.g., show user profile, search results, error message).

Component Design

Core Components

a. App Component (App.jsx)

Purpose: Root application component managing routing and authentication state.

Key Features:

- Route configuration and protection
- Loading state handling
- Authentication-based navigation guards

b. MainLayout (layouts/MainLayout.jsx)

Purpose: Primary layout wrapper providing consistent navigation structure.

Key Features:

- Top navigation bar integration
- Hidden sidebar navigation
- Dark mode toggle functionality

c. AuthTabs (components/AuthTabs.jsx)

Purpose: Authentication interface handling login, registration, and password recovery.

Key Features:

- Tabbed interface for different auth modes
- OTP verification integration
- Google OAuth support
- Form validation and error handling
- Session management

d. RelevantArticles (components/RelevantArticles.jsx)

Purpose: Main article display and interaction component.

Key Features:

- Article search and display
- Voting system (upvote/downvote)
- Article saving functionality
- Synthesis generation from multiple articles
- Citation handling with reference links
- Analysis page navigation

e. AnalysisPage (components/AnalysisPage.jsx)

Purpose: Comprehensive article analysis dashboard.

Key Features:

- Multiple analysis types: sentiment, toxicity, fact-checking, bias detection
- Sequential API calls with loading states
- Reference citation linking
- Analysis result visualization
- Article metadata display

Navigation Components

a.TopNavBar (components/TopNavBar.jsx)

Features:

- Search functionality integration
- Page navigation

b. HiddenSideNavBar (components/HiddenSideNavBar.jsx)

Features:

- Collapsible sidebar navigation
- User profile quick access
- Logout functionality
- Dark mode toggle
- Responsive behavior: Hidden when not in use by the user.

User Management Components

a. UserProfilePage (components/UserProfilePage.jsx)

Features:

- Tabbed interface for different profile sections
- Profile statistics display
- Navigation to profile editing

b. ProfileEditor (components/ProfileEditor.jsx)

Features:

- User information editing
- Avatar upload functionality
- Password change capability
- Account deletion with confirmation

c. SavedArticles (components/SavedArticles.jsx)

Features:

- Paginated article list display
- Article metadata presentation

- External link navigation

User Interface Design

a. Design Principles

The application follows a **mobile-first design approach**, ensuring optimal performance and usability on smaller devices before scaling up to larger screens. A structured **breakpoint strategy** is implemented using Tailwind CSS, with breakpoints defined at sm (640px), md (768px), lg (1024px), and xl (1280px) to support responsive behavior across a range of device sizes. To achieve adaptive and dynamic layouts, the design leverages **CSS Grid and Flexbox**, enabling flexible component arrangement and consistent spacing across screen resolutions.

b. Component Styling Approach

The primary method for styling components is through **Tailwind CSS utility classes**. This atomic CSS approach allows for rapid UI development directly within JSX. For example, a card component might use a class string such as `<div className="bg-white dark:bg-gray-800 shadow-lg rounded-xl p-6">` to define its appearance across light and dark themes. In addition to utility classes, the project incorporates **custom CSS properties** defined in a `customTheme.css` file. These variables, such as `--color-primary-a0`, `--color-surface-a0`, and `--color-dark-surface-a0`, enable consistent theming across components and simplify future design changes. Finally, for elements that require unique or complex styling behaviors, **component-specific styles** are used. These are defined through custom CSS classes, such as `.hidden-bar`, which adapts its background color based on the

active theme using the `@variant dark` directive. This layered approach balances consistency, reusability, and fine-grained control over individual components.

c. Visual Hierarchy

The visual hierarchy is established through a designed typography scale that combines system fonts with the Poppins font family to ensure readability and style consistency. The color system centers around a **primary lime green theme**, which reinforces branding and draws attention to key elements. Consistent spacing is maintained by utilizing Tailwind's spacing scale which creates balanced layouts and improving overall aesthetics.

State Management

Authentication Context API

The AuthContext provides a centralized authentication state management system that makes user data and authentication functions available throughout the entire application component tree.

The user object contains complete profile information that components can access without additional API calls, including display data like `userName` and `avatarLink` for UI rendering. The login function accepts either a `userId` for standard authentication or a complete user data object for OAuth flows, returning a Promise to handle asynchronous authentication processes. The logout function provides immediate session termination and state cleanup across all components simultaneously.

The loading boolean prevents race conditions during authentication state changes, ensuring UI elements don't render incorrectly while authentication status is being determined. The `updateUser` function enables real-time profile updates without requiring full re-authentication, maintaining seamless user experience during profile modifications.

The code below show the implementation of the logic of “Authentication Context API”

```
{
  user: {
    userId: string,
    userName: string,
    email: string,
    bio: string,
    avatarLink: string
  },
  login: (userIdOrData) => Promise<void>,
  logout: () => void,
  loading: boolean,
  updateUser: (fields) => void
}
```

Local Loading States

```
const [loading, setLoading] = useState(false);
const [error, setError] = useState("");
```

This standardized loading pattern ensures consistent user experience across all components by providing predictable feedback during asynchronous operations. The loading

boolean state triggers UI changes such as showing spinners, disabling buttons, or displaying skeleton screens while API calls are in progress. The error string state captures and displays user-friendly error messages when operations fail, replacing generic technical errors with actionable feedback. This pattern prevents user confusion by clearly indicating when the application is processing requests and provides immediate feedback when something goes wrong, following React best practices for handling asynchronous state changes in functional components.

Routing & Navigation

Route Configuration

Public Routes

- / - Homepage (MainHeader)
- /login - Authentication interface
- /about-us - Information page
- /relevant-articles - Public article search

Protected Routes

- /profile - User profile dashboard
- /profile-edit - Profile editing interface
- /analysis - Article analysis page
- /saved-articles - User's saved articles

API Integration

Service Integration Patterns

Endpoints Used:

- POST /api/get_cached_result - Article search
- POST /api/save - Save articles to user account
- POST /api/get_up_vote / POST /api/get_down_vote - Voting system
- POST /api/get_user_vote - Retrieve user's vote status
- POST /api/get_total_upvotes - Get article popularity

Analysis Service Integration

Sequential Analysis Pattern: The AnalysisPage implements a sequential analysis system:

1. POST /api/factcheck/ - fact-checking → 15s delay
2. POST /api/summary/ - summary→ 15s delay
3. POST /api/sentiment_analysis - sentiment analysis → 15s delay
4. POST /api/toxicity_analysis/ - toxicity analysis → 15s delay
5. POST /api/biascheck/ - bias detection

This whole analysis uses progressive enhancement - strategy of presenting partial results to the user as soon as they become available, rather than waiting for the entire analysis process to complete. This enhances user experience by providing immediate feedback and reducing the perceived loading time.

User Service Integration

Profile Management:

- User profile data fetching
- Avatar upload with file handling
- Saved articles with pagination
- Account management operations

Authentication Service Integration

Security Features:

- Session-based authentication
- OTP verification flow
- Google OAuth integration
- Secure logout with session cleanup

ViFinanceCommon & Java Code Base

ViFinanceCommon Module Details

The **ViFinanceCommon** module acts as a centralized shared library used by both **AuthService** and **UserService**, providing core utilities, models, and data access components that foster consistency and code reuse.

Models Package

This package defines the core domain entities shared across services, including:

- **Account:** Represents the user profile with attributes such as username, avatarLink, and bio. Contains account-specific information including hashed user IDs for secure referencing.
- **Identifier:** Holds authentication-related information such as email, password hash, OTP data, and logout status.

These models form the backbone for data interchange and persistence operations across the application layers.

3.9.1.2 Configs Package

Centralizes configuration management including environment variables, database connection settings, Redis cache configurations, and other application-wide constants. This promotes uniform access to configuration and easy environment-based adjustments.

3.9.1.3 DAOs Package

Implements reusable Data Access Objects abstracting database operations for the shared models. DAOs expose CRUD interfaces to services, promoting code reuse and simplifying database interaction. Examples include AccountDAO and IdentifierDAO.

3.9.1.4 Utilities Package

A collection of helper classes and methods used throughout the system, including:

- Hashing tools and verification utilities
- Deletion scheduler for database
- Email and OTP management tools
- Redis caching utilities

These utilities reduce boilerplate and centralize complex logic for easier maintenance.

3.9.1.5 Maven Module Structure and Dependency

ViFinanceCommon is developed as a Maven module under the parent project vifinancenews. The parent POM manages the module's versioning and dependencies. Other services include ViFinanceCommon as a dependency to utilize its shared resources, ensuring consistency across the code base.

3.9.2 Java Code Base Overview

This section provides a high-level overview of the overall Java code structure for the ViFinanceNews system, highlighting the interaction between services and the common library.

3.9.2.1 Service Packages

- **AuthService (auth Package):** Handles authentication-related workflows, including login (local and Google OAuth), registration, password resets, account deletion/reactivation, and session management.
- **UserService (user Package):** Manages user profile operations such as viewing, updating, deleting profiles, and returning saved articles with efficient search algorithms (e.g., cursor pagination).

Both services leverage ViFinanceCommon for shared business logic, models, and data access.

3.9.2.2 Common Library and Utilities

As previously described, ViFinanceCommon consolidates cross-cutting concerns that span multiple services.

3.9.2.3 Class Diagram

The class diagram (Figure 15) illustrates the main classes and their relationships across the code base:

- Core domain entities (Account, Identifier) reside in ViFinanceCommon.
- Services and DAOs extend these models to implement business logic.
- Utilities support various operations like authentication, OTP management, caching, and configuration access.

This layered and modular design promotes separation of concerns, scalability, and maintainability

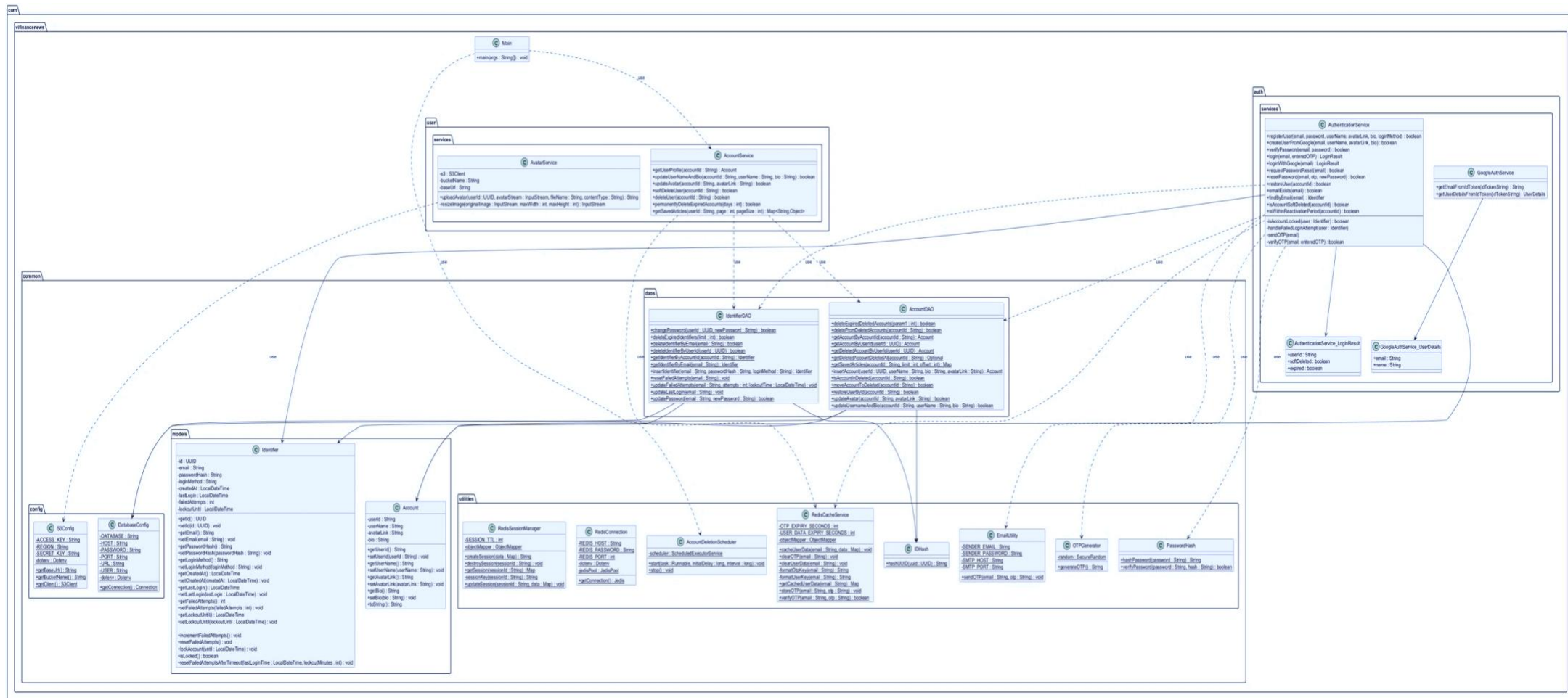


Figure 19 26: The class diagram represents the structure of the ViFinanceCommon package & the AuthService & the UserService

ViFinanceCrawLib

The ViFinanceCrawLib is a utility package that our team developed for implementing tasks such as summarization, article analysis, and searching for articles.

3.10.1: The Package Structure

The next page provides the class diagram of the ViFinanceCrawLib and showing the relationship between the objects in the package.

The package contains these modules (name & their purpose)

- article_database (to handle the article entity, and manage the interaction with the Databases, implementing the search functionality)
- QuantAna (Quantitative Analysis - to produce the Quantitative Analysis of articles entity)
- QualAna (Qualitative Analysis - to provide the Qualitative Analysis of article entity)
- Summarizer (to provide the Summarization for 1 article, and produce synthesis on multiple articles)

3.10.1.1 article_database

The Article Database was the interaction with the Database; however, the module included the tool for implementing support the data preparation as Searching, Data-Processing, Adding the Data to the Database, etc.

The package article_database would include these modules

- Database (for handling the Azure SQL database)
- DatabaseCockroach (handle the Cockroach Postgres Database)
- ArticleQueryDatabase (AQD) (an interface to add the data from the cache to DB)
- ArticleScraper (for scrapping article)

- SearchEngine (searching)
- TextCleaning (cleaning the text)
- TitleCompleter (finish the title)

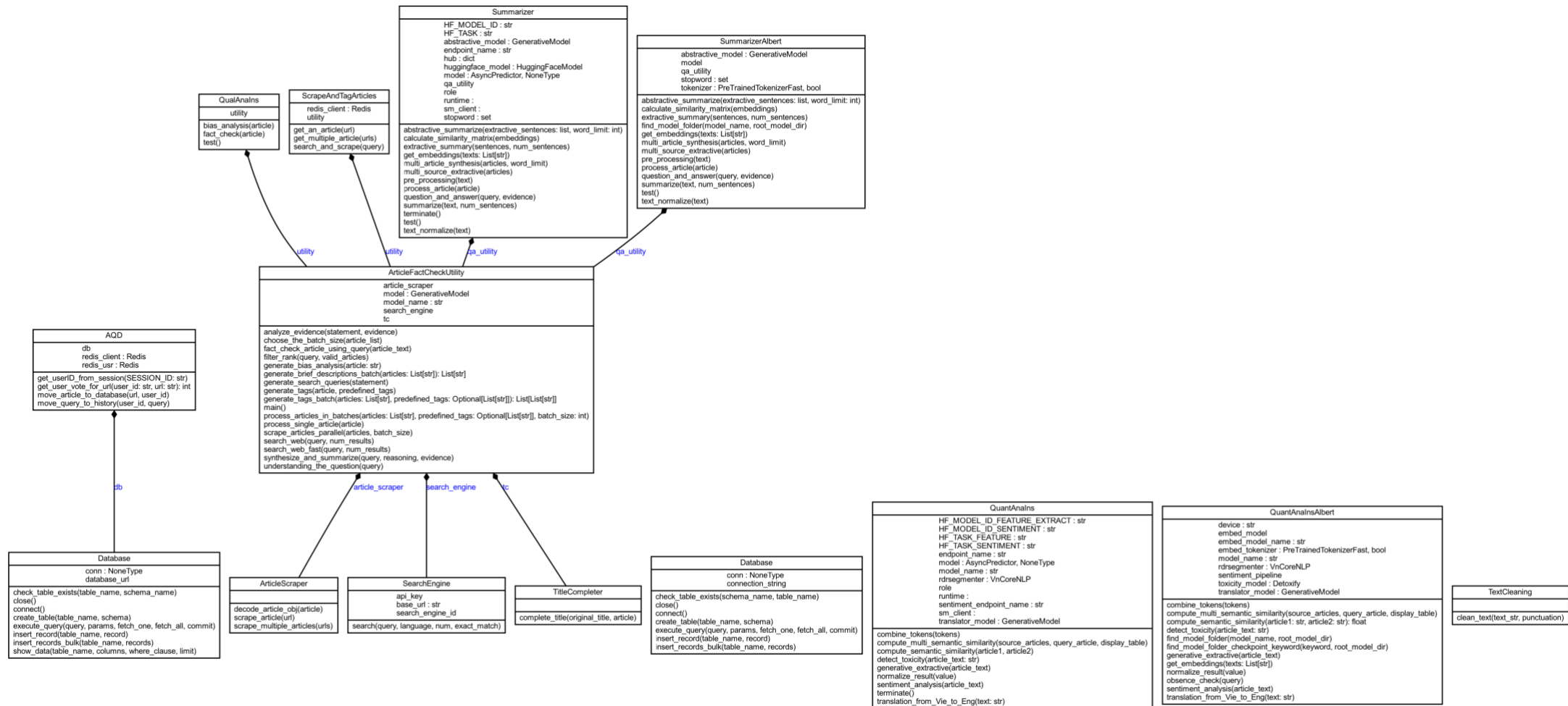


Figure 20: The Class Diagram for the ViFinanceCrawlLib Package

a. Database

Database
conn : NoneType connection_string
<pre> check_table_exists(schema_name, table_name) close() connect() create_table(table_name, schema) execute_query(query, params, fetch_one, fetch_all, commit) insert_record(table_name, record) insert_records_bulk(table_name, records) </pre>

Figure 27: The Database object description

Purpose

This Database class provides utility methods for interacting with an Azure SQL Database.

It is designed to manage database connections, create tables, insert records, check the existence of tables, and execute custom SQL queries.

The class ensures that operations on the database are efficient, including bulk inserts and custom queries.

Dependencies

Library/Tool

Purpose

pyodbc

Python ODBC interface for connecting to SQL databases.

Library/Tool	Purpose
os	Used for environment variable management (e.g., loading connection strings).
dotenv	Loads environment variables from a .env file for secure management of sensitive information (like database connection strings).

Attributes

Attribute Name	Type	Description
connection_string	str	The SQL Server connection string required for establishing a connection.
conn	pyodbc.Connection	The active database connection object used to execute SQL commands.

Main Methods

Method Name	Description
init (self, connection_string)	Initializes the Database object with a connection string for connecting to an Azure SQL Database.

Method Name	Description
<code>connect(self)</code>	Establishes a connection to the SQL database using the provided connection string.
<code>create_table(self, table_name, schema)</code>	Creates a table in the database if it doesn't already exist, using the provided schema dictionary.
<code>check_table_exists(self, schema_name, table_name)</code>	Checks whether a specified table exists in the database under a given schema.
<code>insert_record(self, table_name, record)</code>	Inserts a single record into the specified table. The record is provided as a dictionary.
<code>insert_records_bulk(self, table_name, records)</code>	Inserts multiple records into the specified table in an optimized bulk operation.
<code>execute_query(self, query, params=None, fetch_one=False, fetch_all=False, commit=False)</code>	Executes a custom SQL query, with options for fetching results and committing the transaction.
<code>close(self)</code>	Closes the database connection.

b. DatabaseCockroach

Database
conn : NoneType database_url
<pre> check_table_exists(table_name, schema_name) close() connect() create_table(table_name, schema) execute_query(query, params, fetch_one, fetch_all, commit) insert_record(table_name, record) insert_records_bulk(table_name, records) show_data(table_name, columns, where_clause, limit) </pre>

Figure 28: DatabaseCockroach module has its own implementation of Database

Purpose

This is another version of the Database class that facilitates interactions with a CockroachDB database (PostgreSQL-base server).

It supports various operations such as creating tables, inserting records (individually or in bulk), checking the existence of tables, and executing custom SQL queries.

The class ensures a secure and efficient database interaction by handling database connections, query execution, and result fetching in a structured manner.

Dependencies**Library/Tool****Purpose**

psycopg2

PostgreSQL database adapter for Python (used to interact with CockroachDB, which is PostgreSQL-compatible).

Library/Tool	Purpose
os	Used to handle environment variables (e.g., loading the database URL).
dotenv	Loads environment variables from a .env file, for securely managing sensitive configurations.
uuid	Used for generating unique identifiers when necessary.
datetime	Used to handle date and time-related functionalities (such as inserting timestamped data).

Attributes

Attribute Name	Type	Description
database_url	str	The CockroachDB connection string required for establishing a connection.
conn	psycopg2.Connection	The active connection object used to execute SQL commands in CockroachDB.

Main Methods

Method Name	Description
init (self, database_url)	Initializes the Database object with a connection string for CockroachDB.

Method Name	Description
<code>connect(self)</code>	Establishes a connection to the CockroachDB using the provided connection URL.
<code>create_table(self, table_name, schema)</code>	Creates a table in CockroachDB if it does not already exist. The schema is passed as a dictionary where keys are column names and values are SQL data types.
<code>check_table_exists(self, table_name, schema_name='public')</code>	Checks whether a specified table exists in a particular schema within the database. Returns True if the table exists, otherwise False.
<code>insert_record(self, table_name, record)</code>	Inserts a single record (provided as a dictionary) into the specified table.
<code>insert_records_bulk(self, table_name, records)</code>	Performs a bulk insert operation for multiple

Method Name	Description
	records into the specified table.
<code>execute_query(self,query,params=None,fetch_one=False,fetch_all=False, commit=False)</code>	Executes a custom SQL query. Can optionally fetch one or more rows and commit the transaction.
<code>show_data(self,table_name,columns=None,where_clause=None,limit=None)</code>	Displays data from a specific table, with optional filters (where clause) and limits on the number of rows to fetch.
<code>close(self)</code>	Closes the database connection.

c . ArticleQueryDatabase (AQD)

Purpose

The ArticleQueryDatabase (AQD) module is designed to handle the transition of news article data and user query logs from a Redis-based cache to a CockroachDB instance.

It ensures articles retrieved and processed earlier can be safely persisted in a relational database and logs user search queries for analytics or personalization.

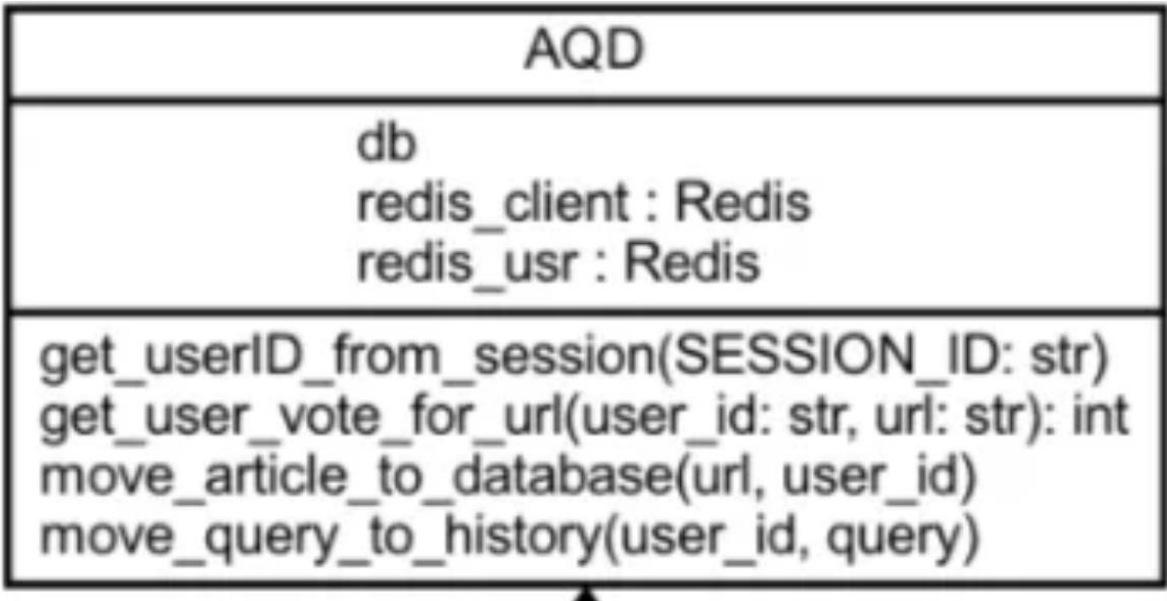


Figure 29: Object description of the AQR (ArticleQueryRepository)

Dependencies

Library/Tool	Purpose
dotenv	Loads environment variables from a .env file for database and Redis configuration.
os	Accesses system environment variables to retrieve credentials and connection details.
redis	Connects to a Redis instance and retrieves cached article data.
json	Decodes JSON-encoded article data stored in Redis.
time	Gets the current timestamp for logging user queries.

Library/Tool	Purpose
Database (article_database.DataBaseCockroach)	Custom database utility class for interacting with CockroachDB, including query execution and connection management.

Attributes

Attribute Name	Type	Description
db	Database	An instance of the custom Database class used to interact with CockroachDB.
redis_client	redis.Redis	A Redis client instance configured using environment variables for retrieving temporary article data.

Main Methods

Method Name	Description
init()	Initializes the class by loading environment variables, establishing a Redis client, Redis user and preparing the database connection utility.

Method Name	Description
<code>get_userID_from_session(SESSION_ID: str)</code>	get the SESSION_ID from currently working user
<code>get_user_vote_for_url(user_id: str, url: str): int</code>	get the vote from the user with selected article's url
<code>move_article_to_database(url, user_id)</code>	moving the article from permanent storage
<code>move_query_to_history(user_id, query)</code>	storing the user query history with provided SESSION_ID

d. ArticleScraper

Purpose

The ArticleScraper module is responsible for fetching and decoding news article content from URLs using the NewsPlease scraping library.

It extracts metadata and main text content from single or multiple URLs and converts them into a standardized format suitable for storage or further processing.

The module abstracts the variability and structure of different online sources, offering a consistent interface for other components that rely on article content (such as Summarizer, ArticleFactCheckUtility, or SearchEngine).

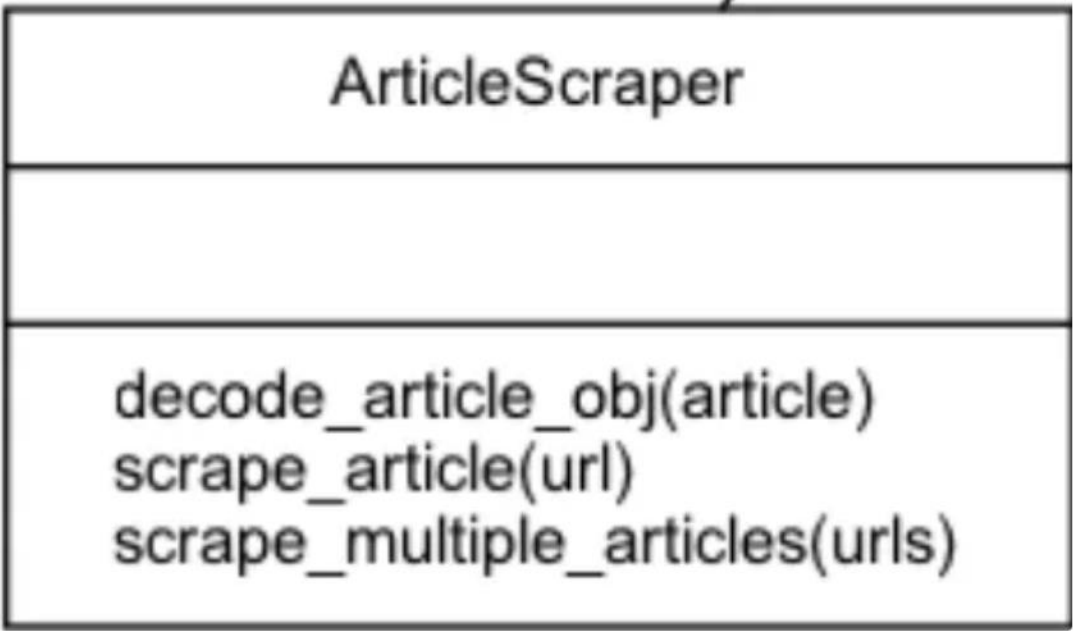


Figure 30 The Article Scrapper Object Description

Dependencies

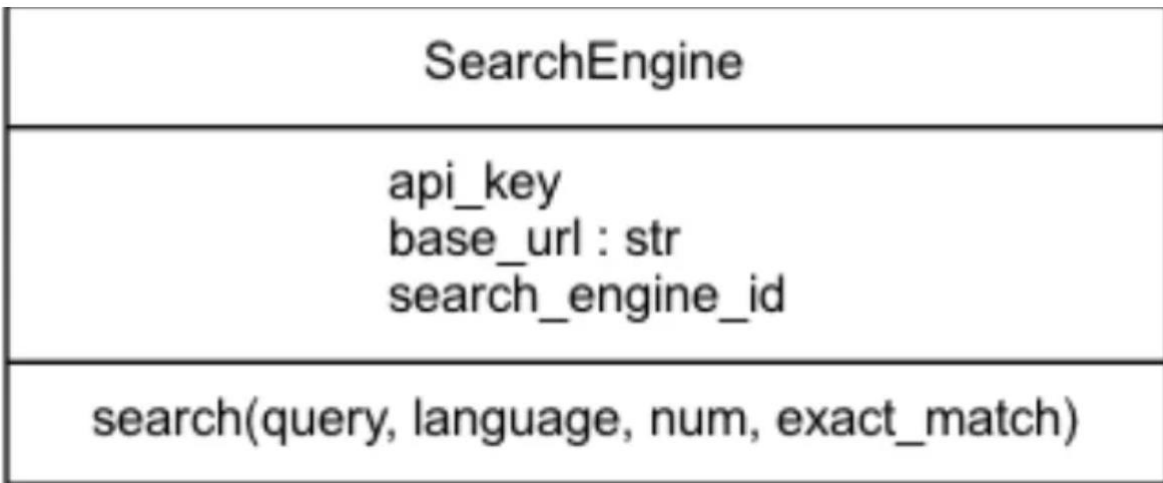
Library/Tool	Purpose
newsplease	Library used to scrape news articles from given URLs and extract metadata and text.
pprint	(Imported but unused) Normally used for pretty-printing Python objects for debugging.
datetime (via NewsPlease)	Used implicitly for formatting dates from article metadata.

Attributes (Currently Deprecated)

Attribute Name	Type	Description
NEUTRAL	int	A constant used to initialize the default vote type for articles. Value is 0. (Deprecated)

Main Methods

Method Name	Description
<code>init()</code>	Initializes the class. As the class only contains static methods, this is effectively a no-op.
<code>scrape_article(url)</code>	Scrapes an individual article from a URL using NewsPlease, returning a dictionary containing structured metadata and content such as title, author, date, etc.
<code>scrape_multiple_articles(urls)</code>	Scrapes multiple articles concurrently from a list of URLs using <code>NewsPlease.from_urls()</code> and returns a list of article dictionaries.
<code>decode_article_obj(article)</code>	Converts a <code>NewsArticle</code> object returned by <code>NewsPlease</code> into a standardized dictionary format with fields such as title, author, publish date, and more. Useful for further formatting or storage.

e. SearchEngine*Figure 31: Class Description of the SearchEngine***Purpose**

The SearchEngine module enables querying news articles using Google's Custom Search API.

It abstracts away the HTTP request and query formatting, allowing users to perform simple or advanced search queries and retrieve relevant article metadata (e.g., titles and links).

This module abstracts the complexity of querying and ranking results, ensuring consistent and relevant outputs for downstream modules (e.g., evidence extraction, summarization).

Dependencies

Library/Tool	Purpose
httpx	Performs HTTP requests to the Google Custom Search API endpoint, offering async and sync capabilities with better performance than requests.
Google Custom Search API	External API service used to fetch search results based on keyword queries, configurable by engine ID and API key.

Attributes

Attribute Name	Type	Description
api_key	str	Your Google Custom Search API key. Required to authenticate and make queries.
search_engine_id	str	The custom search engine ID (CX) that scopes the search results.
base_url	str	Default API URL for Google's Custom Search. Defaults to " https://www.googleapis.com/customsearch/v1 ".

Main Methods

Method Name	Description
init (api_key, search_engine_id, base_url)	Initializes the SearchEngine with API key, search engine ID, and optionally a base URL.
search (query, language="nil", num=10, exact_match=False, **extra_params)	Sends a search request to Google's Custom Search API. Returns a list of dictionaries with title and link of the found articles. Supports language filters, result count, exact match option, and other API parameters.

f. TextCleaning

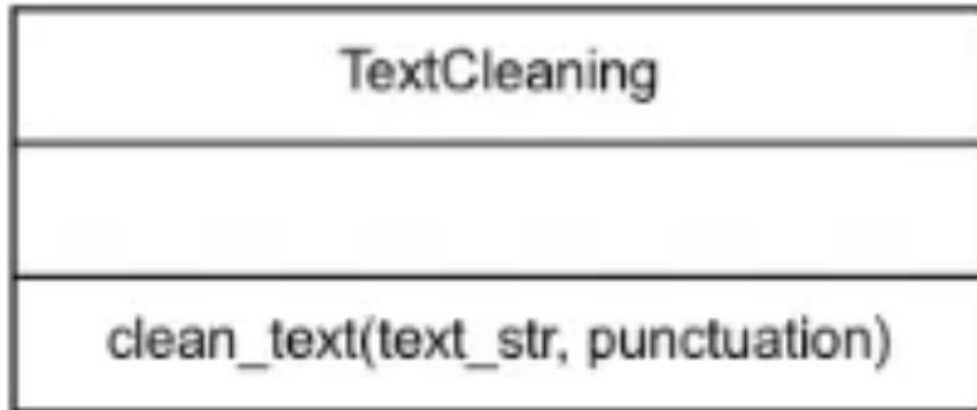


Figure 32: `TextCleaning` objects description

Purpose

The TextCleaning module provides simple text preprocessing functionality.

It focuses on normalizing whitespace, optionally removing punctuation, and cleaning extra spacing in input strings.

This is a lightweight yet essential module that supports upstream modules & service as summarizers, extractors, and analyzers by preparing clean input text.

Dependencies

Library/Tool	Purpose
textacy.preprocessing	Provides easy-to-use text normalization and cleaning utilities (e.g., whitespace and punctuation handling).
regex (re)	Enables advanced regular expression matching and replacement for additional fine-grained cleaning tasks.

Attributes (not exist)

Main Methods

Method Name	Description
<code>init()</code>	Initializes the TextCleaning class. No parameters or internal state are set.
<code>clean_text(text_str, punctuation=True)</code>	Cleans the given input text string by normalizing whitespace, optionally removing punctuation, and reducing redundant spacing.

3.11.1.2: QuantAna (Quantitative Analysis)

The QuantAna module provides quantitative analysis tools for evaluating and validating articles and their content, especially in the context of fact-checking and

summarization. This module contains two primary submodules: **QuantAnaIns** and **QuantAnaInsAlbert**.

Both modules focus on semantic similarity, tokenization, embedding-based analysis, and sentiment tasks to provide detailed assessments of text reliability and meaning alignment.

QuantAnaIns
HF_MODEL_ID_FEATURE_EXTRACT : str HF_MODEL_ID_SENTIMENT : str HF_TASK_FEATURE : str HF_TASK_SENTIMENT : str endpoint_name : str model : AsyncPredictor, NoneType model_name : str rdrsegmenter : VnCoreNLP role runtime : sentiment_endpoint_name : str sm_client : translator_model : GenerativeModel
combine_tokens(tokens) compute_multi_semantic_similarity(source_articles, query_article, display_table) compute_semantic_similarity(article1, article2) detect_toxicity(article_text: str) generative_extractive(article_text) normalize_result(value) sentiment_analysis(article_text) terminate() translation_from_Vie_to_Eng(text: str)

a. QuantAnaIns

Figure 33: QuantAnaIns class description

Purpose:

The QuantAnaIns class enables natural language processing for Vietnamese and multilingual text - which provide the implementation for SearchService and AnalysisService.

This version was for hosting the model on AWS Sagemaker

Dependencies

Library / Tool	Purpose
boto3	AWS SDK to interact with SageMaker (deploy and query endpoints).
sagemaker.huggingface	Simplified deployment of HuggingFace models to SageMaker.
google.generativeai	Gemini Pro API for text generation and translation.
scikit-learn	Cosine similarity computation (cosine_similarity).
vncorenlp	Vietnamese word segmentation.
transformers	Model IDs and configurations for HuggingFace inference.
json , os	Configuration and environment handling.
numpy , torch	Backend computations for NLP outputs.

Attributes (Fields)

Attribute Name	Type	Description
device	str	Specifies the device for model computation (e.g., "cpu", "cuda").
sentiment_pipeline	Pipeline	A Hugging Face pipeline that handles sentiment analysis tasks.
toxicity_model	Detoxify	A model used for detecting toxic content in the article.
embedding_model	ALBERT	A pre-trained ALBERT model used to generate sentence embeddings for semantic similarity.

segmentation_model	VnCoreNLP	A tool used for Vietnamese text segmentation, splitting the text into tokens.
translation_model	Translation	A model used for translating Vietnamese text to English for accurate toxicity analysis.

Main Methods

Method Name	Description
<i>init()</i>	<i>Initializes model configs, sets up SageMaker roles, and deploys endpoints if necessary.</i>
<i>_set_up_vncorenlp()</i>	<i>Loads and configures the VnCoreNLP segmenter.</i>
<i>_set_up_sagemaker_role()</i>	<i>Retrieves or creates the IAM execution role for SageMaker.</i>
<i>_check_endpoint_exists()</i>	<i>Checks if a given SageMaker endpoint already exists and is active.</i>
<i>_check_model_exists()</i>	<i>Checks if a model with the given name is registered in SageMaker.</i>
<i>_deploy_endpoint()</i>	<i>Deploys a HuggingFace model to a SageMaker endpoint.</i>
<i>compute_semantic_similarity()</i>	<i>Computes cosine similarity between two articles using sentence embeddings.</i>
<i>compute_multi_semantic_similarity()</i>	<i>Computes semantic similarity between multiple articles, optionally with respect to a query article.</i>

<i>generative_extractive()</i>	<i>Uses Gemini model to extract and reorder the most important sentences from an article.</i>
<i>sentiment_analysis()</i>	<i>Classifies sentiment of a given article using a multilingual sentiment model on SageMaker.</i>
<i>detect_toxicity(article_text: str)</i>	<i>Detects toxicity and misinformation in the article.</i>
<i>translation_from_Vie_to_Eng()</i>	<i>Generate a clear and concise translation from Vietnamese text to English for Toxicity Analysis</i>
<i>combine_tokens()</i>	<i>Combines tokens segmented by VnCoreNLP into coherent Vietnamese text.</i>

b. QuantAna_albert

QuantAnaInsAlbert
device : str embed_model embed_model_name : str embed_tokenizer : PreTrainedTokenizerFast, bool model_name : str rdrsegmenter : VnCoreNLP sentiment_pipeline toxicity_model : Detoxify translator_model : GenerativeModel
combine_tokens(tokens) compute_multi_semantic_similarity(source_articles, query_article, display_table) compute_semantic_similarity(article1: str, article2: str): float detect_toxicity(article_text: str) find_model_folder(model_name, root_model_dir) find_model_folder_checkpoint_keyword(keyword, root_model_dir) generative_extractive(article_text) get_embeddings(texts: List[str]) normalize_result(value) obsence_check(query) sentiment_analysis(article_text) translation_from_Vie_to_Eng(text: str)

Figure 34: QuantAnaInsAlbert class description

Purpose:

- This is the specialized version of the QuantAnaIns submodule, using **Albert-based** models for lightweight & cost-effective (hosting on the local enviroment), while maintaining effectiveness
- This module includes the QuantAnaInsAlbert class with the description below

Dependencies:

Library/Tool	Purpose
PyTorch	Used for deep learning model implementation and inference.
Hugging Face Transformers	Provides pre-trained models for sentiment analysis and semantic similarity (ALBERT).
Detoxify	Toxicity detection library that classifies content into various categories of toxicity (insults, threats, etc.).
VnCoreNLP	Tool for Vietnamese text segmentation and tokenization.
Google Generative AI	Used for extractive summarization tasks to generate concise summaries of articles.

Attributes:

Attribute Name	Type	Description
device	str	Specifies the device for model computation (e.g., "cpu", "cuda").
sentiment_pipeline	Pipeline	A Hugging Face pipeline that handles sentiment analysis tasks.
toxicity_model	Detoxify	A model used for detecting toxic content in the article.
embedding_model	ALBERT	A pre-trained ALBERT model used to generate sentence embeddings for semantic similarity.
segmentation_model	VnCoreNLP	A tool used for Vietnamese text segmentation, splitting the text into tokens.
translation_model	Translation	A model used for translating Vietnamese text to English for accurate toxicity analysis.

Main Methods:

Method Name	Description
sentiment_analysis(text)	Analyzes the sentiment of the provided text, categorizing it into positive, negative, or neutral.
detect_toxicity(text)	Detects various forms of toxicity (e.g., insults, threats, identity attacks) in the given text.
compute_semantic_similarity(text1, text2)	Calculates and returns the semantic similarity between two pieces of text using ALBERT embeddings.

	Computes the semantic similarity between
compute_multi_semantic_similarity(texts)	multiple texts, returning a matrix of similarity scores.
	Generates an extractive summary of the article by extracting the most important sentences using a
generative_extractive(text)	generative model. (We use the Gemini API for efficiency)
	Combines tokenized text back into a readable string by removing unnecessary characters.
combine_tokens(tokens)	
	Generate a clear and concise translation from Vietnamese text to English for Toxicity Analysis
translation_from_Vie_to_Eng(text)	
	Check the obscene level of the query - used by
obsence_check(query):	SearchService to filter out unsafe search-query

3.10.1.3: QualAna (Qualitative Analysis)

The **QualAna** (Qualitative Analysis) module is designed to perform **automated fact-checking and bias analysis** on Vietnamese news articles.

It integrates advanced web search, evidence retrieval, natural language analysis, and bias detection techniques to evaluate the credibility and objectivity of an article.

By leveraging search-based verification and context-aware ranking of external sources, the module can assess how factual and neutral an article is.

The Module includes 2 submodules: QualAna & ArticleFactCheckUtility

a. QualAna module

Purpose

The QualAnaIns class is the central interface for conducting qualitative analysis on news articles, specifically focused on **fact-checking** and **bias detection**.

It integrates multiple utilities to search for evidence, assess truthfulness, and analyze the tone or slant in the article.

The QualAna contains the class QualAnaIns.

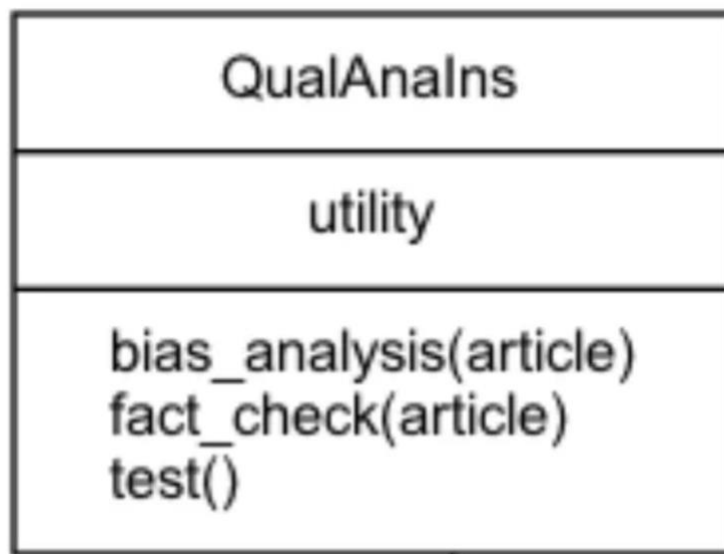


Figure 35: QualAnaIns Class Description

Dependencies

Library/Tool	Purpose
ArticleFactCheckUtility	Handles querying, web search, evidence filtering, and credibility analysis.
TextCleaning (from Textacy)	Performs text normalization and punctuation cleaning for better analysis.
requests	Used internally in utility classes for fetching content from URLs.
httpx	Used in web search within utility class to query external APIs.
BeautifulSoup	Scrapes webpage metadata (e.g., titles) for article enhancement.

Attributes

Attribute Name	Type	Description
utility	ArticleFactCheckUtility	Core utility class for processing, searching, filtering, and analyzing articles.

Main Methods

Method	Description
Name	
fact_check()	Takes a raw article, generates a fact-checking query, retrieves external evidence, ranks them, and returns an analysis.
bias_analysis()	Performs bias detection on the given article text and returns a human-readable explanation of detected bias.
test()	A test/demo method for running built-in examples through the fact_check or bias_analysis methods.

b. ArticleFactCheckUtility

ArticleFactCheckUtility
article_scraper model : GenerativeModel model_name : str search_engine tc
<pre> analyze_evidence(statement, evidence) choose_the_batch_size(article_list) fact_check_article_using_query(article_text) filter_rank(query, valid_articles) generate_bias_analysis(article: str) generate_brief_descriptions_batch(articles: List[str]): List[str] generate_search_queries(statement) generate_tags(article, predefined_tags) generate_tags_batch(articles: List[str], predefined_tags: Optional[List[str]]): List[List[str]] main() process_articles_in_batches(articles: List[str], predefined_tags: Optional[List[str]], batch_size: int) process_single_article(article) scrape_articles_parallel(articles, batch_size) search_web(query, num_results) search_web_fast(query, num_results) synthesize_and_summarize(query, reasoning, evidence) understanding_the_question(query) </pre>

Purpose:

Figure 36: The ArticleFactCheckUtility class description

This was made for support the QualAna module in providing the implementation of the Fact-Check analysis + adding some features such as question & answer responses generate (for deeper & insightful fact-check result)

The Submodule do contain a class share the same name: ArticleFactCheckUtility

Dependencies

Library/Tool	Purpose
Google Gemini	AI model used to generate responses for reasoning and synthesis based on queries.
VnCoreNLP	NLP tool for tokenization, part-of-speech tagging, and dependency parsing on Vietnamese text.
TfidfVectorizer (from sklearn)	For transforming text data into numerical vectors based on term frequency-inverse document frequency (TF-IDF) and computing cosine similarities.
cosine_similarity (from sklearn)	Computes cosine similarity between text vectors to rank articles based on their relevance to a query.
time	Used to implement rate limiting by introducing delays between batch processing.

Attributes

Attribute Name	Type	Description
model	Google Gemini or a similar model object	An instance of the model used for generating reasoning and synthesized responses.
query	str	A string representing the user query to be analyzed.
reasoning	str or None	A string representing the reasoning generated for the query, or None if reasoning is not available.
evidence	list[str]	A list of strings representing the evidence to support the reasoning and conclusions for the query.
valid_articles	list[str]	A list of valid articles that are relevant to the query, to be used for ranking and generating tags.
predefined_tags	list[str], optional	An optional list of predefined tags to be included when generating tags for articles.
article_text	str	The text of an individual article to be analyzed for tags, summaries, and descriptions.

Main Methods

Method Name	Description
generate_search_queries	Generates search queries based on the given statement to be used for further analysis or searching. (to return the critical reasoning questions for analysis the topic)
fact_check_article_using_query	Uses a generated search query to fact-check an article by cross-referencing with web search results.
search_web	Searches the web using a given query and returns a predefined number of results for further analysis.
search_web_fast	A faster version of the search_web method for obtaining web results quickly by implement scrape_articles_parallel - for cost reducing of Gemini-API Quota.
scrape_articles_parallel	Scrapes articles from multiple sources in parallel, processing them in batches for efficiency.
process_single_article	Processes a single article for fact-checking, tagging, and analysis.
analyze_evidence	Analyzes evidence to determine if it supports, contradicts, or is neutral to the statement. (the query or the article of user)
generate_bias_analysis	Performs bias analysis on a given article to evaluate potential biases or slants in the information.

Method Name	Description
understanding_the_question	Analyzes the query to understand its intent, scope, and underlying requirements before proceeding with further processing. -Which return the reasoning process on the question
synthesize_and_summarize	This method takes the original user query, and the reasoning obtained from
understanding_the_question().	Uses Gemini to synthesize evidence (implicitly from its knowledge) and summarize it into a clear and concise answer, guided by the provided reasoning.
filter_rank	Ranks articles based on their relevance to the query using TF-IDF vectorization and cosine similarity for determining the most relevant content.
generate_tags	Generates tags for an article based on predefined tags or dynamic analysis of the article's content.
generate_tags_batch	Processes a batch of articles and generates tags for each article, either from predefined tags or generated dynamically. (Current set only 4 tags per-batches)
generate_brief_descriptions_batch	Produces brief descriptions (10-50 words) for each article in a batch, summarizing their main points.

Method Name	Description
process_articles_in_batches	Handles the processing of multiple articles in batches, ensuring rate limits are respected when interacting with the AI model to generate tags and summaries.
choose_the_batch_size	Dynamically decide the batch size based on the number of articles. Otherwise, process them in batches of 5.

3.10.1.4: Summarizer

This Module included the 2 versions of Summarizer: Summarizer_albert (for using the light-weight ALBERT model) & Summarizer (for hosting the model on the service as SageMaker AWS)

The Algorithm our team used to implement this module was suggested by the model of [\(Vuong et al., 2024\)](#) in producing the Abstractive Summarization (Discussed in Section 1.1.2 of the design)

The Summarizer provides the backbone of implementation for the SummarizerService for article summarization & article synthesis (later the project can use this as the package for implementing the Q&A system)

a. Summarizer_albert

Purpose

The Summarizer_albert sub-module is designed to summarize Vietnamese-language news articles using both **extractive** and **abstractive** approaches.

It uses ALBERT and TextRank for extractive summarization and the Gemini API for generating abstractive summaries. An optional fact-checking utility can be integrated to validate content using external sources.

The sub-module contains the class: SummarizerAlbert

Dependencies

Library/Tool	Purpose
transformers	Loads and runs the ALBERT model for encoding Vietnamese text
torch	Handles tensor operations and model inference
networkx	Constructs graphs and applies TextRank for sentence ranking
scikit-learn	Computes cosine similarity between sentence embeddings
vncorenlp	Segments Vietnamese text into sentences and performs POS tagging
google.generativeai	Accesses the Gemini API for abstractive summarization
VnCoreNLP	External tool for Vietnamese linguistic preprocessing

Library/Tool	Purpose
Gemini API	Cloud-based LLM used to generate coherent, fluent summaries
SentenceEncoder (custom class)	Encodes Vietnamese sentences into vectors using ALBERT
ArticleFactCheckUtility	Provides fact-checking and source credibility analysis
nlTK	

Attributes

Attribute Name	Type	Description
stopword_file	str	The file name of the stopwords list, default is "vietnamese-stopwords-dash.txt".
extractive_model	str	The Hugging Face model path for the extractive summarization model, default is "cservan/multilingual-albert-base-cased-32k".
task	str	Specifies the task for the model, default is 'feature-extraction'.
abstractive_model	str	The Hugging Face model path for the abstractive summarization model, default is <i>'gemini-2.0-flash-thinking-exp-01-21'</i> .

Attribute Name	Type	Description
stopword	set	A set of Vietnamese stopwords used to filter out common words from the input text. Default set of stopwords is hardcoded, but it can be extended from a custom stopwords file if available.
stopword_path	str	Path to the stopwords file.
resource	str	The name of the NLTK tokenizer resource ("punkt_tab") used for tokenization checks.
tokenizer	AutoTokenizer	The tokenizer object used for the extractive summarization model.
model	AutoModel	The extractive summarization model (ALBERT) loaded either from the Hugging Face model hub or from a local directory.
model_root	str	The root directory where the model files are stored locally ("/app/models/hub").
using_volume	bool	A boolean flag indicating whether local models are available or if the model needs to be downloaded from Hugging Face.
albert_folder	str	Path to the folder containing the local ALBERT model files, if they exist.

Attribute Name	Type	Description
abstractive_model	GenerativeModel	A generative AI model used for abstractive summarization. It is initialized with the provided abstractive_model string.
qa_utility	ArticleFactCheckUtility	An instance of the ArticleFactCheckUtility class used for fact-checking and verification.

Main Methods

Method Name	Description
init(...)	Initializes the summarizer and its components
find_model_folder(self, model_name, root_model_dir)	find and load the extractive summarizing model
summarize(self, text, num_sentences = 5)	Full summarization pipeline (extractive, optional Gemini, optional fact-checking)
text_normalize(self, text)	for basic text-normalization (currently set the text to all lower-case)
pre_processing(self, text)	To implement the text-normalization - do implement text_normalize, text_tokenization, and return a list of processed sentences.

Method Name	Description
<code>get_embeddings(self, texts: List[str])</code>	get the embedding (the numerical representation of the text by using extractive-model)
<code>calculate_similarity_matrix(self, embeddings)</code>	Measuring the similarity matrix (based on the embeddings of multiple-text segments)
<code>extractive_summary(self, sentences, num_sentences=3):</code>	Implement the extractive summary - current set at most max number of sentence = 3
<code>abstractive_summarize(self, extractive_sentences: list, word_limit: int = 150):</code>	using the extractive sentence on writing the abstract of the text (current length max = 150 words)
<code>summarize(self, text, num_sentences = 5):</code>	Combine the extractive & extractive pipeline to return the summarization (set the num_sentences for the extractive = 5 for more information being retrieve from source)
<code>process_article(self, article):</code>	Helper function to extract key sentences from a single article.
<code>multi_source_extractive(self, articles)</code>	Extract key sentences from multiple articles using parallel processing.
<code>multi_article_synthesis(self, articles, word_limit=200)</code>	Generate a synthesized summary from multiple articles. - Current word-limit = 200 words

Method Name	Description
<code>question_and_answer(self, query, evidence=None):</code>	Answer a question based on either provided evidence or by searching online. If evidence is provided, skip the search. Otherwise, perform the search.

b. Summarizer

This version of Summarizer - was designed to implement the Summarizer model (especially the extractive model on the Sage-Maker AWS system)

The Submodule contains the Summarizer class

Purpose

The Summarizer class is designed to summarize Vietnamese-language news articles using both **extractive** and **abstractive** approaches.

Implementation is shared with the SummarizerAlbert object as in the above description, with a change in the object configuration to optimize for the SageMaker deployment env

The sub-module contains the class: SummarizerAlbert

Dependencies

Library / Tool	Purpose
<code>init(...)</code>	Calling & Construct the Summariser model with defined configuration - include: <code>stopword_file</code> , <code>extractive_model</code> , task <for extractive model>, <code>instance_type</code> , <code>endpoint_name</code> , <code>abstractive_model</code>
<code>dotenv (load_dotenv)</code>	Loads environment variables from a <code>.env</code> file.
<code>google.generativeai (genai)</code>	Interacts with Google's Generative AI models (e.g., Gemini).
<code>torch</code>	Deep learning framework for tensors and neural networks.
<code>numpy</code>	Numerical operations and array handling.
<code>torch.nn.functional (F)</code>	Functional API for neural network operations.
<code>re</code>	Regular expressions for text processing.
<code>torch.nn.utils.rnn.pad_sequence</code>	Pads sequences to uniform length for batching.

Library / Tool	Purpose
<code>nltk.tokenize.sent_tokenize, word_tokenize</code>	Tokenizes text into sentences and words.
<code>nltk</code>	Toolkit for natural language processing.
<code>typing.List</code>	Type hinting for lists.
<code>networkx</code>	Graph-based algorithms, used for TextRank summarization.
<code>transformers (AutoModel, AutoTokenizer)</code>	Loads pre-trained models from Hugging Face.
<code>boto3</code>	AWS SDK for interacting with AWS services.
<code>sagemaker</code>	Manages and deploys ML models on AWS SageMaker.
<code>sagemaker.huggingface.HuggingFaceModel</code>	Deploys Hugging Face models on SageMaker.
<code>os</code>	Interacts with the file system and environment variables.
<code>json</code>	Parses and writes JSON data.
<code>concurrent.futures, ThreadPoolExecutor</code>	Enables concurrent execution with threads.

Library / Tool	Purpose
time	Time-related functions (e.g., sleep, timestamp). - use in parallel processing controler
ViFinanceCrawLib.QualAna.ArticleFactCheckUtility	Custom utility for article fact-checking.
queue	Thread-safe data queue for concurrency.
tqdm	Displays progress bars for loops.
subprocess	Runs shell commands from Python.

Attributes

Attribute Name	Type	Description
self.stopword	set	Set of Vietnamese stopwords, optionally extended from file.
self.endpoint_name	str	The name of the SageMaker endpoint for feature extraction.
self.runtime	boto3.client	Boto3 SageMaker runtime client used to invoke endpoints.
self.HF_MODEL_ID	str	Hugging Face model ID for extractive summarization.

Attribute Name	Type	Description
self.sm_client	boto3.client	Boto3 SageMaker client for managing models and endpoints.
self.role	str	AWS IAM role ARN for SageMaker execution.
self.HF_TASK	str <i>(conditionally set)</i>	The task for Hugging Face model, e.g., feature-extraction.
self.hub	dict <i>(conditionally set)</i>	Dictionary with model ID and task for HuggingFaceModel deployment.
self.huggingface_model	HuggingFaceModel <i>(optional)</i>	Hugging Face model object for SageMaker deployment.
self.model	sagemaker.Model <i>(optional)</i>	Deployed SageMaker model object (if endpoint created).
self.abstractive_model	genai.GenerativeModel	Google Generative AI model for abstractive summarization.
self.qa_utility	ArticleFactCheckUtility	Custom utility class for fact-checking and QA on news articles.

Main Methods

Method Name	Description
text_normalize	Performs basic text normalization (trimming and lowercasing).

Method Name	Description
pre_processing	Cleans and tokenizes text into sentences and tokens, applies normalization and stopwords filtering.
get_embeddings	Computes vector embeddings for a list of texts using a model endpoint with batch processing.
calculate_similarity_matrix	Builds a cosine similarity matrix from sentence embeddings using mean pooling.
extractive_summary	Generates an extractive summary using TextRank (via PageRank over a similarity graph).
abstractive_summarize	Produces a fluent, abstractive summary from extractive sentences using a language model (Gemini).
summarize	Orchestrates summarization of a single article by combining extractive and abstractive steps.
process_article	Preprocesses and summarizes a single article, returning structured extractive information.
multi_source_extractive	Processes multiple articles in parallel and extracts key sentences from each.
multi_article_synthesis	Synthesizes a single summary from multiple articles using their extractive content and Gemini.
question_and_answer	Answers a user question using either provided evidence or via web search and synthesis.

Method Name	Description
terminate	Terminates the inference endpoint for the summarization model (use with caution).

3.10.2: The Workflows and Algorithm

This section mentions the aspects of the algorithm including:

- Summarizer Module
 - Mention the algorithm of the Extractive Summarization, Abstractive Summarization, and whole Summarize Writing algorithm
- QualAna Module
 - Mention the algorithm of the fact-check and bias-analysis task
- QuantAna Module
 - Mention the algorithm of the sentiment_analysis; detect_toxicity

3.10.2.1. Summarizer module algorithm

This section will include the algorithms of Abstractive Summarization, Extractive Summarization, and the whole Summarization Algorithm that our team has used

a. The Algorithm of Extractive Summarization

The `extractive_summary` method implements an **extractive summarization algorithm** using **sentence embeddings**, a **similarity matrix**, and **PageRank** to select the most important sentences from a list.

The goal is to choose a subset of sentences that best represent the content of the original text.

High-Level Explanation

1. Input:

- `sentences`: A list of sentences from the input article or paragraph.
- `num_sentences`: The number of sentences to include in the summary (default is 3).

2. Output:

- A list of the top `num_sentences` that best represent the full content based on semantic similarity.

3. Main Idea:

- Compute semantic embeddings for each sentence.
- Build a similarity matrix using these embeddings.
- Use **PageRank** (a graph-based ranking algorithm) to score the importance of each sentence.
- Return the top `num_sentences` based on their PageRank scores.

Detailed Steps

1. Check Length Condition:

- If the input list of sentences is shorter than the number of requested summary sentences, return the original list and warn the user.

2. Get Sentence Embeddings in Parallel:

- Each sentence is converted into an embedding using the `self.get_embeddings()` method.
- A `ThreadPoolExecutor` is used to speed up the embedding process by computing them in parallel.

3. Prepare the Embedding Tensor:

- `pad_sequence` is used to create a uniform tensor shape from the list of embeddings so they can be processed in batch form.

4. Calculate Sentence Similarity:

- A pairwise similarity matrix is computed using the sentence embeddings. Each cell (i, j) represents the similarity between sentence i and j .

5. Apply Text-Rank: (a variation of [PageRank](#) algorithm)

- The similarity matrix is treated as a graph where each node is a sentence and edges represent similarity.
- The algorithm ranks sentences based on their connectivity in this graph.
- Our team refer to algorithm base on these resource: ([Gulati et al., 2023](#), [kunjjan6902, 2020](#))

6. Select Top Sentences:

- Sentences are ranked by their PageRank score, and the top `num_sentences` are selected.

- The selected indices are sorted to preserve the original order of appearance in the article.
-

Pseudocode

```
Function extractive_summary(sentences, num_sentences = 3):

    If length of sentences < num_sentences:

        Warn user

        Return sentences

    Parallely compute sentence embeddings:

        sentence_embeddings = [get_embeddings([sentence]) for sentence in sentences]

    Pad embeddings to make a tensor:

        sentence_tensor = pad_sequence(sentence_embeddings)

    Calculate similarity matrix:

        similarity_matrix = calculate_similarity_matrix(sentence_tensor)

    Build graph from similarity matrix:

        graph = build_graph(similarity_matrix)

    Compute PageRank scores for each sentence:
```



```
scores = pagerank(graph)
```

Rank sentences by `score`:

```
top_indices = indices of top num_sentences scores
```

Sort selected indices by original `order`:

```
sorted_indices = sort(top_indices)
```

Return sentences at `sorted_indices`

The Flow Chart

As described in Figure 30 on the next page

b. The Algorithm of Abstractive Summarization

The `abstractive_summarize` method uses a **large language model (LLM)**—in this case, **Gemini** is prompted to rewrite a summary in natural language, based on the most important sentences selected by an extractive summarizer.

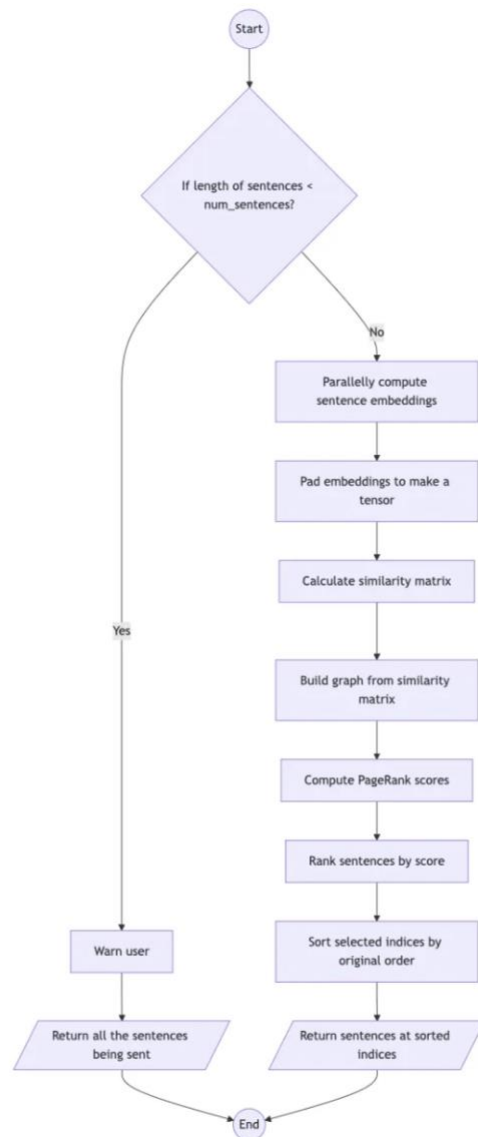


Figure 37: The Flow Chart Describe the Extractive Summarization Pipeline

Purpose

The goal of the abstractive summarizer is: To transform extractive key sentences into a coherent, concise, and natural summary that:

- *Rephrases (not just copies) the input sentences,*
 - *Preserves meaning,*
 - *Is cohesive and well-structured,*
 - *Adheres to a word limit. (we set the limit to about 150 to 200 words)*
-

High-Level Explanation

1. Input:

- `extractive_sentences`: A list of key sentences obtained from extractive summarization (i.e., the most relevant or central ideas from the original text).
- `word_limit`: The desired maximum number of words in the final summary (default is 150).

2. Output:

- A single-paragraph abstractive summary (`str`) that conveys the main points naturally and concisely.
- Returns a message if the input is empty, or `None` if an error occurs.

3. Main Idea:

- Construct a detailed prompt in Vietnamese that explains how to convert the extractive sentences into a fluent summary.
- Include sample input and output in the prompt to guide the generative model (Gemini).
- Feed the prompt into the Gemini model to generate a natural, rephrased, and cohesive paragraph.
- Return the generated summary as the final output.

Detailed Steps

1. Input Validation:

- If no extractive sentences are provided, return an informative message.

2. Prompt Engineering:

- A structured **Vietnamese-language prompt** is created.
- It provides:
 - A role definition for the AI model.
 - A list of extractive key sentences formatted with dashes.
 - Clear instructions (no bullets, no emojis, rephrase naturally).
 - An example to help the model learn how to respond.

3. Content Generation:

- The prompt is sent to the `abstractive_model` (a Gemini model).
- The response is parsed and returned as the final summary.

4. Error Handling:

- If the model fails or throws an exception, the function catches the error and returns `None`.

Pseudocode

```
Function abstractive_summarize(extractive_sentences, word_limit=150):
```

```
    If extractive_sentences is empty:
```

```
Return "No extractive sentences provided for summarization."
```

Construct a detailed natural language prompt:

- Include AI role
- Format extractive_sentences as bullet points
- Add clear instructions and example output
- Specify the desired word limit

Try:

Send prompt to the abstractive_model

Extract and return the cleaned summary text

Except if error occurs:

Print error

Return None

The prompt that the model has been instructed:

```
prompt = f"""
```

Bạn là một trợ lý AI chuyên tóm tắt văn bản.

Dưới đây là các câu then chốt được trích xuất từ văn bản gốc:

```
{" ".join(f" - {sentence}" for sentence in extractive_sentences)}
```

****Yêu cầu:****

1. Tóm tắt nội dung chính một cách tự nhiên, dễ hiểu, có tính liên kết.
2. Không lặp lại nguyên văn, hãy diễn đạt lại súc tích, giữ nguyên ý nghĩa.
3. Không cần tiêu đề, dấu gạch đầu dòng, emoji, hoặc bất kỳ tag nào.
4. Chỉ xuất ra một đoạn văn duy nhất, không kèm thêm phần mở đầu, tiêu đề hay kết luận.
5. Giới hạn độ dài tối đa khoảng {word_limit} từ.

****Ví dụ:****

Câu then chốt:

- Chủ đề A có tác động đáng kể đến lĩnh vực B.
- Nhiều chuyên gia cho rằng xu hướng C sẽ tiếp tục phát triển.

- Một số yếu tố bên ngoài như D cũng ảnh hưởng đến tình hình chung.

Tóm tắt mẫu:

Chủ đề A đang ảnh hưởng mạnh đến lĩnh vực B, đồng thời xu hướng C được kỳ vọng sẽ tiếp tục phát triển trong thời gian tới. Các yếu tố bên ngoài như D cũng góp phần định hình bối cảnh hiện tại.

Bây giờ, hãy áp dụng cách làm tương tự với các câu then chốt sau và chỉ trả về đoạn tóm tắt:

""

Flowchart

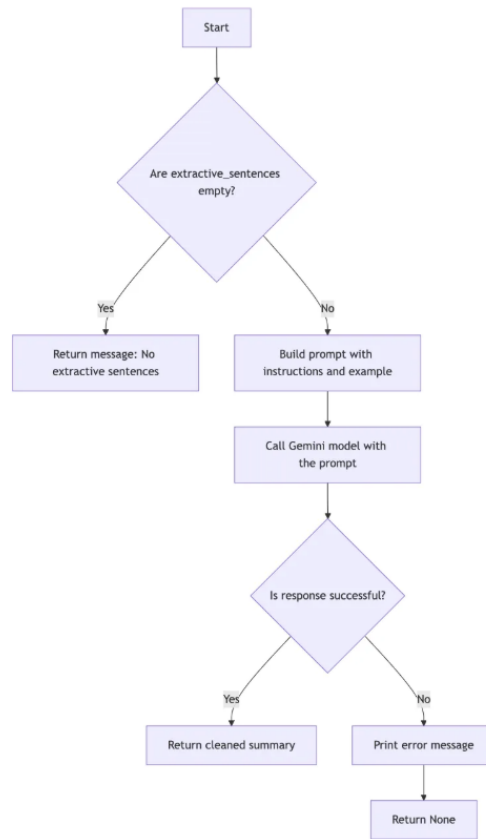


Figure 38: The flow-chart represents the abstractive summarization process

c. The Summarization Algorithm

Our pipeline of summarization has adapted from the idea of (Vuong, et al 2024) - including extractive summarization and abstractive summarization

However, we optimized the Abstractive Document Summarization by using the prompt-engineering technique as we mentioned in the Abstractive Summarization section - which reduced the burden in the post-processing section due to the outcome of the LLM return being constrained with predefined format.

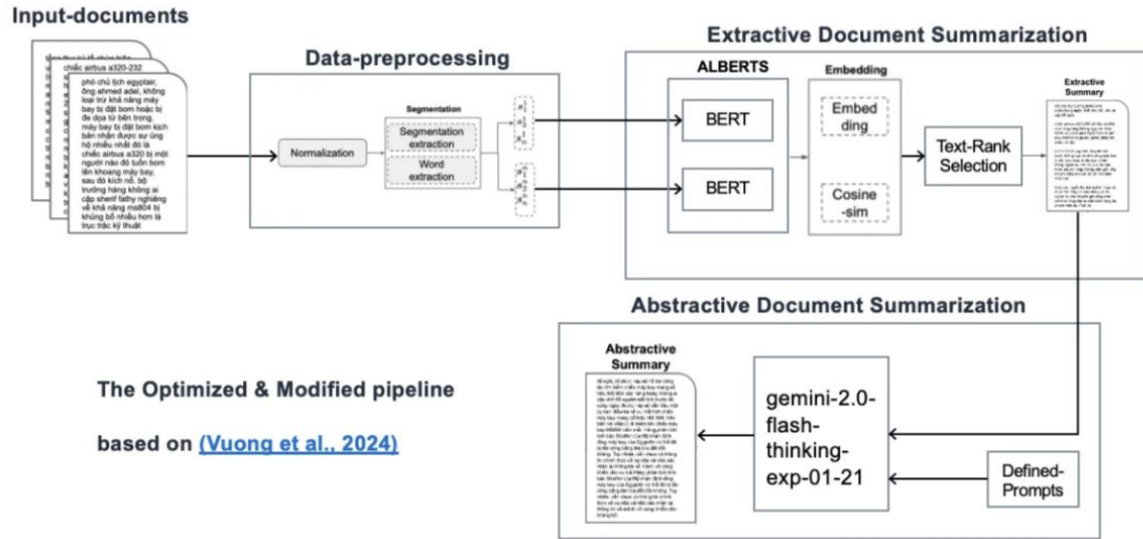


Figure 39: The whole pipeline of summarization

Pseudocode of the pipeline

```
function summarize_document(input_documents):

    # 1. Data Preprocessing

    normalized_texts = []

    for doc in input_documents:

        normalized = normalize(doc)

        segments = segment_text(normalized)    # Sentence segmentation

        words = extract_words(segments)        # Word segmentation if needed

        normalized_texts.append(words)
```

```
# Flatten all sentences from all documents
```

```
all_sentences = flatten_sentences(normalized_texts)
```

```
# 2. Extractive Document Summarization
```

```
embeddings = []
```

```
for sentence in all_sentences:
```

```
    embedding = get_bert_embedding(sentence, model="ALBERT") # ALBERTS embedding
```

```
    embeddings.append(embedding)
```

```
similarity_matrix = calculate_cosine_similarity(embeddings)
```

```
graph = build_graph(similarity_matrix)
```

```
pagerank_scores = compute_pagerank(graph)
```

```
top_sentences = select_top_sentences(all_sentences, pagerank_scores, top_k=5)
```

```
# 3. Abstractive Document Summarization (Using Gemini)
```

```
prompt = generate_prompt_from_sentences(top_sentences, word_limit=150)
```

```
summary = gemini_generate(prompt, model="gemini-2.0-flash-thinking-exp-01-21")  
  
return summary
```

3.10.2.2. QualAna Module Algorithm

This section would include the algorithm for the *fact-checking* task and *bias-analysis* task

a. fact-checking (/api/factchecking)

High-Level Explanation

1. Input

- article (str): The full content of a news article or claim that needs to be verified.

2. Output

- A fact-checking **analysis report** (typically a structured string or dict) that indicates:
 - Whether the article is **accurate**, **misleading**, or **biased**
 - Supporting evidence gathered from online sources

3. Main Idea

The algorithm performs **automated fact-checking** by:

- Generating a search query from the article.
- Searching the web for related information.
- Extracting and structuring snippets as **evidence**.
- Ranking this evidence by relevance.

- Analyzing and comparing the evidence with the article to produce a final **verdict or summary**.
-

Pseudocode

```
function fact_check(article):  
  
    # Step 1: Generate a fact-checking query based on the input article  
  
    query = fact_check_article_using_query(article)  
  
  
    # Step 2: Perform a fast web search using the query  
  
    search_results = search_web_fast(query)  
  
  
    # Step 3: Collect and structure all evidence from the search results  
  
    all_evidence = []  
  
    for result in search_results:  
  
        evidence_entry = format(  
  
            "Source: {title}\nAuthor: {author}\nURL: {url}\nSnippet: {main_text}",  
  
            result  
  
        )  
  
        all_evidence.append(evidence_entry)
```

```
# Step 4: Rank and filter the evidence based on its relevance and credibility
```

```
ranked_evidence = filter_rank(query, all_evidence)
```

```
# Step 5: Join evidence snippets for analysis
```

```
evidence_string = join(ranked_evidence, delimiter="\n\n")
```

```
# Step 6: Analyze the article against the selected evidence
```

```
analysis = analyze_evidence(article, evidence_string)
```

```
return analysis
```

Supporting Utility Methods

- `fact_check_article_using_query(article)`
→ Generates a concise, information-rich, bias-free search query from the article.
- `search_web_fast(query)`
→ Uses a fast web search (e.g., Google/Bing API or a scraping engine) to return relevant results.
- `filter_rank(query, valid_articles)`

→ Filters and ranks the search results using similarity scores, reliability, or other signals.

- `analyze_evidence(article, evidence_string)`

→ Compares the article to evidence and outputs a factual analysis or verdict.

→ The analysis was prompt engineering based on the guideline of NATO on measuring information reliability - check here: [NATO-Information Reliability Metric](#)

b. bias-analysis (/api/biasanalysis)

High-Level Explanation

1. Input:

- article (str): A news article or passage that needs to be assessed for **bias** and **logical fallacies**.

2. Output:

- A structured **bias report** generated by an LLM, including:
 - Detected **type of bias** (e.g., political, cultural, confirmation).
 - **Severity level** (e.g., light, moderate, severe).
 - A concise **contextual explanation**.
 - **3-5 Socratic questions** to guide the reader in critical reflection.

3. Main Idea:

- The algorithm leverages a **prompted large language model (LLM)** to examine the input article.
- The LLM is guided to avoid surface-level keyword detection and instead focus on **argument logic, rhetorical tone, and context**.

- The system produces a human-like **bias analysis** and reflective questions that promote objective reading.
-

Pseudocode

```
function bias_analysis(article):  
  
    # Step 1: Validate article content  
  
    if article is empty or whitespace:  
  
        return "⚠️ Bài viết không có nội dung. Vui lòng cung cấp bài viết hợp lệ để phân tích."  
  
    # Step 2: Call the utility method to generate bias analysis prompt and invoke LLM  
  
    analysis = utility.generate_bias_analysis(article)  
  
    return analysis
```

Supporting Utility Method

- **generate_bias_analysis(article: str)**

Purpose: Constructs a detailed prompt that instructs the LLM to:

- **Analyze bias and logical fallacies** in a nuanced way.
- Structure the response with labeled sections and questions.

Prompt Highlights:

- Acts as a **critical journalist** or debater.
- Evaluates **argument style, tone, and assumptions**.

- Explicitly avoids shallow keyword-based decisions.
- Ensures neutral articles are marked as such.
- Produces output with this structured format as below:

```
- Loại thiên kiến: [Bias Type]

- Mức độ ảnh hưởng: [Severity]

- Phân tích ngắn gọn: [Explanation]

Câu hỏi phản biện để giúp người đọc có góc nhìn khách quan hơn:

- [Socratic Question 1]

- [Socratic Question 2]

...
```

Pseudocode for generate_bias_analysis

```
function generate_bias_analysis(article):

    # Step 1: Build a context-aware prompt

    prompt = format("""

        Bạn là một nhà báo phân tích phản biện, chuyên đánh giá độ chính xác và khách quan của thông tin.

        Hãy phân tích bài viết sau để xác định các thiên kiến và lỗi lập luận có thể có...

        (đầy đủ nội dung prompt như yêu cầu trên)

        Bài viết cần phân tích:
```

```

"""{article}"""

"""

# Step 2: Call the LLM to generate content based on prompt

try:

    response = model.generate_content(prompt)

    if response is empty:

        return "⚠ Warning: Empty response from AI model."

    return response.text

except Exception as e:

    return f"✖ Error during bias analysis: {e}"

```

3.10.2.3. QuantAna Module Algorithm

This section will mention the algorithm and the pipeline been used for 2 tasks:

Sentiment Analysis and the Toxicity Analysis

a. Sentiment Analysis (/api/sentiment_analysis)

High-Level Explanation

1. Input:

- `article_text` (str): The raw text of an article to be analyzed for sentiment polarity.

2. Output:

- A dictionary with:
 - `sentiment_label`: The sentiment category (POSITIVE, NEGATIVE, NEUTRAL, etc.).
 - `sentiment_score`: The confidence score of the prediction (float between 0 and 1).
- Returns None if an error occurs during the analysis.

3. Main Idea:

- The method uses a **pretrained sentiment analysis pipeline** (likely from Hugging Face Transformers).
 - It first **cleans up** numbered prompts (e.g., [Câu 1]) to avoid noise in the analysis.
 - It then sends the article text to the `sentiment_pipeline`, which predicts sentiment using a fine-tuned transformer model.
 - The most confident prediction is returned with its label and score.
-

Pseudocode

```
function sentiment_analysis(article_text):
```

```
    # Step 1: Clean article text by removing numbered markers like [Câu 1]
```

```
article_text = remove_numbered_tags(article_text)

# Step 2: Run text through the sentiment analysis pipeline

try:

    result = sentiment_pipeline(article_text)

    top_result = result[0] # Take the highest confidence label

    sentiment_label = top_result["label"]

    sentiment_score = top_result["score"]

    return {

        "sentiment_label": sentiment_label,

        "sentiment_score": sentiment_score

    }

except Exception as e:

    print_error("Sentiment Analysis Failed", e)

    return None
```

Supporting Utility Component

self.sentiment_pipeline

- A **preloaded pipeline** typically built from `transformers.pipeline("sentiment-analysis")`.
- It wraps a **pretrained transformer model** (ALBERT) fine-tuned on sentiment classification tasks.
- Automatically:
 - Tokenizes input text.
 - Runs inference.
 - Returns the most likely sentiment labels and associated confidence scores.

generative_extractive (QuantAnaInsAlbert)

- Connect to the Gemini-API to implement extractive summarization with 10 sentences to reduce the level of computation from the original output
- Automatically:
 - Return the sentences with the format as follows: (need to post-processed in the analysis phase)

```
### **Định dạng đầu ra (một đoạn văn chứa 10 câu nguyên văn):**  
  
"[Câu 1] [Câu 2] [Câu 3] ... [Câu 10]"
```

b. Toxicity Analysis (/api/toxicity_analysis)

High-Level Explanation

1. Input:

- `article_text (str)`: A Vietnamese-language article that needs to be checked for toxicity, insults, threats, identity attacks, and obscene content.

2. Output:

- A dictionary containing toxicity scores for five categories:
 - "Tính Độc Hại" (Toxicity)
 - "Tính Xúc Phạm" (Insult)
 - "Tính Đe Doạ" (Threat)
 - "Công kích danh tính" (Identity Attack)
 - "Mức Độ Thô Tục" (Obscene)
- Each value is a normalized float score between 0 and 1.
- If an error occurs, a dictionary with an "error" key and error "details" is returned.

3. Main Idea:

- This algorithm performs **Vietnamese-to-English translation** to enable compatibility with an English-trained **toxicity detection model**.
 - It uses **tokenization and preprocessing** to prepare the Vietnamese input text.
 - The text is translated, then passed to a **pretrained multi-label classification model** to predict various forms of toxic content.
 - Results are postprocessed and returned with Vietnamese labels for interpretation.
-

Pseudocode

```
function detect_toxicity(article_text):  
  
    try:  
  
        # Step 1: Vietnamese sentence segmentation  
  
        tokenized_text = rdrsegmenter.tokenize(article_text)  
  
  
        # Step 2: Reconstruct full sentences from tokenized output  
  
        processed_text = combine_tokens(tokenized_text[0])  
  
  
        # Step 3: Translate Vietnamese to English  
  
        translated_text = translation_from_Vie_to_Eng(processed_text)  
  
  
        # Step 4: Predict toxicity scores using pre-trained English model  
  
        toxicity_scores = toxicity_model.predict(translated_text)  
  
  
        # Step 5: Normalize and map scores to Vietnamese labels  
  
    return {  
  
        "Tính Độc Hại": normalize_result(toxicity_scores["toxicity"]),
```



```
"Tính Xúc Phạm": normalize_result(toxicity_scores["insult"]),

"Tính Đe Doạ": normalize_result(toxicity_scores["threat"]),

"Công kích danh tính":    normalize_result(toxicity_scores["identity_attack"]),

"Mức Độ Thô Tục": normalize_result(toxicity_scores["obscene"])

}

except Exception as e:

    print("An error occurred:", e)

    traceback.print_exc()

    return {

        "error": "An error occurred while detecting toxicity",

        "details": str(e)

    }
```

Supporting Utility Methods

Method Name	Description
<code>rdrsegmenter.tokenize(text)</code>	Performs Vietnamese sentence segmentation (word-level tokenization).
<code>combine_tokens(tokens: List[str])</code>	Reconstructs full sentences from the tokenized words.
<code>translation_from_Vie_to_Eng(text: str)</code>	Translates preprocessed Vietnamese text to English.
<code>toxicity_model.predict(text: str)</code>	Applies a pretrained multi-label toxicity classifier (e.g., Detoxify).
<code>normalize_result(score: float)</code>	Normalizes raw scores (e.g., rounding or thresholding to 0–1 range).

4. Contribution

Really thankful for these members (as list in the table with their email in the hyperlink) for contributing in ViFinanceNews project – your contribution are essential to the success of our product.

Team's Member	Designing & Implementation Contribution
Tran Tien Dat	• Major ViFinanceCrawLib contributor (QuantAna Module, QualAna, article_database, Summarizer) • Summarization Algorithm • SummariserService • SearchService (the get_articles() method) • get_save_list algorithm • Architectural Design • Supporting User-Interface Designer & Consultant
Nguyen Hai An	• Supporting ViFinanceCrawLib contributor (article_database) • SearchService (voting mechanism) • LoggingService

<u>Nguyen Kim Tuan Cuong</u>	• Supporting ViFinanceCrawlLib contributor (QuantAna & QualAna submodule) • Extractive-Summarization algorithm contribution • Tester • Major User Interface Designer & Consultant
<u>Nguyen Khanh Viet Dung</u>	• Front-end Designer • ViFinanceCrawlLib & ViFinanceCommon consultant • Tester • Supporting U.I Designer & U.I Consultant
<u>Nguyen Duy Duc</u>	• Major ViFinanceCommon Designer • AuthService • UserService • Prototype implementers

ViFinanceNews

Group 2 – ViFinanceNews Team

Dat Tran Tien

Nguyen Kim Tuan Cuong

Nguyen Khanh Viet Dung

Nguyen Hai An

Nguyen Duy Duc

Software Feasibility Study Documentation

Version: (1.0.0) Date: (May/26/2025)

Introduction of this Documentation

This documentation is the third part of ViFinanceNews project documentation – which focusing on the feasibility of our project from the perspective of Software Market, the Human Resource and potential and essential technical stack that our team can leverage to deliver the project with the most attainable and qualifiable product.

The documentation includes 6 essential sections:

1. The Project Overview
2. Market Feasibility
3. Technical Feasibility & Financial Feasibility
4. Organizational & Managerial Feasibility
5. Project Timeline

With the section project timeline – in this version of Feasibility Study we had updated the official launch & demonstration of the system

TABLE OF CONTENT

<u>Introduction of this Documentation</u>	198
<u>1. Project Overview</u>	200
<u>2. Market Feasibility</u>	200
<u>3. Technical Feasibility & Financial Feasibility</u>	202
<u>4. Organizational & Managerial Feasibility</u>	213
<u>5. Project Timeline</u>	214

Feasibility Study Documentation (Version 1.0.0) – Updated in May, 26,2025

(For Web App Development in the Vietnamese Market)

1. Project Overview

Project Name: ViFinanceNews

Project Description:

An Intelligent Application working for Vietnamese article readers in the Economic & financial field.

Target Audience:

- ☐ Adolescents (13–17)
- ☐ Teenagers (18–24)
- ☐ Millennials (25–40)
- ☐ Elder Users (40+)

Key Features:

- ☐ Personalized Content ☐ User-Friendly UI
- ☐ Accessibility Options ☐ AI-Based Recommendations
- ☐ AI-Based Bias & Content Analysis ☐ Connect to the Reliable news-source
- ☐ Availability with affordable price (under 50k / month)

2. Market Feasibility

Industry Overview: (Describe digital media trends in Vietnam.)

- Vietnamese article readers are on the rise & major consuming & information searching by using social media & E-news publishers.

- However, the problem with Vietnamese readers is their poor capability in filtering information, detecting bias, and the easy to believe in fake news & sensations & hyping from labels or markets, especially with millennial & elder readers.
- The information from Vietnamese is rich – which can be troublesome to the reader because of information overload -> needs an assistance tool to support.
- The free & open-source model A. I developed for the Vietnamese market.

Competitor Analysis

- **ChatGPT:** Provides AI-powered search and content generation but requires a paid subscription for optimal analysis. It is not specifically tailored for Vietnamese news and finance-related content.

- **Ground.News:** Analyzes media bias and political orientation but is designed for the American market, with no dedicated support for Vietnamese-language sources.

- **Elicit:** An AI-powered search tool for scholarly articles, focused on academic research rather than mainstream news consumption.

- **Google Search Tool:** An currently has the Generative AI that able to summarize for the user on returned results based on user's query, but a massive amount of data can dilute the result and not support on Vietnamese community

How ViFinanceNews Differentiates

- **Vietnamese Market Optimization:** Unlike competitors, ViFinanceNews is built specifically for Vietnamese-language content, ensuring better contextual understanding and relevance.
- **Customized Machine Learning Model:** Tailored to Vietnam's information demographics, reducing irrelevant and explicit content.
- **Focused on Economist & Finance News:** Unlike broad AI search tools, it is optimized for financial and news-related data.
- **Open-Source & Public Accessibility:** Unlike Google AI Search and premium ChatGPT, ViFinanceNews is open-source, making it accessible for both users and the research community.

3. Technical Feasibility & Financial Feasibility

Technology Stack:

- **Frontend:**

- ☐ React, Node.js

- ☐ RestAPIs

- ☐ Python

- ☐ Java

- **Backend:**

- ☐ Flask

☐ Node.js

☐ Other: _____

• **Database:**

☐ Redis

☐ AzureSQL

☐ PostgreSQL (Offer with CockroachDB)

• **AI/ML Usage:**

☐ TensorFlow

☐ Hugging Face

☐ SageMaker AI

☐ Other: _____

• **Deployment & Security:**

☐ Docker

☐ Google Kubernetes Engine

☐ GitHub

☐ HashiCorp (Key & Secret Encryption Vault)

☐ Linode Virtual Machine

Infrastructure Needs:

• Hosting Services

Server: Deploying the frontend and backend

Option	Price
Vercel + Render (1)	0\$ (with limited features)
AWS (2)	Free Tier for first years
Railway (3)	8 GB RAM / 8 vCPU for \$5 monthly
DigitalOcean (4)	Student plan (1 year)
CloudFlare (5)	Free
Using V.M on Linode (6)	Linux VM for hosting the Docker-containerized application with 2 possible options: Shared CPU Instance (\$5/month) – for low demand application Dedicated CPU instance (\$36/month)

Estimated Price:

0\$/Month if using option 1,5

2-3 can be in-short-term (first year)

Option 3 takes 5\$/month

Option 6 with 2 possible options based on [Linode's price-tier](#)

Option 1: Shared CPU Instance

- \$5/month (\$0.0075/hour)
- 1 GB RAM, 1 Shared vCPU
- 25 GB SSD Storage
- Includes 1 TB transfer
- Best for:
 - Low-traffic apps or MVPs
 - Dev/testing environments
 - Personal projects or student demos
 - CPU is shared with others → Performance may vary if neighbors use too much CPU

Option 2: Dedicated CPU Instance

- \$36/month (\$0.05/hour)
- 4 GB RAM, 2 Dedicated vCPUs
- 80 GB SSD Storage
- Includes 4 TB transfer
- Best for:
 - Production apps or APIs
 - CPU-intensive Docker workloads
 - Hosting multiple containers
 - Guaranteed performance – full, constant access to CPU cores
- For the Dev & Initial step of development with low demand from customers, we could implement the 1st option, and when demand was higher the 2nd option would be preferable

• **Cloud Database (Redis Cache, Azure SQL, CockroachDB): (include the Database & Cache Memory)**

- Redis-Cache (0\$)

Free-Tier on UpStash: (per Month)

500K Commands/Month (include read & write)

50GB of BandWidth

256MB of storage

No Back-Up

10.000 Command/Sec

1MB request-size

- Azure-SQL (Free-per-month – but would take-price if exceed the Quota)

Free-Tier: (0\$)

100K vCores Seconds (vCores == Virtual CPU) = 27.78 vCore Hours = 1.16 vCore-days (1 days & 4 Hours)

32 GB of Storage

The Priced Quota:

Minimum vCores: \$0.378/vCore-hour

Price/Month: **4.78 USD (for exceeding quota)**

Acceptable price range (max: 10\$)

- CockroachDB (Free-per-month – but would take price if exceeds the Quota & Do offer

400\$ first of free-credit with Student Email = 160 Million of Request Units - RUs)

Free-Tier: (0\$/Month)

Provide 50 Million RUs (with the metrics of 500 Rus = 1VCPU)

Storage: first 10GB of free Data-Storage

Cloud Availability: Using AWS or GCP (Google Cloud Platform)

99.99% of Availability

- **Model Hosting Service: AWS SageMaker Real-Time Inference**

Configuration: using ml.t2.medium (hosting for 30 days/month, 9 hours (8 to 17))

Memory: 4GiB

Network performance: Low to moderate,

Cost: 1 model & 1 end-point. - 0.056 USD per hour instance cost

Estimated Cost: \$182 (for 12 months, 270h/months)

Service Needed to: Host Extractive Summary Inference (ViDeBERTa model)

Estimated Monthly Cost: \$15.12

- **Model Hosting Service: Viettel Object Cloud Storage (for storing the model file in S3-bucket)**

Configuration: (base configuration with small-web app)

Memory: 50GiB

Inbound data traffic (GB): 250GB

Outbound data traffic (GB): 250GB

Number of Query 51.840.000/Month

Estimated Monthly Cost: 33.000 VND = 1.27\$ USD

- **Application Logging Service:** App-Insight on Azure

Estimated Cost: \$0 & Unlimited with Education Service

Service Needed to: Tracking the Application behavior & Error Message

Estimated Monthly Cost: \$0

- **File Storing Service:** AWS S3 (Simple Storage) – for user Avatar's (Profile picture)

Estimated Cost: (This is used with Standard Tier that is priced based on the [AWS announcement](#))

- First 50 TB/Month: \$0.023/GB of data
- Next 450 TB/Month: \$0.022\$ /GB of Data
- Over 500 TB/Month: \$0.021\$ /GB of Data

Data Request Price using HTML command:

\$0.005/ 1000 Requests (PUT, COPY, POST, LIST)

\$0.0004/ 1000 Requests (GET, SELECT)

Estimated Monthly Cost: Under 10\$ - about 440 GB of Data (Data-Cleaning plan can be implemented later to manage the use of Data)

- **Search Engine Service: Custom-Search API on Google Cloud Console**

Estimated Cost:

\$0 = 100 Search Query/Days for Free

\$5 per 1000 queries, up to 10k queries per day. (if exceed the free-tier)

Service Needed to:

Estimated Monthly Cost: \$0 (ideal using rate)

- **Generative Model API: Google AI Studio API) [Updated in 17 May 2025]**

Model using: gemini-2.0-flash-thinking-exp-01-21 (Experimentation Model)

Free-Tier do offer

Free of charge with the input & output generation

Support with Google Search up to 500 RPD- Request Per Day

Estimated Cost: (Not Free-Tier)

\$0.10 for input & \$0.40 output Token

Limit: 2000RPM (rate per minute) & with 15 RPM & 1500 req/day

API Pricing per 1M tokens = 1\$

Service Needed to: Generate Analysis & Synthesis data

Estimated Monthly Cost: 0\$ for Free-tier & TBD With Paid-Tier

• **Other Services - including Deployment Service as:**

DockerCloudBuild,

Github Automation,

Kubernetes

Estimated Cost: TBD

Service Needed:

Docker for containerization

GitHub Actions for CI/CD automation

Network System for secure data transmission and communication

Kubernetes for Load Balancing (Google Kubernetes offers free-tier with: \$74.40 in monthly credits per billing account applied to zonal and Autopilot clusters.)

Estimated Monthly Cost: \$TBD

Potential Challenges:

- Limited API requests for gathering information
- Depend on another service
- Backups → Not available
- Session Handling???

2. Data Security & Privacy: Protecting sensitive data (e.g., user information, transaction data).

- Data Types & Sensitivity
 - Collected: User information (name, mail, phone number,)
 - Access permission: Employee
 - Backups → Not available
 - Secure Storage (using third parties)
- Data Encryption
 - Using Hashi-Corp as the Keys Vault
 - Do Offer Free-Option with K8s for Self-Management Vault
 - Pros: Economical & Self-Control
 - Cons: Risk of Data Leak or Loss & Adding complications in system design
 - Paid-Tier with Hashi-Corp management
 - Pros: Easier Deployment Platform & safer in data & secret storing & simpler deployment pipeline
 - Cons: Expensive if misused or quota expected
 - Development Tier: 0.62\$ / K8s Cluster for 1 hours
 - Standard/ Plus Tier: 1.58\$ / K8s Cluster for 1 hours
 - With 112\$ monthly cost/ Vault Client

3. Real-Time Performance: Delivering real-time responses, particularly in AI/ML applications.

Solution: Using the SageMaker in combination with Google AI Studio API

For the price estimation please check the session as we mentioned:

AWS SageMaker Real-Time Inference

Viettel Object Cloud Storage (for storing the model file in S3-bucket)

Generative Model API: Google AI Studio API

- Challenges can occur:
 - The number of requests that the software can handle is unknown
 - Worst case of sessions and processes handling (the software only has a limited amount of API requests for searching)
 - Conversation speed between software and dependencies is unknown
 - Availability: needs to be always
 - It needs a lot of testing before published

4. Project Robustness (capability to handle error):

- No backup plan for data loss
- Not having sufficient budget and human resources to scale the project
- Suggest deploying the system on the Docker Container managed by Kubernetes to balance the traffic to avoid the overload – but with strict price management)

4. Organizational & Managerial Feasibility

Team Members & Roles: (List key roles and responsibilities.)

- Project Manager & Leader: Dat Tran Tien (P.M.L)
- Technical Development Team: (T.D.T)
 - Nguyen Kim Tuan Cuong (Analysis Module development)
 - Nguyen Hai An (Search Module Development)
 - Tran Tien Dat (Analysis Module & Search Module Development)
 - Nguyen Duy Duc (Security Module & User Data Management Development)
 - Nguyen Khanh Viet Dung (Web front-end Development)
- Feasibility Research Team: (F.R.T)
 - Nguyen Kim Tuan Cuong
 - Nguyen Khanh Viet Dung
- Architectural Designing Team: (A.D.T)
 - Nguyen Hai An
 - Tran Tien Dat (Leader in Architectural Design)
 - Nguyen Duy Duc
- Resource Management Team: (R.M.T)
 - Nguyen Hai An
 - Nguyen Duy Duc
- Requirement Development Team: (R.D.T)
 - Tran Tien Dat
 - Nguyen Kim Tuan Cuong (Requirement Verifier)
- Tester Team (T.T)
 - Nguyen Kim Tuan Cuong
 - Nguyen Khanh Viet Dung

- Deployment & Maintenance Team (D&M)
 - Nguyen Khanh Viet Dung (website hosting & Building the Web)
 - Nguyen Kim Tuan Cuong (Analysis Module development)
 - Nguyen Duy Duc
- User-Interface Development (U.I.D)
 - Nguyen Kim Tuan Cuong
 - Tran Tien Dat
 - Nguyen Khanh Viet Dung

5. Project Timeline

Development Timeline:

(THE PROJECT DEADLINE MUST BE BEFORE 17.May.2025)

- Prototype Development: 17.May.2025
 - Beta Testing: No Implementation
 - Official Launch: 26.May.2025 (Official Demonstration)
-

RACI Table for Team Members & Roles

The RACI table is the systematic representation that our team use to manage & communicate the role of the member in the project – this include 4 roles: Responsible ®, Accountable (A) , Consulted (C) and Informed (I)

This would a clear and rigourous as the credit for the team contribution and show as the table in the next page

Task/Role	P.M.L	T.D.T	F.R.T	A.D.T	R.D.T	T.T	R.M.T	D&M	U.I.D
Project Planning	R, A	C	I	I	I	I	I	I	I
Technical Research	I	R, A	C	C	I	I	R	I	I
Feasibility Analysis	I	C	R, A	C	I	I	I	I	C
Architecture Design	I	C	I	R, A	C	I	I	C	I
Requirements Gathering	I	C	I	C	R, A	I	I	I	I
System Development	I	R, A	I	C	C	I	R	C	I
Quality Assurance	C	R	I	C	C	R	I	C	A
Deployment Planning	A	R	I	C	C	I	R	R, A	I
Final Review & Testing	A	R	I	I	C	R, A	I	I	R

Documentation	R	C	I	C	A	I	I,R	I	R
---------------	---	---	---	---	---	---	-----	---	---

RACI Annotation:

- **R (Responsible):** Responsible for the outcome & does the work.
- **A (Accountable):** Approves and oversees work.
- **C (Consulted):** Provides input or advice.
- **I (Informed):** Receives updates and information.